Week 1 exercises

Week 1 - Programming in Haskell

In this week we will look at the basics of Haskell programming, including type classes. While we do have to spend some time on the minutiae of syntax and basic definitions, our true goal is to start on acquiring an understanding of how to structure programs along abstract and principled lines. Type classes are the most explicit example of this idea.

Slides and material

- Tuesday

Suggested Reading

- Part 1 of *Programming in Haskell*. Most of this is standard functional programming material, so feel free to skim those parts. The most interesting chapter is chapter 8.
- Course Notes Chapter 1

Going Beyond

- Real World Haskell (a bit dated, however).

Exercises

The mandatory assignment builds directly on the concepts *and code* developed during the exercises. You are strongly advised to complete the exercises before embarking on the assignment. It may also be a useful exercise to do other exercises (unrelated to the assignment), for example the ones in *Programming in Haskell*.

In these exercises you will be developing an interpreter for a very simple language of arithmetic expressions, which we call *APL* for *AP Language*.

Your starting point is found in handout/ and complete solutions are in solutions/. For some of the tasks, partial or complete solutions are also included inline below.

The starting point is a skeletal Haskell program consisting of the modules `APL.AST` (in `src/APL/AST.hs`), `APL.Eval` (in `src/APL/Eval.hs`) and `APL.Eval_Tests` (in `src/APL/Eval_Tests.hs`), as well as `week1.cabal` with build instructions. You should start by running `cabal test` to ensure that your Haskell setup is functional. This compiles everything and runs the (currently empty) test suite. When going through these exercises, run `cabal test` frequently to see whether your code still works. It is also a good idea to use `ghci` to interactively debug and test your work.

The `APL.AST` module will contain the definition of APL. Initially it is almost empty.

The `APL.Eval` module will import `APL.AST` and define an evaluation function for APL programs.

The `APL.Eval_Tests` module should import definitions from `APL.AST` and `APL.Eval` and then tests them. It also imports unit testing modules from `tasty`. Initially it imports almost nothing, so you will have to extend the `import` statements with the things you need.

The `runtests.hs` file is a very simple program that runs the test suite. You should not modify it.

Getting started

We will represent expressions by a Haskell datatype `Exp`. It will initially be tiny, and then we will extend it as we go along. In the `APL.AST` module, add the following definition:

```
data Exp
  = CstInt Integer
  deriving (Eq, Show)
```

The `CstInt` constructor represents an expression that is a constant integer, like `1`. The `Eq` instance is so we can compare expressions with `==`, and the `Show` is so we can print them to the screen (critical for debugging).

You should also export the `Exp` type, including its constructors, from the `APL.AST` module so it can be imported by other modules. Do this by adding `Exp(..)` to the module export list.

The result of *evaluating* an expression is a *value*, which we represent with the type `Val`. Add the following definition to the `APL.Eval` module:

```
data Val
  = ValInt Integer
  deriving (Eq, Show)
```

(The `Exp` and `Val` types are currently isomorphic, but will not remain so for long.)

We will now define an evaluation function that can transform an expression to a value. In `APL.Eval`, define a function `eval` with the following type:

```
eval :: Exp -> Val
```

Implement the `eval` function case for the `CstInt` case. A constant integer evaluates (unsurprisingly) to the corresponding integer value.

Open this to see the answer

```
eval (CstInt x) = ValInt x
```

Export the `eval` function from `APL.Eval`, and add a test to `Eval_Tests.hs` that tests whether your implementation of `CstInt` is correct. You will need to extend the `import` statements at the beginning of the module to include at least the following:

```
import APL.AST (Exp (..))
import APL.Eval (Val (..), eval)
import Test.Tasty.HUnit (testCase, (@?=))
```

Sabotage either your implementation of `eval` or the test itself such that the test fails, and ensure that this is detected by `cabal test`. Then fix it before continuing.

Implementing arithmetic

Add the following constructors to `Exp`:

```
data Exp
  = ...
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
```

These correspond to addition, subtraction, multiplication, division, and exponentiation. Implement the corresponding cases in `eval` and add appropriate tests to the test suite.

Examples of `Exp` The following examples show how various terms in (relatively) conventional mathematical notation are written as `Exp`.

Math	`Exp`
2+3	`Add (CstInt 2) (CstInt 3)`
2^3	`Pow (CstInt 2) (CstInt 3)`
2^3+4	`Pow (CstInt 2) (Add (CstInt 3) (CstInt 4))`
2+3*4	`Add (CstInt 2) (Mul (CstInt 3) (CstInt 4))`
(2+3)*4	`Mul (Add (CstInt 2) (CstInt 3)) (CstInt 4)`

Note that the parentheses that we use in mathematical notation are not present in `Exp` - this is because the structure of the Haskell value already precisely encodes which expressions are operands of which operators. In the nomenclature of programming languages, the `Exp` value is an *Abstract Syntax Tree* (AST), with unnecessary information excluded.

Hints

- The Haskell function for integer division is `div`. You can use it infix by enclosing it in backticks: `x `div` y`.
- The Haskell function for integer exponentiation is `^`.

Consider What happens with division by zero or using a negative exponent? Can you test for this?

Open this to see the answer

The Haskell functions `div` and `^` are *partial*, meaning they can crash with a so-called *imprecise exception* (an exception that is not visible in the type, also sometimes called *IO exceptions*). It is possible to catch and handle such exceptions, but not with the parts of Haskell you have seen so far, so it is not expected that you have tests for these cases - *yet*.

Representing Failure

Now we will extend the interpreter to make it clear in the type of `eval` that some operations can fail. For now, the only failure cases are division by zero and taking a negative exponent.

Add a type synonym for error messages:

```
type Error = String
```

the type `Error` is completely interchangeable with `String`, but is named to make its intended use in function types explicit. In a more sophisticated system, we might use a more complicated `Error` type with various constructors for indicating different kinds of errors. For now, we will treat it as a human-readable string.

Now change the type of `eval` to be the following:

```
eval :: Exp -> Either Error Val
```

The `Either` type is often used to express computations that can fail. It is defined as follows (it is already defined in the implicit Haskell prelude, so you should *not* add it to `APL.AST`):

```
data Either a b = Left a
                | Right b
```

By convention, the `Left` constructor is used to represent failure, and the `Right` constructor to represent success. It is documented at Data.Either.

You must now modify the definition of `eval` to wrap the return value in either `Left` or `Right` depending on whether evaluation succeeds. Evaluation of an expression succeeds if its subexpressions succeed, and the expression itself does not fail (by dividing with zero or taking a negative exponent).

You must also modify your tests. Make sure to add tests for the failure cases.

Hints It is a good idea to start by commenting out all your existing code so you do not get overwhelmed with type errors.

As a starting point, here is the clause for `CstInt`:

```
eval (CstInt x) = Right $ ValInt x
```

When handling compound operations such as `Add`, use `case` to determine whether the subexpressions succeeded.

Your implementations of `Add`, `Sub`, `Mul`, and `Div` will be quite similar. Consider constructing a helper function to reduce the duplication of logic.

Solution (partial) Open this to see the answer

```
eval (Add e1 e2) =
  case (eval e1, eval e2) of
    (Left err, _) -> Left err
    (_, Left err) -> Left err
    (Right (ValInt x), Right (ValInt y)) -> Right $ ValInt $ x + y
```

Add conditionals

To get slightly closer to a programming language, we will now add boolean values and control flow.

Add the following constructors to `Exp`:

```
data Exp
  = ...
  | CstBool Bool
  | Eql Exp Exp
  | If Exp Exp Exp
```

Add the following constructor to `Val`:

```
data Val
  = ...
  | ValBool Bool
```

Your task is to implement the corresponding cases in `eval`.

- A `CstBool` should evaluate to a `ValBool`, similar to `CstInt`.
- A `Eql` should return `ValBool True` if the operands evaluate to the same value, and `ValBool False` otherwise.
- `If` should evaluate the first expression. If true, it should then evaluate the second expression, and otherwise the third expression.

Whenever we add new features, we also add potential for new bugs. We have added the following new ways for evaluation to fail:

- Comparing an integer and a boolean with `Eql` (mixing types is not allowed).
- The first expression in an `If` (the condition) evaluates to an integer.

These should result in a `Left` value, similar to division by zero.

Consider Would it be an acceptable implementation for `If` to start by evaluating all three subexpressions (including propagating errors), similar to how we implement `Add`? If not, make sure to write a test that would fail if that is how `If` was implemented.

Solution Open this to see the answer

```
eval (Eql e1 e2) =
  case (eval e1, eval e2) of
    (Left err, _) -> Left err
    (_, Left err) -> Left err
    (Right (ValInt x), Right (ValInt y)) -> Right $ ValBool $ x == y
    (Right (ValBool x), Right (ValBool y)) -> Right $ ValBool $ x == y
    (Right _, Right _) -> Left "Invalid operands to equality"
eval (If cond e1 e2) =
  case eval cond of
    Left err -> Left err
    Right (ValBool True) -> eval e1
    Right (ValBool False) -> eval e2
    Right _ -> Left "Non-boolean conditional."
```

Adding Variables

We will now add variable bindings, through which an expression can be bound to a name and used multiple times. We will assume you are already familiar with the concept of variables.

Add the following type synonym:

```
type VName = String
```

Add the following constructors to `Exp`:

```
data Exp
  = ...
  | Var VName
  | Let VName Exp Exp
  deriving (Eq, Show)
```

`Var` is a reference to a variable, and `Let` binds a name to the result of evaluating the first expression while evaluating the second expression.

Environments To implement variables, we need to maintain *environments*. An environment maps a name to a value. Evaluation of an expression takes place in an environment. We will represent environments with this `Env` type, which you must add to `APL.Eval`:

```
type Env = [(VName, Val)]
```

An environment consists of a list of pairs, which each pair comprising a variable and its associated value. We call this a *variable binding*. For operating on environments we will make use of definitions with the following types:

```
-- | Empty environment, which contains no variable bindings.
envEmpty :: Env

-- | Extend an environment with a new variable binding,
-- producing a new environment.
envExtend :: VName -> Val -> Env -> Env

-- | Look up a variable name in the provided environment.
-- Returns Nothing if the variable is not in the environment.
envLookup :: VName -> Env -> Maybe Val
```

Finish the definition of `envEmpty`, `envExtend`, and `envLookup`. You may find the standard `lookup` function useful, but you can also write it explicitly as a recursive function.

Open this to see the answer

```
envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env
```

Evaluating in an Environment We are ready to extend `eval` itself with environments. As a starting point, change the type of `eval` to the following:

```
eval :: Env -> Exp -> Either Error Val
```

Update your existing code such that it merely propagates this new parameter to recursive `eval` applications. For the "top level" application of `eval` (for example in your tests), pass `envEmpty`.

Now add `eval` clauses for `Var` and `Let`.

For `Var`, use `envLookup` to perform a lookup in the environment. If the variable is not in the environment, then that should be reported as a failure.

For `Let`, evaluate the first expression. If that fails, evaluation of the entire expression also fails. Otherwise, use `envExtend` to add a binding of the variable to the environment, and use that extended environment when evaluating the second expression.

Examples

```
> eval envEmpty (Let "x" (CstInt 3) (Add (Var "x") (Var "x")))
Right (ValInt 6)
> eval envEmpty (Let "x" (CstInt 3) (Add (Var "x") (Var "y")))
Left "Unknown variable: y"
```

Solution Open this to see the answer

```
eval env (Var v) = case envLookup v env of
  Just x -> Right x
  Nothing -> Left $ "Unknown variable: " ++ v
eval env (Let var e1 e2) =
  case eval env e1 of
    Left err -> Left err
    Right v -> eval (envExtend var v env) e2
```

Week 1 solutions

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  deriving (Eq, Show)
```

APL/Eval.hs

```
module APL.Eval
  ( Val (..),
    Env,
    envEmpty,
    eval,
  )
where

import APL.AST (Exp (..), VName)

data Val
  = ValInt Integer
  | ValBool Bool
  deriving (Eq, Show)

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

type Error = String

evalIntBinOp :: (Integer -> Integer -> Either Error Integer) -> Env -> Exp -> Exp -> Either Error Val
evalIntBinOp f env e1 e2 =
  case (eval env e1, eval env e2) of
    (Left err, _) -> Left err
    (_, Left err) -> Left err
    (Right (ValInt x), Right (ValInt y)) -> case f x y of
      Left err -> Left err
      Right z -> Right $ ValInt z
    (Right _, Right _) -> Left "Non-integer operand"

evalIntBinOp' :: (Integer -> Integer -> Integer) -> Env -> Exp -> Exp -> Either Error Val
evalIntBinOp' f env e1 e2 =
  evalIntBinOp f' env e1 e2
  where
    f' x y = Right $ f x y

eval :: Env -> Exp -> Either Error Val
eval _env (CstInt x) = Right $ ValInt x
eval _env (CstBool b) = Right $ ValBool b
eval env (Var v) = case envLookup v env of
  Just x -> Right x
  Nothing -> Left $ "Unknown variable: " ++ v
eval env (Add e1 e2) = evalIntBinOp' (+) env e1 e2
eval env (Sub e1 e2) = evalIntBinOp' (-) env e1 e2
eval env (Mul e1 e2) = evalIntBinOp' (*) env e1 e2
eval env (Div e1 e2) = evalIntBinOp checkedDiv env e1 e2
  where
    checkedDiv _ 0 = Left "Division by zero"
    checkedDiv x y = Right $ x `div` y
eval env (Pow e1 e2) = evalIntBinOp checkedPow env e1 e2
  where
    checkedPow x y =
      if y < 0
        then Left "Negative exponent"
        else Right $ x ^ y
eval env (Eql e1 e2) =
  case (eval env e1, eval env e2) of
    (Left err, _) -> Left err
```

```
    (_, Left err) -> Left err
    (Right (ValInt x), Right (ValInt y)) -> Right $ ValBool $ x == y
    (Right (ValBool x), Right (ValBool y)) -> Right $ ValBool $ x == y
    (Right _, Right _) -> Left "Invalid operands to equality"
eval env (If cond e1 e2) =
  case eval env cond of
    Left err -> Left err
    Right (ValBool True) -> eval env e1
    Right (ValBool False) -> eval env e2
    Right _ -> Left "Non-boolean conditional."
eval env (Let var e1 e2) =
  case eval env e1 of
    Left err -> Left err
    Right v -> eval (envExtend var v env) e2
```

APL/Eval_Tests.hs

```
module APL.Eval_Tests (tests) where

import APL.AST (Exp (..))
import APL.Eval (Val (..), envEmpty, eval)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (testCase, (@?=))

tests :: TestTree
tests =
  testGroup
    "EValuation"
    [ testCase "Add" $
        eval envEmpty (Add (CstInt 2) (CstInt 5))
          @?= Right (ValInt 7),
      --
      testCase "Add (wrong type)" $
        eval envEmpty (Add (CstInt 2) (CstBool True))
          @?= Left "Non-integer operand",
      --
      testCase "Sub" $
        eval envEmpty (Sub (CstInt 2) (CstInt 5))
          @?= Right (ValInt (-3)),
      --
      testCase "Div" $
        eval envEmpty (Div (CstInt 7) (CstInt 3))
          @?= Right (ValInt 2),
      --
      testCase "Div0" $
        eval envEmpty (Div (CstInt 7) (CstInt 0))
          @?= Left "Division by zero",
      --
      testCase "Pow" $
        eval envEmpty (Pow (CstInt 2) (CstInt 3))
          @?= Right (ValInt 8),
      --
      testCase "Pow0" $
        eval envEmpty (Pow (CstInt 2) (CstInt 0))
          @?= Right (ValInt 1),
      --
      testCase "Pow negative" $
        eval envEmpty (Pow (CstInt 2) (CstInt (-1)))
          @?= Left "Negative exponent",
      --
      testCase "Eql (false)" $
        eval envEmpty (Eql (CstInt 2) (CstInt 3))
          @?= Right (ValBool False),
      --
      testCase "Eql (true)" $
        eval envEmpty (Eql (CstInt 2) (CstInt 2))
          @?= Right (ValBool True),
      --
      testCase "If" $
        eval envEmpty (If (CstBool True) (CstInt 2) (Div (CstInt 7) (CstInt 0)))
          @?= Right (ValInt 2),
      --
      testCase "Let" $
        eval envEmpty (Let "x" (Add (CstInt 2) (CstInt 3)) (Var "x"))
          @?= Right (ValInt 5),
      --
      testCase "Let (shadowing)" $
        eval
          envEmpty
          ( Let
              "x"
              (Add (CstInt 2) (CstInt 3))
              (Let "x" (CstBool True) (Var "x"))
```

```
67            )
68            @?= Right (ValBool True)
69      ]
```

Advanced Programming 2026

Assignment One

Interpreting Arithmetic Expressions

Due: 2026-9-13

Synopsis: Implementing an interpreter for a small language of arithmetic expressions.

1 Introduction

In this assignment you will be adding various new features to the language implemented during the exercise classes. If you wish, you can base the assignment on your own solutions to the exercises, rather than ours, but only do so if you are quite sure that they are correct. Specifically, you will be implementing the following:

- Support for `for`-loops in the interpreter.
- Support for functions in the interpreter.
- Support for `try-catch` in the interpreter.
- A pretty-printer for printing expressions in conventional (infix) notation.

For all tasks you are expected to add appropriate tests to the files `AST_Tests.hs` and `Eval_Tests.hs`. Do not rename any of the definitions already present in the handout. Do not change the types of any exported functions.

Most of the tasks are intended to test your ability to extend a Haskell datatype and corresponding recursive functions.

2 Tasks

Task 1: Loops

For this task you must augment APL with `for` loops. A `for` loop is represented by the following constructor:

```
data Exp
  = ...
  | ForLoop (VName, Exp) (VName, Exp) Exp
```

An expression

```
ForLoop (p,initial) (i,bound) body
```

is evaluated as follows:

1. Evaluate expression `initial` to a value v.
2. Evaluate expression `bound` to an integer n. It is an error `"Non-integral loop bound"` if `bound` evaluates to a non-integer.
3. Bind variable `i` to the integer 0.
4. Bind variable `p` to v.
5. While `i` $< n$: evaluate expression `body` and bind `p` to the result. Increment `i`.
6. Return the final value of `p`.

Essentially, `for`-loops evaluate the given `body` expression a number of times given by `bound`, each time binding the loop parameter `p` to the result of the body.

Your task: Add the `ForLoop` constructor to `Exp`, implement evaluation of `ForLoop` in `APL.Eval`, add appropriate tests in `APL.Eval_Tests`.

Examples

```
> eval envEmpty (ForLoop ("p", CstInt 0) ("i", CstInt 10)
                         (Add (Var "p") (Var "i")))
Right (ValInt 45)
```

Task 2: Functions

In this task you will implement function expressions, function application, and function values in APL. A lambda expression and function application is represented by the following `Exp` constructors:

```
data Exp
  = ...
  | Lambda VName Exp
  | Apply Exp Exp
```

A function value is represented by the following `Value` constructor:

```
data Val
  = ...
  | ValFun Env VName Exp
```

A `Lambda` comprises a parameter name and a body expression. When evaluated, `Lambda` produces a function value, represented by `ValFun`. Apart from storing the parameter and body, a value function also *captures* the environment at the point at which it was constructed.

An `Apply` applies a function expression to an argument expression. The function expression must evaluate to a `ValFun`. The argument expression can evaluate to an argument value of any type. Application then happens by starting from the environment stored in the `ValFun`, extending it with a binding of the parameter name to the argument value, then using this environment to evaluate the body. The result of evaluating the body is the result of the application. The order of evaluation of the `Apply` subexpressions is not specified.

Your task: Add the necessary constructors to `Exp` and `Val`, implement the appropriate cases in `APL.Eval`, add tests in `APL.Eval_Tests`.

Examples

```
> eval [] (Let "x" (CstInt 2)
                   (Lambda "y" (Add (Var "x") (Var "y"))))
Right (ValFun [("x",ValInt 2)] "y"
              (Add (Var "x") (Var "y")))
> eval [] (Apply (Let "x" (CstInt 2)
```

```
                  (Lambda "y" (Add (Var "x") (Var "y"))))
                 (CstInt 3))
Right (ValInt 5)
```

Task 3: `try-catch`

In this task you are implementing a mechanism for error handling. We will

add the following constructor to `Exp`:

```
data Exp
  = ...
  | TryCatch Exp Exp
```

When evaluating an expression `TryCatch e1 e2`, we start by evaluating `e1`. If `e1` evaluates successfully, then that is the result of the `TryCatch` expression. If `e1` encounters a failure (`Left`), then we evaluate `e2`, and its result is the overall result of evaluation.

Your task: Add the necessary constructors to `Exp` and `Val`, implement `TryCatch` in `APL.Eval`, add tests in `APL.Eval_Tests`.

Examples

```
> eval [] (TryCatch (CstInt 0) (CstInt 1))
Right (ValInt 0)
> eval [] (TryCatch (Var "missing") (CstInt 1))
Right (ValInt 1)
```

Task 4: Pretty-printer

This task tests your ability to write recursive functions from scratch. The function

```
printExp :: Exp -> String
```

produces a representation of the provided expression in Haskell-like notation, including infix notation for binary operators.

Specifically:

- The binary operators are printed as `x + y`, replacing + with the appropriate operator. Exponentiation is written with the ** operator.
- An `If` expression is printed as `if x then y else z`.
- A `Let` expression is printed as `let x = y in z`.
- A `ForLoop (p,initial) (i,bound) body` expression is printed as `loop p = initial for i < bound do body`.
- A `Lambda` expression is printed as `\x -> y`.
- A `Apply` expression is printed as `x y`.
- A `TryCatch` expression is printed as `try x catch y`.
- A `CstInt` is printed as a decimal integer (use `show`).
- A `CstBool` is printed as `true` or `false` (note: lowercase).

Make sure to add parentheses as appropriate. It is acceptable to introduce more parentheses than strictly necessary. In fact, we recommend that you enclose every expression in parentheses, as this is much easier to implement. Parentheses *must* be inserted in the following cases:

- Any argument in an `Apply` must be parenthesized unless that argument is a constant or variable.
- The function part of an `Apply` must be parenthesized unless it is a constant, variable, or another `Apply`.

Your task: Implement the function `printExp` in `APL.AST`, add appropriate tests in `APL.AST_Tests`.

3 Code handout

The code handout consists of the following nontrivial files.

- `a1.cabal`: Cabal build file. **Do not modify this file.**
- `runtests.hs`: Test execution program. **Do not modify this file.**

- `src/APL/AST.hs`: The APL AST definition.
- `src/APL/AST_Tests.hs`: Placeholder for AST tests.
- `src/APL/Eval.hs`: Evaluation functions for APL.
- `src/APL/Eval_Tests.hs`: Evaluation tests. Some tests are already included. You are expected to add more.

4 Your Report

You are expected to comment on the *interesting* details of your implementation. You are *not* expected to give a line-by-line walkthrough of your code. Most importantly, you are expected to reflect on the *quality* of your code:

- Do you think it is functionally correct? Why or why not?
- Is there some improvement you'd have liked to make, but didn't have the time?

It is more important to be aware of the strengths or shortcomings of your solution, than it is to have a complete solution.

4.1 The structure of your report

Your report must be structured exactly as follows:

Introduction: Briefly mention general concerns, any ambiguities in the problem text and how you resolved them, and your own estimation of the quality of your solution. Briefly mention whether your solution is functional, which test cases cover its functionality, which test cases it fails for (if any), and what you think might be wrong.

A section answering the following numbered questions:

1. What is the observable evaluation order for `Apply`, what difference does it make, and which of your test(s) demonstrate this?

2. Would it be a correct implementation of `eval` for `TryCatch` to *first* call `eval` on *both* subexpressions? Why or why not? If not, under which assumptions might it be a correct implementation?
3. Is it possible for your implementation of `eval` to go into an infinite loop for some inputs?
4. For each of the following equalities, explain under which circumstances they hold.

   a) `Apply (Lambda "v" x) y` = `x`

   b) `TryCatch (TryCatch x y) z` = `TryCatch x y`

   Note that these are not Haskell equalities, but mathematical equalities.
5. Suppose we wanted APL to be dynamically scoped. What would need to change in your implementation?
6. What is the asymptotic complexity of `printExp` on a left-nested expression such as `Add (Add (Add ... ) ...) ...`?
7. State properties that it would be natural for `printExp` to satisfy (you are encouraged to hypothesize additional machinery in order to state such properties).
8. Give a reason for why `Val` is defined in `APL.Eval` rather than in `APL.AST`.

All else being equal, **a short report is a good report**.

5 Deliverables for This Assignment

You must submit the following items:

- A single PDF file, A4 size, no more than 5 pages, structured as specified in the report section above.
- A single zip/tar.gz file with all code relevant to the implementation, including at least all the files from the handout. For this assignment it is not necessary to add additional files.

Remember to follow the general assignment rules listed on the course homepage.

6 Assessment

You will get written qualitative feedback, and points from zero to four. There are no resubmissions, so please hand in what you managed to do, even if you have not solved the assignment completely.

A1 handout

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
    printExp,
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  -- TODO: add cases
  deriving (Eq, Show)

printExp :: Exp -> String
printExp = undefined -- TODO
```

APL/AST_Tests.hs

```
module APL.AST_Tests (tests) where

import APL.AST (Exp (..))
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (testCase, (@?=))

tests :: TestTree
tests =
  testGroup
    "Prettyprinting"
    []
```

APL/Eval.hs

```
module APL.Eval
  ( Val (..),
    Env,
    envEmpty,
    eval,
  )
where

import APL.AST (Exp (..), VName)

data Val
  = ValInt Integer
  | ValBool Bool
  deriving (Eq, Show)

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

type Error = String

evalIntBinOp :: (Integer -> Integer -> Either Error Integer) -> Env -> Exp -> Exp -> Either Error Val
evalIntBinOp f env e1 e2 =
  case (eval env e1, eval env e2) of
    (Left err, _) -> Left err
    (_, Left err) -> Left err
    (Right (ValInt x), Right (ValInt y)) -> case f x y of
      Left err -> Left err
      Right z -> Right $ ValInt z
    (Right _, Right _) -> Left "Non-integer operand"

evalIntBinOp' :: (Integer -> Integer -> Integer) -> Env -> Exp -> Exp -> Either Error Val
evalIntBinOp' f env e1 e2 =
  evalIntBinOp f' env e1 e2
  where
    f' x y = Right $ f x y

eval :: Env -> Exp -> Either Error Val
eval _env (CstInt x) = Right $ ValInt x
eval _env (CstBool b) = Right $ ValBool b
eval env (Var v) = case envLookup v env of
  Just x -> Right x
  Nothing -> Left $ "Unknown variable: " ++ v
eval env (Add e1 e2) = evalIntBinOp' (+) env e1 e2
eval env (Sub e1 e2) = evalIntBinOp' (-) env e1 e2
eval env (Mul e1 e2) = evalIntBinOp' (*) env e1 e2
eval env (Div e1 e2) = evalIntBinOp checkedDiv env e1 e2
  where
    checkedDiv _ 0 = Left "Division by zero"
    checkedDiv x y = Right $ x `div` y
eval env (Pow e1 e2) = evalIntBinOp checkedPow env e1 e2
  where
    checkedPow x y =
      if y < 0
        then Left "Negative exponent"
        else Right $ x ^ y
eval env (Eql e1 e2) =
  case (eval env e1, eval env e2) of
    (Left err, _) -> Left err
```

```
67      (_, Left err) -> Left err
68      (Right (ValInt x), Right (ValInt y)) -> Right $ ValBool $ x == y
69      (Right (ValBool x), Right (ValBool y)) -> Right $ ValBool $ x == y
70      (Right _, Right _) -> Left "Invalid operands to equality"
71  eval env (If cond e1 e2) =
72    case eval env cond of
73      Left err -> Left err
74      Right (ValBool True) -> eval env e1
75      Right (ValBool False) -> eval env e2
76      Right _ -> Left "Non-boolean conditional."
77  eval env (Let var e1 e2) =
78    case eval env e1 of
79      Left err -> Left err
80      Right v -> eval (envExtend var v env) e2
81
82  -- TODO: Add cases after extending Exp.
```

APL/Eval_Tests.hs

```
module APL.Eval_Tests (tests) where

import APL.AST (Exp (..))
import APL.Eval (Val (..), envEmpty, eval)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (testCase, (@?=))

-- -- Consider this example when you have added the necessary constructors.
-- -- The Y combinator in a form suitable for strict evaluation.
-- yComb :: Exp
-- yComb =
--   Lambda "f" $
--     Apply
--       (Lambda "g" (Apply (Var "g") (Var "g")))
--       ( Lambda
--           "g"
--           ( Apply
--               (Var "f")
--               (Lambda "a" (Apply (Apply (Var "g") (Var "g")) (Var "a")))
--           )
--       )

-- fact :: Exp
-- fact =
--   Apply yComb $
--     Lambda "rec" $
--       Lambda "n" $
--         If
--           (Eql (Var "n") (CstInt 0))
--           (CstInt 1)
--           (Mul (Var "n") (Apply (Var "rec") (Sub (Var "n") (CstInt 1))))

tests :: TestTree
tests =
  testGroup
    "Evaluation"
    [ testCase "Add" $
        eval envEmpty (Add (CstInt 2) (CstInt 5))
          @?= Right (ValInt 7),
      --
      testCase "Add (wrong type)" $
        eval envEmpty (Add (CstInt 2) (CstBool True))
          @?= Left "Non-integer operand",
      --
      testCase "Sub" $
        eval envEmpty (Sub (CstInt 2) (CstInt 5))
          @?= Right (ValInt (-3)),
      --
      testCase "Div" $
        eval envEmpty (Div (CstInt 7) (CstInt 3))
          @?= Right (ValInt 2),
      --
      testCase "Div0" $
        eval envEmpty (Div (CstInt 7) (CstInt 0))
          @?= Left "Division by zero",
      --
      testCase "Pow" $
        eval envEmpty (Pow (CstInt 2) (CstInt 3))
          @?= Right (ValInt 8),
      --
      testCase "Pow0" $
        eval envEmpty (Pow (CstInt 2) (CstInt 0))
          @?= Right (ValInt 1),
      --
      testCase "Pow negative" $
        eval envEmpty (Pow (CstInt 2) (CstInt (-1)))
```

```
67            @?= Left "Negative exponent",
68        --
69        testCase "Eql (false)" $
70          eval envEmpty (Eql (CstInt 2) (CstInt 3))
71            @?= Right (ValBool False),
72        --
73        testCase "Eql (true)" $
74          eval envEmpty (Eql (CstInt 2) (CstInt 2))
75            @?= Right (ValBool True),
76        --
77        testCase "If" $
78          eval envEmpty (If (CstBool True) (CstInt 2) (Div (CstInt 7) (CstInt 0)))
79            @?= Right (ValInt 2),
80        --
81        testCase "Let" $
82          eval envEmpty (Let "x" (Add (CstInt 2) (CstInt 3)) (Var "x"))
83            @?= Right (ValInt 5),
84        --
85        testCase "Let (shadowing)" $
86          eval
87            envEmpty
88            ( Let
89                "x"
90                (Add (CstInt 2) (CstInt 3))
91                (Let "x" (CstBool True) (Var "x"))
92            )
93            @?= Right (ValBool True)
94            --
95            -- TODO - add more
96      ]
```

Week 2 exercises

Week 2 - Monads

This week we will begin studying monadic programming in Haskell. This topic is foundational to the remainder of the course.

Suggested Reading

- Philip Wadler's The Essence of Functional Programming.
- Chapter 12 in *Programming in Haskell*. Chapter 10 may also be of interest.
- All About Monads
- Course Notes Chapter 2

Going Beyond

- Finding Success in Haskell

Exercises

In these exercises you will construct an interpreter that is similar in functionality to the one developed in week 1. However, this time we will implement the interpreter via an evaluation monad that abstracts away book-keeping of environments and failures. The second mandatory assignment will ask you to extend this monadic interpreter.

Your starting point is found in handout/ and complete solutions are in solutions/. For some of the tasks, partial or complete solutions are also included inline below. In terms of modules, the structure is identical to week 1.

Getting Started

The `APL.AST` and `APL.Eval` modules already contains various definitions that should be familiar to you from week 1. `APL.Eval` also contains some stub definitions for you to fill out.

To start with, we will define `EvalM` such that it tracks evaluation failure:

1. Change the definition of `EvalM a` such that it can represent two cases: either a value of type `a`, or an error of type `Error`.
2. Implement `Functor`, `Applicative`, and `Monad` instances for `EvalM`.
3. Implement `runEval`.
4. Implement a function `failure` that is used for signalling failure in the `EvalM` monad:

```
failure :: String -> EvalM a
```

Hints This is completely equivalent to the `Either` monad, but you should *not* just say `type EvalM = Either`. It must still be a `newtype`.

When implementing the `>>=` method, you should start by checking whether the left-hand side represents a failure, and if so, return that failure.

Solution Open this to see the answer

```
newtype EvalM a = EvalM (Either Error a)

instance Functor EvalM where
  fmap _ (EvalM (Left e))  = EvalM $ Left e
  fmap f (EvalM (Right x)) = EvalM $ Right $ f x

  -- Alternatively: fmap = liftM

instance Applicative EvalM where
  pure x = EvalM $ Right x
  EvalM (Left e)  <*> _               = EvalM (Left e)
  _               <*> EvalM (Left e)  = EvalM (Left e)
  EvalM (Right f) <*> EvalM (Right x) = EvalM (Right (f x))
```

```
  -- Alternatively: (<*>) = ap

instance Monad EvalM where
  EvalM x >>= f = EvalM $ case x of
    Left err -> Left err
    Right x' ->
      let EvalM y = f x'
       in y

failure :: String -> EvalM a
failure s = EvalM $ Left s

runEval :: EvalM a -> Either Error a
runEval (EvalM x) = x
```

Monadic evaluation

Implement the `eval` function to make use of the `EvalM` monad. You should *not* directly construct (or destruct) `EvalM` values in the `eval` function; leave that to the `Monad` instance and the `failure` function.

Skip `TryCatch` for now; we will implement that in the following task.

When you are done, you can use `runEval (eval [] e)` to evaluate an expression `e`. Remember to add appropriate tests to `APL_Tests.hs`.

Hints The approach is largely the same as in week 1, where we define a function class for every language construct. However, to evaluate subexpressions, we have to use monadic binding, e.g:

```
eval env (Add e1 e2) = eval env e1 >>= \x ->
                       eval env e2 >>= \y -> ...
```

Or with `do`-notation:

```
eval env (Add e1 e2) = do
  x <- eval e1 env
  y <- eval e2 env
  ...
```

Once you have made it work, you should still consider writing a helper function to isolate the common behaviour for binary operators, similar to week 1.

Solution (partial) Open this to see the answer

```
eval :: Env -> Exp -> EvalM Val
eval _ (CstInt x) = pure $ ValInt x
eval _ (CstBool b) = pure $ ValBool b
eval env (Var v) = do
  case envLookup v env of
    Just x -> pure x
    Nothing -> failure $ "Unknown variable: " ++ v
eval env (Add e1 e2) = do
  x <- eval env e1
  y <- eval env e2
  case (x, y) of
    (ValInt x', ValInt y') -> pure $ ValInt $ x' + y'
    _ -> failure "Non-integer operand"
```

Implementing `TryCatch`

Now we will implement `TryCatch`. Start by defining the function `catch`:

```
catch :: EvalM a -> EvalM a -> EvalM a
```

An application `catch x y` should execute `x`. If `x` succeeds, then its result is the result of `catch`. If `x` fails, then `catch` should execute `y`, and the result of `y` is the result of `catch`. This means `y` is only executed if `x` fails.

`catch` should be considered part of the implementation of `EvalM`, and is allowed to directly construct and destruct `EvalM` values.

Once you have implemented `catch`, implement the `eval` case for `TryCatch` (which is very simple).

Hints `catch` will need to take apart both of its operands:

```
catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $
  ...
```

Solution Open this to see the answer

```
catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $
  case m1 of
    Left _ -> m2
    Right x -> Right x

eval env (TryCatch e1 e2) =
  eval env e1 `catch` eval env e2
```

Adding an environment to `EvalM`

Factoring error handling into `EvalM` itself has significantly reduced the amount of boilerplate code in `eval`, but there is still one piece left: the explicit passing of the environment when evaluating subexpressions. In this task you will extend `EvalM` to maintain an environment, similar to the `Reader` monad.

Start by changing the definition of `EvalM` such that it maintains environment of type `Env`, similar to the `Reader` monad. You will need to modify every bit of that directly uses the `EvalM` constructor, but you should not need to modify `eval`. Then using this new information, implement these two functions:

```
askEnv :: EvalM Env

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
```

`askEnv` retrieves the current environment, and `localEnv` executes an action in an environment that has been modified using the given function.

When using `runEval` to run an `EvalM` action, the environment is initially `envEmpty`.

Change the type of `eval` to be the following:

```
eval :: Exp -> EvalM Val
```

And update the implementation to make use of `askEnv` and `localEnv` instead of passing around the environment explicitly.

Hints To evaluate an expression `e` in an environment extended with a mapping of `var` to `val`: `localEnv (envExtend var val) (eval e)`.

To evaluate an expression `e` in a completely different environment: `localEnv (\_ -> env) (eval e)`.

Solution (partial) Open this to see the answer

```
newtype EvalM a = EvalM (Env -> Either Error a)

instance Functor EvalM where
  fmap f (EvalM x) =
    EvalM $ \env -> case x env of
      Right v -> Right $ f v
      Left err -> Left err

instance Applicative EvalM where
  pure x = EvalM $ \_env -> Right x
  EvalM ef <*> EvalM ex = EvalM $ \env ->
    case (ef env, ex env) of
      (Left err, _) -> Left err
      (_, Left err) -> Left err
      (Right f, Right x) -> Right (f x)

instance Monad EvalM where
  EvalM x >>= f = EvalM $ \env ->
    case x env of
      Left err -> Left err
```

```
      Right x' ->
        let EvalM y = f x'
         in y env

askEnv :: EvalM Env
askEnv = EvalM $ \env -> Right env

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f (EvalM m) = EvalM $ \env -> m (f env)

failure :: String -> EvalM a
failure s = EvalM $ \_env -> Left s

catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env ->
  case m1 env of
    Left _ -> m2 env
    Right x -> Right x

runEval :: EvalM a -> Either Error a
runEval (EvalM m) = m envEmpty

eval :: Exp -> EvalM Val
eval (CstInt x) = pure $ ValInt x
eval (CstBool b) = pure $ ValBool b
eval (Var v) = do
  env <- askEnv
  case envLookup v env of
    Just x -> pure x
    Nothing -> failure $ "Unknown variable: " ++ v
eval (Add e1 e2) = do
  x <- eval e1
  y <- eval e2
  case (x, y) of
    (ValInt x', ValInt y') -> pure $ ValInt $ x' + y'
    _ -> failure "Non-integer operand"
eval (Let var e1 e2) = do
  v1 <- eval e1
  localEnv (envExtend var v1) $ eval e2
eval (ForLoop (loopparam, initial) (iv, bound) body) = do
  initial_v <- eval initial
  bound_v <- eval bound
  case bound_v of
    ValInt bound_int ->
      loop 0 bound_int initial_v
    _ ->
      failure "Non-integral loop bound"
  where
    loop i bound_int loop_v
      | i >= bound_int = pure loop_v
      | otherwise = do
          loop_v' <-
            localEnv (envExtend iv (ValInt i) . envExtend loopparam loop_v) $
              eval body
          loop (succ i) bound_int loop_v'
eval (Lambda var body) = do
  env <- askEnv
  pure $ ValFun env var body
eval (Apply e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValFun f_env var body, arg) ->
      localEnv (const $ envExtend var arg f_env) $ eval body
    (_, _) ->
      failure "Cannot apply non-function"
```

Week 2 solutions

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  | ForLoop (VName, Exp) (VName, Exp) Exp
  | Lambda VName Exp
  | Apply Exp Exp
  | TryCatch Exp Exp
  deriving (Eq, Show)
```

APL/Eval.hs

```
module APL.Eval
  ( Val (..),
    eval,
    runEval,
    Error,
  )
where

import APL.AST (Exp (..), VName)
import Control.Monad (ap, liftM)

data Val
  = ValInt Integer
  | ValBool Bool
  | ValFun Env VName Exp
  deriving (Eq, Show)

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

type Error = String

newtype EvalM a = EvalM (Env -> Either Error a)

instance Functor EvalM where
  fmap = liftM

instance Applicative EvalM where
  pure x = EvalM $ \_env -> Right x
  (<*>) = ap

instance Monad EvalM where
  EvalM x >>= f = EvalM $ \env ->
    case x env of
      Left err -> Left err
      Right x' ->
        let EvalM y = f x'
         in y env

askEnv :: EvalM Env
askEnv = EvalM $ \env -> Right env

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f (EvalM m) = EvalM $ \env -> m (f env)

failure :: String -> EvalM a
failure s = EvalM $ \_env -> Left s

catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env ->
  case m1 env of
    Left _ -> m2 env
    Right x -> Right x

runEval :: EvalM a -> Either Error a
runEval (EvalM m) = m envEmpty

evalIntBinOp :: (Integer -> Integer -> EvalM Integer) -> Exp -> Exp -> EvalM Val
```

```
67  evalIntBinOp f e1 e2 = do
68    v1 <- eval e1
69    v2 <- eval e2
70    case (v1, v2) of
71      (ValInt x, ValInt y) -> ValInt <$> f x y
72      (_, _) -> failure "Non-integer operand"
73
74  evalIntBinOp' :: (Integer -> Integer -> Integer) -> Exp -> Exp -> EvalM Val
75  evalIntBinOp' f e1 e2 =
76    evalIntBinOp f' e1 e2
77    where
78      f' x y = pure $ f x y
79
80  eval :: Exp -> EvalM Val
81  eval (CstInt x) = pure $ ValInt x
82  eval (CstBool b) = pure $ ValBool b
83  eval (Var v) = do
84    env <- askEnv
85    case envLookup v env of
86      Just x -> pure x
87      Nothing -> failure $ "Unknown variable: " ++ v
88  eval (Add e1 e2) = evalIntBinOp' (+) e1 e2
89  eval (Sub e1 e2) = evalIntBinOp' (-) e1 e2
90  eval (Mul e1 e2) = evalIntBinOp' (*) e1 e2
91  eval (Div e1 e2) = evalIntBinOp checkedDiv e1 e2
92    where
93      checkedDiv _ 0 = failure "Division by zero"
94      checkedDiv x y = pure $ x `div` y
95  eval (Pow e1 e2) = evalIntBinOp checkedPow e1 e2
96    where
97      checkedPow x y =
98        if y < 0
99          then failure "Negative exponent"
100         else pure $ x ^ y
101 eval (Eql e1 e2) = do
102   v1 <- eval e1
103   v2 <- eval e2
104   case (v1, v2) of
105     (ValInt x, ValInt y) -> pure $ ValBool $ x == y
106     (ValBool x, ValBool y) -> pure $ ValBool $ x == y
107     (_, _) -> failure "Invalid operands to equality"
108 eval (If cond e1 e2) = do
109   cond' <- eval cond
110   case cond' of
111     ValBool True -> eval e1
112     ValBool False -> eval e2
113     _ -> failure "Non-boolean conditional."
114 eval (Let var e1 e2) = do
115   v1 <- eval e1
116   localEnv (envExtend var v1) $ eval e2
117 eval (ForLoop (loopparam, initial) (iv, bound) body) = do
118   initial_v <- eval initial
119   bound_v <- eval bound
120   case bound_v of
121     ValInt bound_int ->
122       loop 0 bound_int initial_v
123     _ ->
124       failure "Non-integral loop bound"
125   where
126     loop i bound_int loop_v
127       | i >= bound_int = pure loop_v
128       | otherwise = do
129           loop_v' <-
130             localEnv (envExtend iv (ValInt i) . envExtend loopparam loop_v) $
131               eval body
132           loop (succ i) bound_int loop_v'
133 eval (Lambda var body) = do
134   env <- askEnv
```

```
135   pure $ ValFun env var body
136 eval (Apply e1 e2) = do
137   v1 <- eval e1
138   v2 <- eval e2
139   case (v1, v2) of
140     (ValFun f_env var body, arg) ->
141       localEnv (const $ envExtend var arg f_env) $ eval body
142     (_, _) ->
143       failure "Cannot apply non-function"
144 eval (TryCatch e1 e2) =
145   eval e1 `catch` eval e2
```

APL/Eval_Tests.hs

```
module APL.Eval_Tests (tests) where

import APL.AST (Exp (..))
import APL.Eval (Error, Val (..), eval, runEval)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (testCase, (@?=))

eval' :: Exp -> Either Error Val
eval' = runEval . eval

tests :: TestTree
tests =
  testGroup
    "EValuation"
    [ testCase "Add" $
        eval' (Add (CstInt 2) (CstInt 5))
          @?= Right (ValInt 7),
      --
      testCase "Add (wrong type)" $
        eval' (Add (CstInt 2) (CstBool True))
          @?= Left "Non-integer operand",
      --
      testCase "Sub" $
        eval' (Sub (CstInt 2) (CstInt 5))
          @?= Right (ValInt (-3)),
      --
      testCase "Div" $
        eval' (Div (CstInt 7) (CstInt 3))
          @?= Right (ValInt 2),
      --
      testCase "Div0" $
        eval' (Div (CstInt 7) (CstInt 0))
          @?= Left "Division by zero",
      --
      testCase "Pow" $
        eval' (Pow (CstInt 2) (CstInt 3))
          @?= Right (ValInt 8),
      --
      testCase "Pow0" $
        eval' (Pow (CstInt 2) (CstInt 0))
          @?= Right (ValInt 1),
      --
      testCase "Pow negative" $
        eval' (Pow (CstInt 2) (CstInt (-1)))
          @?= Left "Negative exponent",
      --
      testCase "Eql (false)" $
        eval' (Eql (CstInt 2) (CstInt 3))
          @?= Right (ValBool False),
      --
      testCase "Eql (true)" $
        eval' (Eql (CstInt 2) (CstInt 2))
          @?= Right (ValBool True),
      --
      testCase "If" $
        eval' (If (CstBool True) (CstInt 2) (Div (CstInt 7) (CstInt 0)))
          @?= Right (ValInt 2),
      --
      testCase "Let" $
        eval' (Let "x" (Add (CstInt 2) (CstInt 3)) (Var "x"))
          @?= Right (ValInt 5),
      --
      testCase "Let (shadowing)" $
        eval'
          ( Let
              "x"
```

```
67                (Add (CstInt 2) (CstInt 3))
68                (Let "x" (CstBool True) (Var "x"))
69            )
70            @?= Right (ValBool True),
71        --
72        testCase "ForLoop" $
73          eval'
74            (ForLoop ("p", CstInt 0) ("i", CstInt 10) (Add (Var "p") (Var "i")))
75            @?= Right (ValInt 45),
76        --
77        testCase "Lambda/Apply" $
78          eval'
79            (Apply (Lambda "x" (Mul (Var "x") (Var "x"))) (CstInt 4))
80            @?= Right (ValInt 16),
81        --
82        testCase "TryCatch" $
83          eval'
84            (TryCatch (Div (CstInt 7) (CstInt 0)) (CstBool True))
85            @?= Right (ValBool True)
86      ]
```

Advanced Programming 2026

Assignment Two

Monadic Programming

Due: 2026-9-20

Synopsis: Implementing a monadic interpreter and type checker for a small language of arithmetic expressions.

1 Introduction

In this assignment you will be adding various new features to the language implemented during the exercise classes. You will also be implementing a very simple type checker. If you wish, you can base the assignment on your own solutions to the exercises, rather than ours, but only do so if you are quite sure that they are correct. Specifically, you will be implementing the following:

- Extending the interpreter with a notion of state.
- Support for printing values.
- Support for key-value stores.
- Implementing a type checker.

For all tasks you are expected to add appropriate tests to the file `Eval_Tests.hs`. Do not rename any of the definitions already present in the handout. Do not change the types of any functions.

2 Tasks

2.1 Task 1: Printing

In this task you will extend the `EvalM` monad with a simple notion of state. Specifically, we will add the ability to print values in an APL program.

Change the type of `runEval` to the following:

```
runEval :: EvalM a -> ([String], Either Error a)
```

The `[String]` that is returned will be the list of strings that were printed during execution of the program. We will not actually perform true IO during execution, so "printing" just means that the string ends up in this list. The list must be ordered such that the first string to be printed comes first in the list. You will also need to change the type of the `eval'` function in `APL.Eval_Tests` accordingly, as well as all the tests in that file.

Add the following constructor to `Exp`:

```
data Exp
  = ...
  | Print String Exp
```

`Print s e` evaluates `e` to a value v, then prints the string "`s:` $p(v)$", where $p(v)$ is the textual representation of v (see below). The expression then returns v.

- If v is an integer, then its textual representation is the integer in decimal notation. (Use `show`.)
- If v is a Boolean, then its textual representation is `True` and `False`. (Use `show`.)
- If v is a function, then its textual representation is `#<fun>`.

Examples

```
> runEval $ eval $ Print "foo" $ CstInt 2
(["foo: 2"],Right (ValInt 2))
> runEval $ eval $ Let "x" (Print "foo" $ CstInt 2)
                   (Print "bar" $ CstInt 3)
(["foo: 2","bar: 3"],Right (ValInt 3))
```

```
> runEval $ eval $ Let "x" (Print "foo" $ CstInt 2)
                           (Var "bar")
(["foo: 2"],Left "Unknown variable: bar")
```

Suggested implementation

Define a type `State` that can store a list of the strings printed. Modify `EvalM` to maintain such a `State`. **Hint:** Consider how the course notes discussed a combined reader-and-state monad. `EvalM` is a little more complicated as it also has to handle the notion of failure.

Define the following helper function:

```
evalPrint :: String -> EvalM ()
```

The function `evalPrint` adds a string to the list of printed strings. Use this to implement the `eval` case for `Print`. You will also need to modify `runEval`.

2.2 Task 2: Key-value store

In this task we will add a slightly more useful notion of effects, by extending APL to support a global key-value store, where a key (which can be any value) can be associated with any value, and later retrieved again.

Add the following constructors to `Exp`:

```
data Exp
  = ...
  | KvPut Exp Exp
  | KvGet Exp
```

- `KvPut k_exp v_exp` evaluates `k_exp` and `v_exp` to values k and v respectively. It then records the association $k \mapsto v$ in the store and returns v. If there is already an association for the value k, it is replaced by the new association.
- `KvGet k_exp` evaluates `k_exp` to a value k, then retrieves the value previously associated with k from the store. If there is no value associated with k, `KvGet` fails.

Examples

```
> runEval $ eval $ Let "x" (KvPut (CstInt 0) (CstBool True))
                           (KvGet (CstInt 0))
([],Right (ValBool True))
> runEval $ eval $ Let "x" (KvPut (CstInt 0) (CstBool True))
                           (KvGet (CstInt 1))
([],Left "Invalid key: ValInt 1")
> runEval $ eval $ Let "x" (KvPut (CstInt 0) (CstBool True))
                     (Let "y" (KvPut (CstInt 0) (CstBool False))
                      (KvGet (CstInt 0)))
([],Right (ValBool False))
```

Suggested implementation

Extend the `State` type to also contain a key-value mapping.

Define the following helper functions:

```
evalKvGet :: Val -> EvalM Val
evalKvPut :: Val -> Val -> EvalM ()
```

Use this to implement the `eval` cases. You will also need to modify the definition of `runEval`, but *not* its type.

2.3 Task 3: Type checking

In this task you will develop a type checker for APL. Since APL is dynamically typed, all we can check for is that the program does not contain references to variables that are not in scope. The purpose of this task is for you to demonstrate that you can construct a monad definition from scratch. Therefore, the code handout is somewhat spartan.

The module `APL.Check` exports a function

```
checkExp :: Exp -> Maybe Error
```

where `Error` is a synonym for `String`. This function must return `Just` (along with an error message) if the provided expression at any point references a variable that is not bound by an enclosing `Let` or `Lambda`. You must implement this function (and the helper functions it will likely invoke) by making use of a monad, similar to the evaluator. In particular,

you must take a monadic approach to error handling, and a monadic approach to tracking which variables are in scope.

Also add appropriate tests to `APL.Check_Tests`.

Examples

Note that the specific error message is up to you.

```
> checkExp $ CstInt 2
Nothing
> checkExp $ Var "x"
Just "Variable not in scope: x"
> checkExp $ Lambda "x" (Var "x")
Nothing
```

3 Code handout

The code handout consists of the following nontrivial files.

- `a2.cabal`: Cabal build file. **Do not modify this file.**
- `runtests.hs`: Test execution program. **Do not modify this file.**
- `src/APL/AST.hs`: The AST definition, which you extend as directed.
- `src/APL/Check.hs`: Where you will implement the type checker.
- `src/APL/Check_Tests.hs`: Tests for the type checker.
- `src/APL/Eval.hs`: The APL evaluation function.
- `src/APL/Eval_Tests.hs`: The evaluation tests. Some tests are already included. You are expected to add more.

4 Your Report

You are expected to comment on the *interesting* details of your implementation. You are *not* expected to give a line-by-line walkthrough of your code. Most importantly, you are expected to reflect on the *quality* of your code:

- Do you think it is functionally correct? Why or why not?
- Is there some improvement you'd have liked to make, but didn't have the time?

It is more important to be aware of the strengths or shortcomings of your solution, than it is to have a complete solution.

4.1 The structure of your report

Your report must be structured exactly as follows:

Introduction: Briefly mention general concerns, any ambiguities in the problem text and how you resolved them, and your own estimation of the quality of your solution. Briefly mention whether your solution is functional, which test cases cover its functionality, which test cases it fails for (if any), and what you think might be wrong.

A section answering the following numbered questions:

1. What is the asymptotic complexity of your implementation of `Print`? Can this be improved?
2. In your implementation, if a program keeps updating the same key with `KvPut`, does the memory usage of the program remain constant or does it increase?
3. In `TryCatch e1 e2`, if `e1` fails after performing some effects, are those effects visible in `e2`, and which of your tests demonstrate this? If you wanted different behaviour (either making the effects visible or invisible), what would you need to change in your implementation? Show exactly what the new code would look like.
4. Would it be possible to define type class instances

```
instance Monad m => Semigroup (m ()) where
  ...

instance Monad m => Monoid (m ()) where
  ...
```

and what might the definitions for `mempty` and <> look like? If so, argue based on the *monad* laws that your instances satisfy the *monoid* laws.

All else being equal, **a short report is a good report**.

5 Deliverables for This Assignment

You must submit the following items:

- A single PDF file, A4 size, no more than 5 pages, structured as specified in the report section above.
- A single zip/tar.gz file with all code relevant to the implementation, including at least all the files from the handout. For this assignment it is not necessary to add additional files.

Remember to follow the general assignment rules listed on the course homepage.

6 Assessment

You will get written qualitative feedback, and points from zero to four. There are no resubmissions, so please hand in what you managed to do, even if you have not solved the assignment completely.

A2 handout

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  | ForLoop (VName, Exp) (VName, Exp) Exp
  | Lambda VName Exp
  | Apply Exp Exp
  | TryCatch Exp Exp
  deriving (Eq, Show)
```

APL/Check.hs

```
module APL.Check (checkExp, Error) where

import APL.AST (Exp (..), VName)

type Error = String

newtype CheckM a = CheckM () -- TODO - give this a proper definition.

check :: Exp -> CheckM ()
check = undefined

checkExp :: Exp -> Maybe Error
checkExp = undefined
```

APL/Check_Tests.hs

```
module APL.Check_Tests (tests) where

import APL.AST (Exp (..))
import APL.Check (checkExp)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (assertFailure, testCase, (@?=))

-- Assert that the provided expression should pass the type checker.
testPos :: Exp -> TestTree
testPos e =
  testCase (show e) $
    checkExp e @?= Nothing

-- Assert that the provided expression should fail the type checker.
testNeg :: Exp -> TestTree
testNeg e =
  testCase (show e) $
    case checkExp e of
      Nothing -> assertFailure "expected error"
      Just _ -> pure ()

tests :: TestTree
tests =
  testGroup
    "Checking"
    []
```

APL/Eval.hs

```
module APL.Eval
  ( Val (..),
    eval,
    runEval,
    Error,
  )
where

import APL.AST (Exp (..), VName)
import Control.Monad (ap, liftM)

data Val
  = ValInt Integer
  | ValBool Bool
  | ValFun Env VName Exp
  deriving (Eq, Show)

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

type Error = String

newtype EvalM a = EvalM (Env -> Either Error a)

instance Functor EvalM where
  fmap = liftM

instance Applicative EvalM where
  pure x = EvalM $ \_env -> Right x
  (<*>) = ap

instance Monad EvalM where
  EvalM x >>= f = EvalM $ \env ->
    case x env of
      Left err -> Left err
      Right x' ->
        let EvalM y = f x'
         in y env

askEnv :: EvalM Env
askEnv = EvalM $ \env -> Right env

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f (EvalM m) = EvalM $ \env -> m (f env)

failure :: String -> EvalM a
failure s = EvalM $ \_env -> Left s

catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env ->
  case m1 env of
    Left _ -> m2 env
    Right x -> Right x

runEval :: EvalM a -> Either Error a
runEval (EvalM m) = m envEmpty

evalIntBinOp :: (Integer -> Integer -> EvalM Integer) -> Exp -> Exp -> EvalM Val
```

```
67  evalIntBinOp f e1 e2 = do
68    v1 <- eval e1
69    v2 <- eval e2
70    case (v1, v2) of
71      (ValInt x, ValInt y) -> ValInt <$> f x y
72      (_, _) -> failure "Non-integer operand"
73
74  evalIntBinOp' :: (Integer -> Integer -> Integer) -> Exp -> Exp -> EvalM Val
75  evalIntBinOp' f e1 e2 =
76    evalIntBinOp f' e1 e2
77    where
78      f' x y = pure $ f x y
79
80  eval :: Exp -> EvalM Val
81  eval (CstInt x) = pure $ ValInt x
82  eval (CstBool b) = pure $ ValBool b
83  eval (Var v) = do
84    env <- askEnv
85    case envLookup v env of
86      Just x -> pure x
87      Nothing -> failure $ "Unknown variable: " ++ v
88  eval (Add e1 e2) = evalIntBinOp' (+) e1 e2
89  eval (Sub e1 e2) = evalIntBinOp' (-) e1 e2
90  eval (Mul e1 e2) = evalIntBinOp' (*) e1 e2
91  eval (Div e1 e2) = evalIntBinOp checkedDiv e1 e2
92    where
93      checkedDiv _ 0 = failure "Division by zero"
94      checkedDiv x y = pure $ x `div` y
95  eval (Pow e1 e2) = evalIntBinOp checkedPow e1 e2
96    where
97      checkedPow x y =
98        if y < 0
99          then failure "Negative exponent"
100         else pure $ x ^ y
101 eval (Eql e1 e2) = do
102   v1 <- eval e1
103   v2 <- eval e2
104   case (v1, v2) of
105     (ValInt x, ValInt y) -> pure $ ValBool $ x == y
106     (ValBool x, ValBool y) -> pure $ ValBool $ x == y
107     (_, _) -> failure "Invalid operands to equality"
108 eval (If cond e1 e2) = do
109   cond' <- eval cond
110   case cond' of
111     ValBool True -> eval e1
112     ValBool False -> eval e2
113     _ -> failure "Non-boolean conditional."
114 eval (Let var e1 e2) = do
115   v1 <- eval e1
116   localEnv (envExtend var v1) $ eval e2
117 eval (ForLoop (loopparam, initial) (iv, bound) body) = do
118   initial_v <- eval initial
119   bound_v <- eval bound
120   case bound_v of
121     ValInt bound_int ->
122       loop 0 bound_int initial_v
123     _ ->
124       failure "Non-integral loop bound"
125   where
126     loop i bound_int loop_v
127       | i >= bound_int = pure loop_v
128       | otherwise = do
129           loop_v' <-
130             localEnv (envExtend iv (ValInt i) . envExtend loopparam loop_v) $
131               eval body
132           loop (succ i) bound_int loop_v'
133 eval (Lambda var body) = do
134   env <- askEnv
```

```
135   pure $ ValFun env var body
136 eval (Apply e1 e2) = do
137   v1 <- eval e1
138   v2 <- eval e2
139   case (v1, v2) of
140     (ValFun f_env var body, arg) ->
141       localEnv (const $ envExtend var arg f_env) $ eval body
142     (_, _) ->
143       failure "Cannot apply non-function"
144 eval (TryCatch e1 e2) =
145   eval e1 `catch` eval e2
```

APL/Eval_Tests.hs

```
module APL.Eval_Tests (tests) where

import APL.AST (Exp (..))
import APL.Eval (Error, Val (..), eval, runEval)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (testCase, (@?=))

eval' :: Exp -> Either Error Val
eval' = runEval . eval

evalTests :: TestTree
evalTests =
  testGroup
    "EValuation"
    [ testCase "Add" $
        eval' (Add (CstInt 2) (CstInt 5))
          @?= Right (ValInt 7),
      --
      testCase "Add (wrong type)" $
        eval' (Add (CstInt 2) (CstBool True))
          @?= Left "Non-integer operand",
      --
      testCase "Sub" $
        eval' (Sub (CstInt 2) (CstInt 5))
          @?= Right (ValInt (-3)),
      --
      testCase "Div" $
        eval' (Div (CstInt 7) (CstInt 3))
          @?= Right (ValInt 2),
      --
      testCase "Div0" $
        eval' (Div (CstInt 7) (CstInt 0))
          @?= Left "Division by zero",
      --
      testCase "Pow" $
        eval' (Pow (CstInt 2) (CstInt 3))
          @?= Right (ValInt 8),
      --
      testCase "Pow0" $
        eval' (Pow (CstInt 2) (CstInt 0))
          @?= Right (ValInt 1),
      --
      testCase "Pow negative" $
        eval' (Pow (CstInt 2) (CstInt (-1)))
          @?= Left "Negative exponent",
      --
      testCase "Eql (false)" $
        eval' (Eql (CstInt 2) (CstInt 3))
          @?= Right (ValBool False),
      --
      testCase "Eql (true)" $
        eval' (Eql (CstInt 2) (CstInt 2))
          @?= Right (ValBool True),
      --
      testCase "If" $
        eval' (If (CstBool True) (CstInt 2) (Div (CstInt 7) (CstInt 0)))
          @?= Right (ValInt 2),
      --
      testCase "Let" $
        eval' (Let "x" (Add (CstInt 2) (CstInt 3)) (Var "x"))
          @?= Right (ValInt 5),
      --
      testCase "ForLoop" $
        eval'
          (ForLoop ("p", CstInt 0) ("i", CstInt 10) (Add (Var "p") (Var "i")))
          @?= Right (ValInt 45),
```

```
67        --
68        testCase "Let (shadowing)" $
69          eval'
70            ( Let
71                "x"
72                (Add (CstInt 2) (CstInt 3))
73                (Let "x" (CstBool True) (Var "x"))
74            )
75            @?= Right (ValBool True),
76        --
77        testCase "Lambda/Apply" $
78          eval'
79            (Apply (Lambda "x" (Mul (Var "x") (Var "x"))) (CstInt 4))
80            @?= Right (ValInt 16),
81        --
82        testCase "TryCatch" $
83          eval'
84            (TryCatch (Div (CstInt 7) (CstInt 0)) (CstBool True))
85            @?= Right (ValBool True)
86      ]
87
88  printTests :: TestTree
89  printTests =
90    testGroup
91      "Task 1: Printing"
92      []
93
94  kvTests :: TestTree
95  kvTests =
96    testGroup
97      "Task 2: Key-value store"
98      []
99
100 tests :: TestTree
101 tests = testGroup "Evaluation" [evalTests, printTests, kvTests]
```

Week 3 exercises

Week 3 - Monadic Parsing

Suggested Reading

- Course Notes Chapter 3.
- Grammars and parsing with Haskell using Parser Combinators.
- Monadic Parsing in Haskell - one of the classic papers.
- Parsec: A practical parser library - a paper on Parsec, one of the first truly practical parser combinator libraries, which eventually became *Megaparsec*, which you will use.
- Chapter 13 in *Programming in Haskell*, but the material is similar (albeit with different Haskell code) to the course notes.

Going Beyond

- Parse, don't validate

Exercises

In these exercises you will construct a parser for APL, using the same AST definition as in the previous weeks. You are also given a partial interpreter, corresponding to the solution to last week's exercises, which allows you to actually run APL programs from the command line. Feel free to replace this with the fully featured `APL.Eval` module you developed for your assignments.

As always, the code handout is in handout/ and a full solution is in solution.

Getting started

The parser lives in the `APL.Parser` module you will put your tests in `APL.Parser_Tests`. The parser module uses Megaparsec as its parser library. It contains various useful imports already, although you are free to add more if you want.

We will start by implementing a parser for this very simple grammar, written in EBNF:

```
digit = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9";
int = digit {digit};
Exp ::= int "+" int;
```

In the grammar, we write terminals in all-lowercase or enclosed in single or double quotes, and nonterminals are written with a leading capital letter.

For the Haskell code we follow the convention discussed in the course notes where *lexer functions* are named with a leading `l`, and are the only functions that are allowed to directly manipulate whitespace. Parser functions are named with a leading `p`, and must not directly manipulate whitespace (such as by using the `spaces` parser).

Implement a lexer function

```
lInteger :: Parser Integer
```

that parses (non-negative) integers according to the grammar above (that is, one or more decimal digits).

To start, let us not worry about whitespace or similar complexities.

Hints You can use the function `read` to convert a string containing decimal numbers to a Haskell `Integer`. Note that `read` will blow up your program if the input is malformed, so be careful.

Use `parseTest` to easily test your parser in the REPL:

```
> parseTest lInteger "123"
123
```

Use `isDigit` from `Data.Char` to determine whether a character corresponds to a decimal digit.

Use the `some` combinator to repeatedly apply a parser and return a list of the results.

Use the `satisfy` combinator to parse a single character satisfying some predicate.

Solution Open this to see the answer

```
lInteger :: Parser Integer
lInteger = read <$> some (satisfy isDigit)
```

Handling whitespace

`lInteger` does not properly handle whitespace, which can be seen by trying to parse an integer followed by end-of-input (`eof`), when the string has trailing whitespace:

```
> parseTest (lInteger <* eof) "123 "
1:4:
  |
1 | 123
  |    ^
unexpected space
expecting end of input
```

Remember (from the course material) that the way we handle whitespace is for each function to consume *trailing* whitespace.

Define a combinator

```
lexeme :: Parser a -> Parser a
```

that applies a given parser, consumes trailing whitespace, then returns the value of that parser.

Hints Use the `<*` operator.

Open this to see the answer

```
lexeme :: Parser a -> Parser a
lexeme p = p <* space
```

Now use `lexeme` to modify `lInteger` to consume trailing whitespace such as in the `parseTest` example above.

Open this to see the answer

```
lInteger :: Parser Integer
lInteger = lexeme $ read <$> some (satisfy isDigit)
```

Correct tokenisation

The `lInteger` definition still has a subtle issue that will become a problem later, but we might as well fix it immediately to establish good habits. To illustrate the example, suppose that we also have a parser for parsing identifiers like `xyz`. (We will write such a parser in a bit, but for now it will live only in our imagination). If we then write a combined parser that parses *first* an integer, then an identifier, then a string such as `123xyz` would be parsed into an integer and a variable. This *may* be what we want, but it is not in accordance with normal conventions regarding syntax, where tokens must be separated from other tokens by either non-alphanumeric characters or whitespace.

We want APL to follow normal conventions. That is, we want this result:

```
> parseTest lInteger "123xyz"
1:4:
  |
1 | 123xyz
  |    ^
unexpected 'x'
```

Try it on your current definition and you will see a different result.

Modify your `lInteger` definition such that it only succeeds if *not* followed by an alphabetic character.

Hints Use `notFollowedBy` from Megaparsec.

Solution Open this to see the answer

```
lInteger :: Parser Integer
lInteger =
  lexeme $ read <$> some (satisfy isDigit) <* notFollowedBy (satisfy isAlpha)
```

Integer expressions and tests

Using `lInteger` as a building block, change `pExp` such that it can parse integer expressions (using the `CstInt` constructor).

Then go to `APL.Parser_Tests` and add some tests that test whether you can parse integers. Use the provided `parserTest` and `parserTestFail` helper functions. Consider using nested `testGroup`s to organise the tests.

Solution Open this to see the answer

In `APL.Parser`:

```
pExp :: Parser Exp
pExp = CstInt <$> lInteger
```

In `APL.Parser_Tests`:

```
parserTests :: TestTree
parserTests =
  testGroup
    "Parsing"
    [ testGroup
        "Constants"
        [ parserTest "123" $ CstInt 123,
          parserTest " 123" $ CstInt 123,
          parserTest "123 " $ CstInt 123,
          parserTestFail "123xyz"
        ]
    ]
```

Parsing identifiers

An APL identifier follows this grammar:

```
alphabetic = ? any alphabetic character ?;
alphanumeric = ? any alphanumeric character ?;
var = alphabetic {alphanumeric};
```

That is, it consists of an alphabetic character followed by zero or more alphanumeric characters.

Implement a parser

```
lVName :: Parser VName
```

that parses APL identifiers, including consuming trailing whitespace.

Also modify the `pExp` parser to handle identifiers (producing `Var` nodes) and add appropriate tests.

Hints Use `isAlpha` and `isAlphaNum` from `Data.Char`.

First read one character (which must be alphabetic), then read any number of characters (which must be alphanumeric).

Solution Open this to see the answer

```
lVName :: Parser VName
lVName = lexeme $ do
  c <- satisfy isAlpha
  cs <- many $ satisfy isAlphaNum
  pure $ c:cs

pExp :: Parser Exp
pExp =
  choice
    [ CstInt <$> lInteger,
      Var <$> lVName
    ]
```

Parsing Boolean values

Booleans follow this grammar in APL:

```
bool ::= "true" | "false";
```

Implement a parser function

```
pBool :: Parser Bool
```

that parses booleans. As with numbers, it is important that inputs such as `truee` are not parsed as `true` followed by another character. Also add a case for `CstBool` to `pExp`.

Examples

```
> parseTest pBool "true"
True
> parseTest pBool "truee"
1:5:
  |
1 | truee
  |     ^
unexpected 'e'
> parseTest pBool "true e"
True
```

Hints Consider defining a function

```
lKeyword :: String -> Parser ()
```

that parses a given alphanumeric string, and uses `notFollowedBy` to require that it is not followed by another alphanumeric character.

Solution Open this to see the answer

```
lKeyword :: String -> Parser ()
lKeyword s = lexeme $ void $ try $ chunk s <* notFollowedBy (satisfy isAlphaNum)

pBool :: Parser Bool
pBool =
  choice $
    [ const True <$> lKeyword "true",
      const False <$> lKeyword "false"
    ]

pExp :: Parser Exp
pExp =
  choice
    [ CstInt <$> lInteger,
      CstBool <$> pBool,
      Var <$> lVName
    ]
```

Excluding keywords

In most programming languages, some names are *keywords* (sometimes called *reserved words*) which are not allowed as identifiers. The subset of APL that we have implemented so far has the following keywords: `true`, `false`. Modify `lVName` such that it fails if the parsed name is a keyword. In future tasks, when you implement syntax that adds more keywords (e.g. `if`, `try`), make sure to modify your implementation such that they are recognised appropriately.

Examples

```
> parseTest lVName "if"
1:3:
  |
1 | if
  |   ^
Unexpected keyword
```

Note that the error location is a bit weird. This is because we check whether the identifier is a keyword after it has been read, as far as Megaparsec is concerned. Megaparsec does provide a facility for providing the correct error location, but that is beyond the scope of this course.

Hints Use `fail` to signal a parse error if the identifier is a keyword.

Because we are signaling a parse error after reading input, you must wrap the parser in the `try` combinator to indicate a backtracking point. If you forget to do so, your code will seem to work for now, but you will have subtle issues when we start to make use of keywords.

Solution Open this to see the answer

```
-- Remember to extend this as you support more of APL!
keywords :: [String]
keywords =
  [ "true",
    "false"
  ]


lVName :: Parser VName
lVName = lexeme $ try $ do
  c <- satisfy isAlpha
  cs <- many $ satisfy isAlphaNum
  let v = c : cs
  if v `elem` keywords
    then fail "Unexpected keyword"
    else pure v
```

Implement infix operators

In this task we will implement the infix operators `+`, `-`, `\`, and `*`, without taking operator precedence into account. The challenge is handling left recursion. We will also add explicit parentheses to the language - note that these do not correspond to any AST constructor.

Specifically, our grammar now looks like this (excluding unchanged rules from above):

```
Atom ::= var
       | int
       | bool
       | "(" Exp ")"

Exp ::= Atom
      | Exp "+" Exp
      | Exp "-" Exp
      | Exp "*" Exp
      | Exp "/" Exp
```

First you must transform the grammar to eliminate left recursion. Use the approach explained in the course material. Assume all operators are left-associative. This will give you a new definition of `Exp` that recognises the same language.

Open this to see the left-factorised grammar

```
Atom ::= var
       | int
       | bool
       | "(" Exp ")"

Exp0' ::=            (* empty *)
        | "+" Atom Exp0'
        | "-" Atom Exp0'
        | "*" Atom Exp0'
        | "/" Atom Exp0'

Exp0 ::= Atom Exp0'

Exp  ::= Exp0
```

Finish the parser `pExp` using your transformed grammar. Consider using the helper function

```
lString :: String -> Parser ()
lString s = lexeme $ void $ chunk s
```

to parse parentheses and operator symbols.

Open this to see the implementation

```
pAtom :: Parser Exp
pAtom =
  choice
    [ CstInt <$> lInteger,
      CstBool <$> lBool,
      Var <$> lVName,
      lString "(" *> pExp <* lString ")"
    ]

pExp0 :: Parser Exp
pExp0 = pAtom >>= chain
  where
    chain x =
      choice
        [ do
            lString "+"
            y <- pAtom
            chain $ Add x y,
          do
            lString "-"
            y <- pAtom
            chain $ Sub x y,
          do
            lString "*"
            y <- pAtom
            chain $ Mul x y,
          do
            lString "/"
            y <- pAtom
            chain $ Div x y,
          pure x
        ]

pExp :: Parser Exp
pExp = pExp0
```

Examples

```
> parseTest pExp "x+y*z"
Mul (Add (Var "x") (Var "y")) (Var "z")
> parseTest pExp "x+(y*z)"
Add (Var "x") (Mul (Var "y") (Var "z"))
```

Running the interpreter

You have now implemented enough of the parser for the interpreter to work for simple programs.

Use `cabal run apl PROG` to run the interpreter program `apl` on a file `PROG`, which must contain a valid APL expression. The resulting value (or an error message) will be printed to standard output and standard error, respectively. You can also do `cabal install` to install the `apl` executable in `$HOME/.local/bin`, after which you can simply do `apl PROG` to run programs.

There is no task here. Congratulate yourself on having written a full Haskell program and continue on.

Implement operator precedence

Starting from the left-factorised grammar, also separate the parser rules into levels corresponding to operator priorities. We have only two levels: `+` and `-` have the same priority, and `*` and `/` have the same priority, with the latter being the highest.

Open this to see the transformed grammar

```
Atom ::= var
       | int
       | bool
       | "(" Exp ")"

Exp1' ::=            (* empty *)
        | "*" Atom Exp1'
        | "/" Atom Exp1'

Exp1 ::= Atom Exp1'

Exp0' ::=            (* empty *)
        | "+" Exp1 Exp0'
        | "-" Exp1 Exp0'

Exp0 ::= Exp1 Exp0'

Exp  ::= Exp0
```

Now implement the corresponding parser.

Open this to see the implementation

```
pAtom :: Parser Exp
pAtom =
  choice
    [ CstInt <$> lInteger,
      CstBool <$> lBool,
      Var <$> lVName,
      lString "(" *> pExp <* lString ")"
    ]

pExp1 :: Parser Exp
pExp1 = pAtom >>= chain
  where
    chain x =
      choice
        [ do
            lString "*"
            y <- pAtom
            chain $ Mul x y,
          do
            lString "/"
            y <- pAtom
            chain $ Div x y,
          pure x
        ]

pExp0 :: Parser Exp
pExp0 = pExp1 >>= chain
  where
    chain x =
      choice
        [ do
            lString "+"
            y <- pExp1
            chain $ Add x y,
          do
            lString "-"
            y <- pExp1
            chain $ Sub x y,
          pure x
        ]

pExp :: Parser Exp
pExp = pExp0
```

Examples

```
> parseTest pExp "x+y*z"
Add (Var "x") (Mul (Var "y") (Var "z"))
```

Implement `if-then-else`

We extend the grammar to be the following:

```
Atom ::= var
       | int
       | bool
       | "(" Exp ")"

LExp ::= "if" Exp "then" Exp "else" Exp
       | Atom

Exp ::= LExp
      | Exp "+" Exp
      | Exp "-" Exp
      | Exp "*" Exp
      | Exp "/" Exp
```

An `if` expression extends as far as possible, meaning `if x then x else x + x` is understood as `if x then x else (x + x)`, rather than as `(if x then x else x) + x`.

Implement this grammar. Remember to handle the new keywords `if`, `then`, and `else`.

Solution Open this to see the implementation

```
pAtom :: Parser Exp
pAtom =
  choice
    [ CstInt <$> lInteger,
      CstBool <$> lBool,
      Var <$> lVName,
      lString "(" *> pExp <* lString ")"
    ]

pLExp :: Parser Exp
pLExp =
  choice
    [ If
        <$> (lKeyword "if" *> pExp0)
        <*> (lKeyword "then" *> pExp0)
        <*> (lKeyword "else" *> pExp0),
      pAtom
    ]

pExp1 :: Parser Exp
pExp1 = pLExp >>= chain
  where
    chain x =
      choice
        [ do
            lString "*"
            y <- pLExp
            chain $ Mul x y,
          do
            lString "/"
            y <- pLExp
            chain $ Div x y,
          pure x
        ]

pExp0 :: Parser Exp
pExp0 = pExp1 >>= chain
  where
    chain x =
```

```
      choice
        [ do
            lString "+"
            y <- pExp1
            chain $ Add x y,
          do
            lString "-"
            y <- pExp1
            chain $ Sub x y,
          pure x
        ]

pExp :: Parser Exp
pExp = pExp0
```

Week 3 solutions

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  | Lambda VName Exp
  | Apply Exp Exp
  | TryCatch Exp Exp
  | Print String Exp
  | KvPut Exp Exp
  | KvGet Exp
  deriving (Eq, Show)
```

APL/Eval.hs

```
module APL.Eval
  ( Val (..),
    Env,
    eval,
    runEval,
    Error,
  )
where

import APL.AST (Exp (..), VName)
import Control.Monad (ap, liftM)

data Val
  = ValInt Integer
  | ValBool Bool
  | ValFun Env VName Exp
  deriving (Eq, Show)

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

type Error = String

newtype EvalM a = EvalM (Env -> Either Error a)

instance Functor EvalM where
  fmap = liftM

instance Applicative EvalM where
  pure x = EvalM $ \_env -> Right x
  (<*>) = ap

instance Monad EvalM where
  EvalM x >>= f = EvalM $ \env ->
    case x env of
      Left err -> Left err
      Right x' ->
        let EvalM y = f x'
         in y env

askEnv :: EvalM Env
askEnv = EvalM $ \env -> Right env

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f (EvalM m) = EvalM $ \env -> m (f env)

failure :: String -> EvalM a
failure s = EvalM $ \_env -> Left s

catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env ->
  case m1 env of
    Left _ -> m2 env
    Right x -> Right x

runEval :: EvalM a -> Either Error a
runEval (EvalM m) = m envEmpty

```

```
67  evalIntBinOp :: (Integer -> Integer -> EvalM Integer) -> Exp -> Exp -> EvalM Val
68  evalIntBinOp f e1 e2 = do
69    v1 <- eval e1
70    v2 <- eval e2
71    case (v1, v2) of
72      (ValInt x, ValInt y) -> ValInt <$> f x y
73      (_, _) -> failure "Non-integer operand"
74
75  evalIntBinOp' :: (Integer -> Integer -> Integer) -> Exp -> Exp -> EvalM Val
76  evalIntBinOp' f e1 e2 =
77    evalIntBinOp f' e1 e2
78    where
79      f' x y = pure $ f x y
80
81  eval :: Exp -> EvalM Val
82  eval (CstInt x) = pure $ ValInt x
83  eval (CstBool b) = pure $ ValBool b
84  eval (Var v) = do
85    env <- askEnv
86    case envLookup v env of
87      Just x -> pure x
88      Nothing -> failure $ "Unknown variable: " ++ v
89  eval (Add e1 e2) = evalIntBinOp' (+) e1 e2
90  eval (Sub e1 e2) = evalIntBinOp' (-) e1 e2
91  eval (Mul e1 e2) = evalIntBinOp' (*) e1 e2
92  eval (Div e1 e2) = evalIntBinOp checkedDiv e1 e2
93    where
94      checkedDiv _ 0 = failure "Division by zero"
95      checkedDiv x y = pure $ x `div` y
96  eval (Pow e1 e2) = evalIntBinOp checkedPow e1 e2
97    where
98      checkedPow x y =
99        if y < 0
100         then failure "Negative exponent"
101         else pure $ x ^ y
102 eval (Eql e1 e2) = do
103   v1 <- eval e1
104   v2 <- eval e2
105   case (v1, v2) of
106     (ValInt x, ValInt y) -> pure $ ValBool $ x == y
107     (ValBool x, ValBool y) -> pure $ ValBool $ x == y
108     (_, _) -> failure "Invalid operands to equality"
109 eval (If cond e1 e2) = do
110   cond' <- eval cond
111   case cond' of
112     ValBool True -> eval e1
113     ValBool False -> eval e2
114     _ -> failure "Non-boolean conditional."
115 eval (Let var e1 e2) = do
116   v1 <- eval e1
117   localEnv (envExtend var v1) $ eval e2
118 eval (Lambda var body) = do
119   env <- askEnv
120   pure $ ValFun env var body
121 eval (Apply e1 e2) = do
122   v1 <- eval e1
123   v2 <- eval e2
124   case (v1, v2) of
125     (ValFun f_env var body, arg) ->
126       localEnv (const $ envExtend var arg f_env) $ eval body
127     (_, _) ->
128       failure "Cannot apply non-function"
129 eval (TryCatch e1 e2) =
130   eval e1 `catch` eval e2
```

APL/Parser.hs

```
module APL.Parser (parseAPL) where

import APL.AST (Exp (..), VName)
import Control.Monad (void)
import Data.Char (isAlpha, isAlphaNum, isDigit)
import Data.Void (Void)
import Text.Megaparsec
  ( Parsec,
    choice,
    chunk,
    eof,
    errorBundlePretty,
    many,
    notFollowedBy,
    parse,
    satisfy,
    some,
    try,
  )
import Text.Megaparsec.Char (space)

type Parser = Parsec Void String

lexeme :: Parser a -> Parser a
lexeme p = p <* space

keywords :: [String]
keywords =
  [ "if",
    "then",
    "else",
    "true",
    "false"
  ]

lVName :: Parser VName
lVName = lexeme $ try $ do
  c <- satisfy isAlpha
  cs <- many $ satisfy isAlphaNum
  let v = c : cs
  if v `elem` keywords
    then fail "Unexpected keyword"
    else pure v

lInteger :: Parser Integer
lInteger =
  lexeme $ read <$> some (satisfy isDigit) <* notFollowedBy (satisfy isAlphaNum)

lString :: String -> Parser ()
lString s = lexeme $ void $ chunk s

lKeyword :: String -> Parser ()
lKeyword s = lexeme $ void $ try $ chunk s <* notFollowedBy (satisfy isAlphaNum)

pBool :: Parser Bool
pBool =
  choice $
    [ const True <$> lKeyword "true",
      const False <$> lKeyword "false"
    ]

pAtom :: Parser Exp
pAtom =
  choice
    [ CstInt <$> lInteger,
      CstBool <$> pBool,
```

```
      Var <$> lVName,
      lString "(" *> pExp <* lString ")"
    ]

pLExp :: Parser Exp
pLExp =
  choice
    [ If
        <$> (lKeyword "if" *> pExp0)
        <*> (lKeyword "then" *> pExp0)
        <*> (lKeyword "else" *> pExp0),
      pAtom
    ]

pExp1 :: Parser Exp
pExp1 = pLExp >>= chain
  where
    chain x =
      choice
        [ do
            lString "*"
            y <- pLExp
            chain $ Mul x y,
          do
            lString "/"
            y <- pLExp
            chain $ Div x y,
          pure x
        ]

pExp0 :: Parser Exp
pExp0 = pExp1 >>= chain
  where
    chain x =
      choice
        [ do
            lString "+"
            y <- pExp1
            chain $ Add x y,
          do
            lString "-"
            y <- pExp1
            chain $ Sub x y,
          pure x
        ]

pExp :: Parser Exp
pExp = pExp0

parseAPL :: FilePath -> String -> Either String Exp
parseAPL fname s = case parse (space *> pExp <* eof) fname s of
  Left err -> Left $ errorBundlePretty err
  Right x -> Right x
```

APL/Parser_Tests.hs

```
module APL.Parser_Tests (tests) where

import APL.AST (Exp (..))
import APL.Parser (parseAPL)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (assertFailure, testCase, (@?=))

parserTest :: String -> Exp -> TestTree
parserTest s e =
  testCase s $
    case parseAPL "input" s of
      Left err -> assertFailure err
      Right e' -> e' @?= e

parserTestFail :: String -> TestTree
parserTestFail s =
  testCase s $
    case parseAPL "input" s of
      Left _ -> pure ()
      Right e ->
        assertFailure $
          "Expected parse error but received this AST:\n" ++ show e

tests :: TestTree
tests =
  testGroup
    "Parsing"
    [ testGroup
        "Constants"
        [ parserTest "123" $ CstInt 123,
          parserTest " 123" $ CstInt 123,
          parserTest "123 " $ CstInt 123,
          parserTestFail "123f",
          parserTest "true" $ CstBool True,
          parserTest "false" $ CstBool False
        ],
      testGroup
        "Basic operators"
        [ parserTest "x+y" $ Add (Var "x") (Var "y"),
          parserTest "x-y" $ Sub (Var "x") (Var "y"),
          parserTest "x*y" $ Mul (Var "x") (Var "y"),
          parserTest "x/y" $ Div (Var "x") (Var "y")
        ],
      testGroup
        "Operator priority"
        [ parserTest "x+y+z" $ Add (Add (Var "x") (Var "y")) (Var "z"),
          parserTest "x+y-z" $ Sub (Add (Var "x") (Var "y")) (Var "z"),
          parserTest "x+y*z" $ Add (Var "x") (Mul (Var "y") (Var "z")),
          parserTest "x*y*z" $ Mul (Mul (Var "x") (Var "y")) (Var "z"),
          parserTest "x/y/z" $ Div (Div (Var "x") (Var "y")) (Var "z")
        ],
      testGroup
        "Conditional expressions"
        [ parserTest "if x then y else z" $ If (Var "x") (Var "y") (Var "z"),
          parserTest "if x then y else if x then y else z" $
            If (Var "x") (Var "y") $
              If (Var "x") (Var "y") (Var "z"),
          parserTest "if x then (if x then y else z) else z" $
            If (Var "x") (If (Var "x") (Var "y") (Var "z")) (Var "z"),
          parserTest "1 + if x then y else z" $
            Add (CstInt 1) (If (Var "x") (Var "y") (Var "z"))
        ],
      testGroup
        "Lexing edge cases"
        [ parserTest "2 " $ CstInt 2,
          parserTest " 2" $ CstInt 2
```

```
67            ]
68        ]
```

Advanced Programming 2026

Assignment Three

Parsing Arithmetic Expressions

Due: 2026-9-27

Synopsis: Implementing a parser for a small arithmetic language.

1 Introduction

In this assignment you will be extending the parser developed during the exercises to support all of APL. Specifically you will add these features to the parser in the `Parser` module:

- Function application
- Additional operators (== and **).
- Syntax for `print`, `put`, and `get`.
- Lambdas, `let`-binding, loops, and `try-catch`.

For all tasks you are expected to add appropriate tests to the `Parser_Tests` module. Do not rename any of the definitions already present in the handout, and do not change their types.

The grammar supported by the handout is as follows, without left-factorisation and handling of operator priority. Each task describes an extension of the grammar which you must implement. You may also do other grammar transformations you find necessary.

```
Atom  ::=  var | int | bool | "(" Exp ")"
FExp  ::=  Atom
LExp  ::=  FExp
        |  "if" Exp "then" Exp "else" Exp
Exp   ::=  LExp
        |  Exp "+" Exp
        |  Exp "-" Exp
        |  Exp "*" Exp
        |  Exp "/" Exp
```

The grammar has some redundancies (mainly the FExp nonterminal) when compared to the one developed during the exercises, but it recognises the same language. The redundancies serve to make it easier to extend in the tasks below. You may find it necessary to adjust the code handout to completely match the grammar.

You must refactor the grammar to handle left recursion and operator precedence as appropriate.

2 Tasks

Task 1: Function application

In this task, you must implement function application by juxtaposition, similar to Haskell notation.

We extend the grammar with the following production:

$$\text{FExp} \quad ::= \quad \ldots \mid \text{FExp FExp}$$

Just as in Haskell, function application is left associative and binds tighter than any infix operator (this part is already expressed in the grammar), meaning `a b c` is parsed as `(a b) c`. In the AST, application is represented by the `Apply` constructor.

Examples

```
> parseAPL "" "x y z"
Right (Apply (Apply (Var "x") (Var "y")) (Var "z"))
> parseAPL "" "x(y z)"
Right (Apply (Var "x") (Apply (Var "y") (Var "z")))
> parseAPL "" "x if x then y else z"
Left ...
```

Task 2: Equality and power operators

In this task you must implement two new operators.

We extend the grammar with the following productions:

$$\begin{array}{lll} \text{Exp} & ::= & \ldots \\ & \mid & \text{Exp “==” Exp} \\ & \mid & \text{Exp “**” Exp} \end{array}$$

The disambiguation rules are as follows:

- == is left-associative and has the lowest precedence of all operators.
- ** is right-associative and has the highest precedence of all operators.

```
> parseAPL "" "x*y**z"
Right (Mul (Var "x") (Pow (Var "y") (Var "z")))
> parseAPL "" "x+y==y+x"
Right (Eql (Add (Var "x") (Var "y")) (Add (Var "y") (Var "x")))
```

2.1 Task 3: Printing, putting, and getting

In this task you must implement the `print`, `put`, and `get` operations.

We extend the grammar with the following productions:

Exp	::=	...
	|	"`print`" `string` Atom
	|	"`get`" Atom
	|	"`put`" Atom Atom

The `print`, `put`, and `get` terminals are keywords. The `string` terminal denotes string literals. These take the form of double quotes enclosing zero or more characters, excluding double quotes. You must implement parsing of these yourself.

Examples

```
> parseAPL "" "put x y"
Right (KvPut (Var "x") (Var "y"))
> parseAPL "" "get x + y"
Right (Add (KvGet (Var "x")) (Var "y"))
> parseAPL "" "getx"
Right (Var "getx")
> parseAPL "" "print \"foo\" x"
Right (Print "foo" (Var "x"))
```

Task 4: Lambdas, let-binding, loops, and try-catch

In this task you implement the final bits of APL syntax. We extend the grammar with the following productions:

```
LExp  ::=  ...
       |   "\" var "->" Exp
       |   "try" Exp "catch" Exp
       |   "let" var "=" Exp "in" Exp
       |   "loop" var "=" Exp "for" var "<" Exp "do" Exp
```

The rightmost Exps in the productions above extend to the right as far as possible; see the last two examples below. In a sense, LExps have the "lowest priority".

Examples

```
> parseAPL "" "let x = y in z"
Right (Let "x" (Var "y") (Var "z"))
> parseAPL "" "let true = y in z"
Left ...
> parseAPL "" "x let v = 2 in v"
Left ...
> parseAPL "" "\\x -> x + x"
Right (Lambda "x" (Add (Var "x") (Var "x")))
> parseAPL "" "(\\x -> x) + x"
Right (Add (Lambda "x" (Var "x")) (Var "x"))
```

3 Code handout

The code handout consists of the following nontrivial files.

- `a3.cabal`: Cabal build file. **Do not modify this file.**
- `src/APL/AST.hs`: AST definition. **Do not modify this file.**
- `src/APL/Eval.hs`: An incomplete evaluator corresponding to the solution to the week 2 exercises. Feel free to replace this module with one of your own.
- `src/APL/Parser.hs`: The incomplete parser where you will do most of your work.
- `src/APL/Parser_Tests.hs`: A parser test suite where you will add plentiful tests.

- `src/runtests.hs`: Test runner. **Do not modify this file.**
- `src/apl.hs`: Interpreter program that ties together parser and evaluator for running APL programs from files.

4 Your Report

You are expected to comment on the *interesting* details of your implementation. You are *not* expected to give a line-by-line walkthrough of your code. Most importantly, you are expected to reflect on the *quality* of your code:

- Do you think it is functionally correct? Why or why not?
- Is there some improvement you'd have liked to make, but didn't have the time?

It is more important to be aware of the strengths or shortcomings of your solution, than it is to have a complete solution.

4.1 The structure of your report

Your report must be structured exactly as follows:

Introduction: Briefly mention general concerns, any ambiguities in the problem text and how you resolved them, and your own estimation of the quality of your solution. Briefly mention whether your solution is functional, which test cases cover its functionality, which test cases it fails for (if any), and what you think might be wrong.

A section answering the following numbered questions:

1. Show the final and complete grammar with left-recursion removed and all ambiguities resolved.
2. Why might or might it not be problematic for your implementation that the grammar has operators * and ** where one is a prefix of the other?

3. Where in your parser is backtracking (`try`) strictly necessary, and where is it merely convenient? For one `try` you could remove, state what changes: the language recognised, the error messages, or the running time?
4. Give an example of where in your parser it matters that `choice` is *ordered* (picks the first successful parse) (or equivalently for `<|>`, if you have used that instead of `choice`). If no such case occurs, explain why.
5. Suppose APL is extended with comments notation (such as line comments or block comments as in Haskell). How would you implement this in your parser, and would it require many modifications?

All else being equal, **a short report is a good report**.

5 Deliverables for This Assignment

You must submit the following items:

- A single PDF file, A4 size, no more than 5 pages, structured as specified in the report section above.
- A single zip/tar.gz file with all code relevant to the implementation, including at least all the files from the handout. For this assignment it is not necessary to add additional files.

Remember to follow the general assignment rules listed on the course homepage.

6 Assessment

You will get written qualitative feedback, and points from zero to four. There are no resubmissions, so please hand in what you managed to do, even if you have not solved the assignment completely.

A3 handout

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  | ForLoop (VName, Exp) (VName, Exp) Exp
  | Lambda VName Exp
  | Apply Exp Exp
  | TryCatch Exp Exp
  | Print String Exp
  | KvPut Exp Exp
  | KvGet Exp
  deriving (Eq, Show)
```

APL/Eval.hs

```
module APL.Eval
  ( Val (..),
    Env,
    eval,
    runEval,
    Error,
  )
where

import APL.AST (Exp (..), VName)
import Control.Monad (ap, liftM)

data Val
  = ValInt Integer
  | ValBool Bool
  | ValFun Env VName Exp
  deriving (Eq, Show)

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

type Error = String

newtype EvalM a = EvalM (Env -> Either Error a)

instance Functor EvalM where
  fmap = liftM

instance Applicative EvalM where
  pure x = EvalM $ \_env -> Right x
  (<*>) = ap

instance Monad EvalM where
  EvalM x >>= f = EvalM $ \env ->
    case x env of
      Left err -> Left err
      Right x' ->
        let EvalM y = f x'
         in y env

askEnv :: EvalM Env
askEnv = EvalM $ \env -> Right env

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f (EvalM m) = EvalM $ \env -> m (f env)

failure :: String -> EvalM a
failure s = EvalM $ \_env -> Left s

catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env ->
  case m1 env of
    Left _ -> m2 env
    Right x -> Right x

runEval :: EvalM a -> ([String], Either Error a)
runEval (EvalM m) = ([], m envEmpty)

```

```
67  evalIntBinOp :: (Integer -> Integer -> EvalM Integer) -> Exp -> Exp -> EvalM Val
68  evalIntBinOp f e1 e2 = do
69    v1 <- eval e1
70    v2 <- eval e2
71    case (v1, v2) of
72      (ValInt x, ValInt y) -> ValInt <$> f x y
73      (_, _) -> failure "Non-integer operand"
74
75  evalIntBinOp' :: (Integer -> Integer -> Integer) -> Exp -> Exp -> EvalM Val
76  evalIntBinOp' f e1 e2 =
77    evalIntBinOp f' e1 e2
78    where
79      f' x y = pure $ f x y
80
81  eval :: Exp -> EvalM Val
82  eval (CstInt x) = pure $ ValInt x
83  eval (CstBool b) = pure $ ValBool b
84  eval (Var v) = do
85    env <- askEnv
86    case envLookup v env of
87      Just x -> pure x
88      Nothing -> failure $ "Unknown variable: " ++ v
89  eval (Add e1 e2) = evalIntBinOp' (+) e1 e2
90  eval (Sub e1 e2) = evalIntBinOp' (-) e1 e2
91  eval (Mul e1 e2) = evalIntBinOp' (*) e1 e2
92  eval (Div e1 e2) = evalIntBinOp checkedDiv e1 e2
93    where
94      checkedDiv _ 0 = failure "Division by zero"
95      checkedDiv x y = pure $ x `div` y
96  eval (Pow e1 e2) = evalIntBinOp checkedPow e1 e2
97    where
98      checkedPow x y =
99        if y < 0
100         then failure "Negative exponent"
101         else pure $ x ^ y
102 eval (Eql e1 e2) = do
103   v1 <- eval e1
104   v2 <- eval e2
105   case (v1, v2) of
106     (ValInt x, ValInt y) -> pure $ ValBool $ x == y
107     (ValBool x, ValBool y) -> pure $ ValBool $ x == y
108     (_, _) -> failure "Invalid operands to equality"
109 eval (If cond e1 e2) = do
110   cond' <- eval cond
111   case cond' of
112     ValBool True -> eval e1
113     ValBool False -> eval e2
114     _ -> failure "Non-boolean conditional."
115 eval (Let var e1 e2) = do
116   v1 <- eval e1
117   localEnv (envExtend var v1) $ eval e2
118 eval (Lambda var body) = do
119   env <- askEnv
120   pure $ ValFun env var body
121 eval (Apply e1 e2) = do
122   v1 <- eval e1
123   v2 <- eval e2
124   case (v1, v2) of
125     (ValFun f_env var body, arg) ->
126       localEnv (const $ envExtend var arg f_env) $ eval body
127     (_, _) ->
128       failure "Cannot apply non-function"
129 eval (TryCatch e1 e2) =
130   eval e1 `catch` eval e2
131 eval e =
132   error $ "Evaluation of this expression not implemented:\n" ++ show e
```

APL/Parser.hs

```
module APL.Parser (parseAPL) where

import APL.AST (Exp (..), VName)
import Control.Monad (void)
import Data.Char (isAlpha, isAlphaNum, isDigit)
import Data.Void (Void)
import Text.Megaparsec
  ( Parsec,
    choice,
    chunk,
    eof,
    errorBundlePretty,
    many,
    notFollowedBy,
    parse,
    satisfy,
    some,
    try,
  )
import Text.Megaparsec.Char (space)

type Parser = Parsec Void String

lexeme :: Parser a -> Parser a
lexeme p = p <* space

keywords :: [String]
keywords =
  [ "if",
    "then",
    "else",
    "true",
    "false"
  ]

lVName :: Parser VName
lVName = lexeme $ try $ do
  c <- satisfy isAlpha
  cs <- many $ satisfy isAlphaNum
  let v = c : cs
  if v `elem` keywords
    then fail "Unexpected keyword"
    else pure v

lInteger :: Parser Integer
lInteger =
  lexeme $ read <$> some (satisfy isDigit) <* notFollowedBy (satisfy isAlphaNum)

lString :: String -> Parser ()
lString s = lexeme $ void $ chunk s

lKeyword :: String -> Parser ()
lKeyword s = lexeme $ void $ try $ chunk s <* notFollowedBy (satisfy isAlphaNum)

pBool :: Parser Bool
pBool =
  choice $
    [ const True <$> lKeyword "true",
      const False <$> lKeyword "false"
    ]

pAtom :: Parser Exp
pAtom =
  choice
    [ CstInt <$> lInteger,
      CstBool <$> pBool,
```

```
      Var <$> lVName,
      lString "(" *> pExp <* lString ")"
    ]

pLExp :: Parser Exp
pLExp =
  choice
    [ If
        <$> (lKeyword "if" *> pExp)
        <*> (lKeyword "then" *> pExp)
        <*> (lKeyword "else" *> pExp),
      pAtom
    ]

pExp1 :: Parser Exp
pExp1 = pLExp >>= chain
  where
    chain x =
      choice
        [ do
            lString "*"
            y <- pLExp
            chain $ Mul x y,
          do
            lString "/"
            y <- pLExp
            chain $ Div x y,
          pure x
        ]

pExp0 :: Parser Exp
pExp0 = pExp1 >>= chain
  where
    chain x =
      choice
        [ do
            lString "+"
            y <- pExp1
            chain $ Add x y,
          do
            lString "-"
            y <- pExp1
            chain $ Sub x y,
          pure x
        ]

pExp :: Parser Exp
pExp = pExp0

parseAPL :: FilePath -> String -> Either String Exp
parseAPL fname s = case parse (space *> pExp <* eof) fname s of
  Left err -> Left $ errorBundlePretty err
  Right x -> Right x
```

APL/Parser_Tests.hs

```
module APL.Parser_Tests (tests) where

import APL.AST (Exp (..))
import APL.Parser (parseAPL)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (assertFailure, testCase, (@?=))

parserTest :: String -> Exp -> TestTree
parserTest s e =
  testCase s $
    case parseAPL "input" s of
      Left err -> assertFailure err
      Right e' -> e' @?= e

parserTestFail :: String -> TestTree
parserTestFail s =
  testCase s $
    case parseAPL "input" s of
      Left _ -> pure ()
      Right e ->
        assertFailure $
          "Expected parse error but received this AST:\n" ++ show e

tests :: TestTree
tests =
  testGroup
    "Parsing"
    [ testGroup
        "Constants"
        [ parserTest "123" $ CstInt 123,
          parserTest " 123" $ CstInt 123,
          parserTest "123 " $ CstInt 123,
          parserTestFail "123f",
          parserTest "true" $ CstBool True,
          parserTest "false" $ CstBool False
        ],
      testGroup
        "Basic operators"
        [ parserTest "x+y" $ Add (Var "x") (Var "y"),
          parserTest "x-y" $ Sub (Var "x") (Var "y"),
          parserTest "x*y" $ Mul (Var "x") (Var "y"),
          parserTest "x/y" $ Div (Var "x") (Var "y")
        ],
      testGroup
        "Operator priority"
        [ parserTest "x+y+z" $ Add (Add (Var "x") (Var "y")) (Var "z"),
          parserTest "x+y-z" $ Sub (Add (Var "x") (Var "y")) (Var "z"),
          parserTest "x+y*z" $ Add (Var "x") (Mul (Var "y") (Var "z")),
          parserTest "x*y*z" $ Mul (Mul (Var "x") (Var "y")) (Var "z"),
          parserTest "x/y/z" $ Div (Div (Var "x") (Var "y")) (Var "z")
        ],
      testGroup
        "Conditional expressions"
        [ parserTest "if x then y else z" $ If (Var "x") (Var "y") (Var "z"),
          parserTest "if x then y else if x then y else z" $
            If (Var "x") (Var "y") $
              If (Var "x") (Var "y") (Var "z"),
          parserTest "if x then (if x then y else z) else z" $
            If (Var "x") (If (Var "x") (Var "y") (Var "z")) (Var "z"),
          parserTest "1 + if x then y else z" $
            Add (CstInt 1) (If (Var "x") (Var "y") (Var "z"))
        ],
      testGroup
        "Lexing edge cases"
        [ parserTest "2 " $ CstInt 2,
          parserTest " 2" $ CstInt 2
```

```
67            ]
68        ]
```

Week 4 exercises

Week 4 - Free Monads

Suggested Reading

- Course Notes Chapter 4
  - Optional: Introduction to Free Monads
- Chapter 10 in *Programming in Haskell*.

Going Beyond

- Free Monads for Less part 1
- Free Monads for Less part 2

Exercises

In these exercises, you will construct a monadic interpreter similar in functionality to the one you developed in week 2. This time, however, you will implement the evaluation monad as a free monad. The fourth mandatory assignment will ask that you then extend the evaluation monad with new effects and ways of interpreting them.

Before you proceed with these exercises, it's important that you've read and understood the week 4 course notes on free monads. If you haven't, now's the time to pause and go read!

Your starting point is found in handout/ and complete solutions are in solutions/. For some of the tasks, partial or complete solutions are also included inline below.

Handout Structure

The handout has the following structure. These exercises will form the starting basis of A4, so pay attention!

```
handout
├── runtests.hs
├── src
│   └── APL
│       ├── AST.hs
│       ├── Eval.hs
│       ├── InterpIO.hs
│       ├── InterpPure.hs
│       ├── Interp_Tests.hs
│       ├── Monad.hs
│       └── Util.hs
└── week4.cabal
```

- `runtests.hs`: Test runner. **Do not modify this file.**
- `week4.cabal`: Cabal build file. **Do not modify this file.**
- `src/APL/AST.hs`: AST definition. **Do not modify this file.**
- `src/APL/Eval.hs`: An incomplete evaluator corresponding to the solution to the week 2 exercises.
- `src/APL/InterpIO.hs`: Contains the incomplete IO-based `runEvalIO` interpreter.
- `src/APL/InterpPure.hs`: Contains the incomplete pure `runEval` interpreter.
- `src/APL/Interp_Tests.hs`: An interpreter test suite where you will add plentiful tests.
- `src/APL/Monad.hs`: Contains all things related to the evaluation monad. Note that some definitions from A2 have moved from `APL.Eval` to `APL.Monad` in this assignment; e.g. `Val` and definitions related to the environment.
- `src/APL/Util.hs`: Utility functions needed for testing IO and some other stuff. You can safely ignore this file. **Do not modify this file.**

Getting Started

- `APL.Eval`: To make testing easier, you should replace the definition of `eval` with your complete evaluator from your solution to A2.

Note that doing this isn't strictly required and you can fully solve and test (by constructing the appropriate `EvalM` values directly or with the interface functions in `APL.Monad`) both these exercises and the assignment without replacing `eval`. But, we think it's probably nicer to have a complete version of `eval` that can handle all of the different expression types. It's also cool to see that you don't have to change your version of `eval` from A2 at all despite the fact that the underlying monad will change rather significantly.

`Functor` and `Monad` instances for `Free e a`

- `APL.Monad`: Complete the skeleton code for the `Functor` and `Monad` instances of `Free e a`.

If you feel the need to review the course notes, feel free to first do so, but try to complete the definitions without directly copying from the notes.

Hints Keep in mind the `Functor e` constraint on each of the instances—this should suggest to you that you'll have to (at least) use `e`'s `fmap` in each of the instances.

Solution Open this to see the answer

```
-- APL.Monad:
instance (Functor e) => Functor (Free e) where
  fmap f (Pure x) = Pure $ f x
  fmap f (Free g) = Free $ fmap (fmap f) g

instance (Functor e) => Monad (Free e) where
  Pure x >>= f = f x
  Free g >>= f = Free $ h <$> g
    where
      h x = x >>= f
```

Defining the evaluation monad

In this task, you'll develop a free monad-based version of the `EvalM` evaluation monad. The first thing to do is define an effect type for`EvalM`, which will include a constructor for each kind of effect we want the `EvalM` monad to support. To keep things simple, we'll add effects one-by-one. Let's start with a `Reader` effect:

```
-- APL.Monad:
data EvalOp a
  = ReadOp (Env -> a)
```

To make `EvalOp` usable with `Free`, it needs a `Functor` instance.

- `APL.Monad`: Implement the `Functor` instance for `EvalOp`.

Our evaluation monad is just a free monad with effects chosen from `EvalOp`:

```
-- APL.Monad:
type EvalM a = Free EvalOp a
```

The next step is to begin building the pure interpretation function to run `EvalM` monads.

- `APL.InterpPure`: Complete the skeleton code for `runEval`.

Solution Open this to see the answer

```
-- APL.Monad:
instance Functor EvalOp where
  fmap f (ReadOp k) = ReadOp $ f . k

-- APL.InterpPure:
runEval :: EvalM a -> a
runEval = runEval' envEmpty
  where
    runEval' :: Env -> EvalM a -> a
    runEval' _ (Pure x) = x
    runEval' r (Free (ReadOp k)) = runEval' r $ k r
```

Extending `EvalM` with more effects

Now that we have the basic structure of `EvalM` in place, it's time to extend it with some more effects.

State effects There are two state effects:

1. You can retrieve the state.
2. You can replace the state with a new state.

We'll have to model both, so let's extend `EvalOp` appropriately:

```
-- APL.Monad:
data EvalOp a
  = ReadOp (Env -> a)
  | StateGetOp (State -> a)
  | StatePutOp State a
```

- `APL.Monad`: Extend the `Functor` instance for `EvalOp` to include the new state effects.
- `APL.InterpPure`: Extend the code for `runEval` to include support for the new state effects.
- `APL.Interp_Tests`: Add tests to `pureTests` to test your new effects.

Hints You'll have to modify `runEval'` to take a new parameter—the state:

```
-- APL.InterpPure:
runEval' :: Env -> State -> EvalM a -> a
```

Solution Open this to see the answer

```
-- APL.Monad:
instance Functor EvalOp where
  fmap f (ReadOp k) = ReadOp $ f . k
  fmap f (StateGetOp k) = StateGetOp $ f . k
  fmap f (StatePutOp s m) = StatePutOp s $ f m
```

```
-- APL.InterpPure:
runEval :: EvalM a -> a
runEval = runEval' envEmpty stateInitial
  where
    runEval' :: Env -> State -> EvalM a -> a
    runEval' _ _ (Pure x) = x
    runEval' r s (Free (ReadOp k)) = runEval' r s $ k r
    runEval' r s (Free (StateGetOp k)) = runEval' r s $ k s
    runEval' r _ (Free (StatePutOp s' m)) = runEval' r s' m
```

Some useful interfaces

Just like in A2, we write interface functions so we don't have to manually construct`EvalM` effect terms. For example, to retrieve the environment we re-implement `askEnv` from A2 to work with the free monad version of `EvalM`:

```
-- APL.Monad:
askEnv :: EvalM Env
askEnv = Free $ ReadOp $ \env -> pure env
```

There are also a few useful interfaces for retrieving/putting/manipulating state.

- Implement `getState` and `putState`. Use them to implement `modifyState`.

We also want a way to make local environments, i.e., we want to implement the function

```
-- APL.Monad:
localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f m = ...
```

This interface is a bit trickier. Somehow we have to take an `EvalM a`—which you can think of as *stack* of effects that returns some `a`—and modify every `ReadOp` effect within that stack using`f`. How do we do that? We're going to need some way to traverse the stack of effects and modify them:[1]

```
-- APL.Monad:
modifyEffects :: (Functor e, Functor h) => (e (Free e a) -> h (Free e a)) -> Free e a -> Free h a
modifyEffects _ (Pure x) = Pure x
```

[1]This is an inefficient way to implement local environments (since you have to traverse over the entire effect stack each time); a better way would be to add a local environment effect. We do it this way for pedagogical reasons—implementing `modifyEffects` forces you to think about what the effect stack looks like and how to traverse it.

```
modifyEffects g (Free e) = ...
```

- `APL.Monad`: Complete the definition of `modifyEffects`.
- `APL.Monad`: Using `modifyEffects`, implement `localEnv`.
- `APL.Interp_Tests`: Add tests to `pureTests` to test your new interfaces. Interpreting `APL` expressions that modify the state and/or generate local environments is probably a good idea, but you can also test interfaces directly:

```
testCase "Let" $
  eval' (Let "x" (Add (CstInt 2) (CstInt 3)) (Var "x"))
    @?= ValInt 5,

testCase "localEnv" $
  runEval
    ( localEnv (const [("x", ValInt 1)]) $
              askEnv
  )
    @?= [("x", ValInt 1)]
```

Hints `modifyEffects` is a recursive function and uses `fmap`.

In your definition of `localEnv`, the function you pass to `modifyEffects` should only modify `ReadOp` effects; for all other effects it should act as the identity function.

Solution Open this to see the answer

```
-- APL.Monad:
modifyEffects :: (Functor e, Functor h) => (e (Free e a) -> h (Free e a)) -> Free e a -> Free h a
modifyEffects _ (Pure x) = Pure x
modifyEffects g (Free e) = Free $ modifyEffects g <$> g e

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f = modifyEffects g
  where
    g (ReadOp k) = ReadOp $ k . f
    g op = op

getState :: EvalM State
getState = Free $ StateGetOp $ \s -> pure s

putState :: State -> EvalM ()
putState s = Free $ StatePutOp s $ pure ()

modifyState :: (State -> State) -> EvalM ()
modifyState f = do
  s <- getState
  putState $ f s
```

Print effects

Let's extend `EvalOp` with an effect for printing:

```
-- APL.Monad:
data EvalOp a
  = ReadOp (Env -> a)
  | StateGetOp (State -> a)
  | StatePutOp State a
  | PrintOp String a
```

- `APL.Monad`: Extend the `Functor` instance of `EvalOp` to include `PrintOp`.

We'll need to change the type of `runEval` and `runEval'` to also spit out a list of printed strings:

```
-- APL.InterpPure:
runEval :: EvalM a -> ([String], a)

runEval' :: Env -> State -> EvalM a -> ([String], a)
```

You'll also have to modify the `Pure` case of `runEval` to play nice with the new output type. You'll also need to appropriately change the type signature of `eval'` in `APL.Interp_Tests`.

On each `PrintOp s m` effect, `runEval` should include the string `s` in the final output.

- `APL.Monad`: Using `PrintOp`, fill in the definition for `evalPrint`.
- `APL.InterpPure`: Extend `runEval` to support `PrintOp` effects.
- `APL.Interp_Tests`: Add tests to `pureTests` for the `PrintOp` effect.

Solution Open this to see the answer

```
-- APL.Monad:
instance Functor EvalOp where
  fmap f (ReadOp k) = ReadOp $ f . k
  fmap f (StateGetOp k) = StateGetOp $ f . k
  fmap f (StatePutOp s m) = StatePutOp s $ f m
  fmap f (PrintOp p m) = PrintOp p $ f m

evalPrint :: String -> EvalM ()
evalPrint p = Free $ PrintOp p $ pure ()
```

```
-- APL.InterpPure:
runEval :: EvalM a -> ([String], a)
runEval = runEval' envEmpty stateInitial
  where
    runEval' :: Env -> State -> EvalM a -> ([String], a)
    runEval' _ _ (Pure x) = ([], x)
    runEval' r s (Free (ReadOp k)) = runEval' r s $ k r
    runEval' r s (Free (StateGetOp k)) = runEval' r s $ k s
    runEval' r _ (Free (StatePutOp s' m)) = runEval' r s' m
    runEval' r s (Free (PrintOp p m)) =
      let (ps, res) = runEval' r s m
       in (p : ps, res)
```

Error effects

The `ErrorOp` effect is for failures. When an `ErrorOp` effect is interpreted, the computation fails and an error is returned as the result. Just like in the previous assignments, we'll use `Either` to be able represent both failure and success results and we'll use the existing `Error` type for errors.

When `runEval` interprets an `ErrorOp e` effect, the result of the entire computation should be `Left e`. For example,

```
> runEval $ Free $ ErrorOp "Oh no!"
([],Left "Oh no!")
```

Let's extend `EvalOp` with error effects:

```
-- APL.Monad:
data EvalOp a
  = ReadOp (Env -> a)
  | StateGetOp (State -> a)
  | StatePutOp State a
  | PrintOp String a
  | ErrorOp Error
```

- `APL.Monad`: Extend the `Functor` instance of `EvalOp` to include `ErrorOp`.
- `APL.Monad`: Using `ErrorOp`, fill in the definition for `failure`.

Change the type of `runEval` and `runEval'` to return an `Either Error a` type:

```
-- APL.Monad:
runEval :: EvalM a -> ([String], Either Error a)

runEval' :: Env -> State -> EvalM a -> ([String], Either Error a)
```

You'll also need to update the `Pure x` case for `runEval'` to play nice with the new return type. You'll also need to appropriately change the type signature of `eval'` in `APL.Interp_Tests`.

- `APL.InterpPure`: Extend `runEval` to support `ErrorOp` effects.

- `APL.Interp_Tests`: Add tests to `pureTests` for the `ErrorOp` effect.

Solution Open this to see the answer

```
-- APL.Monad:
instance Functor EvalOp where
  fmap f (ReadOp k) = ReadOp $ f . k
  fmap f (StateGetOp k) = StateGetOp $ f . k
  fmap f (StatePutOp s m) = StatePutOp s $ f m
  fmap f (PrintOp p m) = PrintOp p $ f m
  fmap _ (ErrorOp e) = ErrorOp e

failure :: String -> EvalM a
failure = Free . ErrorOp
```

```
-- APL.InterpPure:
runEval :: EvalM a -> ([String], Either Error a)
runEval = runEval' envEmpty stateInitial
  where
    runEval' :: Env -> State -> EvalM a -> ([String], Either Error a)
    runEval' _ _ (Pure x) = ([], pure x)
    runEval' r s (Free (ReadOp k)) = runEval' r s $ k r
    runEval' r s (Free (StateGetOp k)) = runEval' r s $ k s
    runEval' r _ (Free (StatePutOp s' m)) = runEval' r s' m
    runEval' r s (Free (PrintOp p m)) =
      let (ps, res) = runEval' r s m
       in (p : ps, res)
    runEval' _ _ (Free (ErrorOp e)) = ([], Left e)
```

Interpreting effects

Since we've only changed the underlying workings of `EvalM`—and not its interface—your `eval` function from A2 (which you should use to replace the version of `eval` given in the handout in the `APL.Eval` module if you haven't) works without modification.

The important thing to understand here is that `eval` now produces a free monad-based stack of uninterpreted effects; it doesn't actually carry out the effects and only produces the uninterpreted structure.

So what actually does the computation/interprets the stack of effects? The interpreters of course! So far, we've only worked on the pure interpreter `runEval` in `APL.InterpPure`. Let's try it out in GHCi:

```
> m =  eval $ Let "x" (Add (CstInt 1) (CstInt 2)) $ Print ("The value of x is") (Var "x")
> runEval m
(["The value of x is: 3"], Right (ValInt 3))
```

Now,`runEval` is just one interpretation of `eval e`'s effect stack. The `APL.InterpIO` module contains another interpreter, `runEvalIO`. This interpreter has IO access and so can generate side effects.

The version of `runEvalIO` included in the handout mimics the workings of `runEval` so far—now's the time to uncomment the code in `APL.InterpIO`—except that it is missing a case for `PrintOp`.

Instead of returning the printed string in the first component of our output, the `runEvalIO` interpretation of a `PrintOp` effect should print to the terminal instead. For example,

```
> runEvalIO envEmpty m
The value of x is: 3
Right (ValInt 3)
```

Notice that this operation is reflected in the type of `runEvalIO`:

```
-- APL.InterpIO:
runEvalIO :: EvalM a -> IO (Either Error a)
```

There's no need to return a list of printed statements (the `[String]` component in the return of `runEval`) because `PrintOp` effects will be directly printed to the terminal as a side effect.

Also notice that in both the `runEval` and `runEvalIO` examples above we ran the interpreters on *the same* `m` computation. The only thing that changed is *how we interpreted* `m`.

The point is that free monads afford us the flexibility to construct a computation just once and then interpret the (effects of) the computation in different ways. Pretty cool!

- `APL.InterpIO`: Add support for `PrintOp` effects to `runEvalIO'`.

Hints You can use `putStrLn` to print strings to the terminal.

Solution Open this to see the answer

```
-- APL.InterpIO:
runEvalIO :: EvalM a -> IO (Either Error a)
runEvalIO = runEvalIO' envEmpty stateInitial
  where
    runEvalIO' :: Env -> State -> EvalM a -> IO (Either Error a)
    runEvalIO' _ _ (Pure x) = pure $ pure x
    runEvalIO' r s (Free (ReadOp k)) = runEvalIO' r s $ k r
    runEvalIO' r s (Free (StateGetOp k)) = runEvalIO' r s $ k s
    runEvalIO' r _ (Free (StatePutOp s' m)) = runEvalIO' r s' m
    runEvalIO' r s (Free (PrintOp p m)) = do
      putStrLn p
      runEvalIO' r s m
    runEvalIO' _ _ (Free (ErrorOp e)) = pure $ Left e
```

Testing `runEvalIO`

To test IO-based interpretation of effects, a special function `captureIO` is provided for you in the `APL.Util` module:

```
captureIO :: [String] -> IO a -> IO ([String], a)
```

You use `captureIO` like this: `captureIO inputs m`, where `inputs` is a list of inputs (lines) you'd like to write to `stdin` during the execution of the `m` action. `captureIO` runs `m` using `inputs` as input to `stdin` and captures any output to `stdout`, returning it as a list of strings (and also returns the result of the computation itself).

We can use `captureIO` to test an IO-based interpretation of a `Print`-expression like so:

```
-- APL.Interp_Tests
testCase "print" $ do
     let s1 = "Lalalalala"
         s2 = "Weeeeeeeee"
     (out, res) <-
       captureIO [] $
         runEvalIO $ do
           evalPrint s1
           evalPrint s2
     (out, res) @?= ([s1, s2], Right ())
```

For print effects, we don't have any input to `stdin` so we just feed `captureIO` an empty list (`[]`). In the assignment, you'll add additional effects that do read from `stdin` and will have to add inputs for `captureIO` to test these additional effects.

- `APL.Interp_Tests`: Add tests to `ioTests` for the `PrintOp` effect.

Week 4 solutions

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  | ForLoop (VName, Exp) (VName, Exp) Exp
  | Lambda VName Exp
  | Apply Exp Exp
  | TryCatch Exp Exp
  | Print String Exp
  | KvPut Exp Exp
  | KvGet Exp
  deriving (Eq, Show)
```

APL/Eval.hs

```
module APL.Eval
  ( eval,
  )
where

import APL.AST (Exp (..))
import APL.Monad

evalIntBinOp :: (Integer -> Integer -> EvalM Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp f e1 e2 = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValInt x, ValInt y) -> ValInt <$> f x y
    (_, _) -> failure "Non-integer operand"

evalIntBinOp' :: (Integer -> Integer -> Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp' f e1 e2 =
  evalIntBinOp f' e1 e2
  where
    f' x y = pure $ f x y

-- Replace with your 'eval' from your solution to assignment 2.
eval :: Exp -> EvalM Val
eval (CstInt x) = pure $ ValInt x
eval (CstBool b) = pure $ ValBool b
eval (Var v) = do
  env <- askEnv
  case envLookup v env of
    Just x -> pure x
    Nothing -> failure $ "Unknown variable: " ++ v
eval (Add e1 e2) = evalIntBinOp' (+) e1 e2
eval (Sub e1 e2) = evalIntBinOp' (-) e1 e2
eval (Mul e1 e2) = evalIntBinOp' (*) e1 e2
eval (Div e1 e2) = evalIntBinOp checkedDiv e1 e2
  where
    checkedDiv _ 0 = failure "Division by zero"
    checkedDiv x y = pure $ x `div` y
eval (Pow e1 e2) = evalIntBinOp checkedPow e1 e2
  where
    checkedPow x y =
      if y < 0
        then failure "Negative exponent"
        else pure $ x ^ y
eval (Eql e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValInt x, ValInt y) -> pure $ ValBool $ x == y
    (ValBool x, ValBool y) -> pure $ ValBool $ x == y
    (_, _) -> failure "Invalid operands to equality"
eval (If cond e1 e2) = do
  cond' <- eval cond
  case cond' of
    ValBool True -> eval e1
    ValBool False -> eval e2
    _ -> failure "Non-boolean conditional."
eval (Let var e1 e2) = do
  v1 <- eval e1
  localEnv (envExtend var v1) $ eval e2
eval (ForLoop (loopparam, initial) (iv, bound) body) = do
  initial_v <- eval initial
  bound_v <- eval bound
  case bound_v of
    ValInt bound_int ->
      loop 0 bound_int initial_v
```

```
    _ ->
      failure "Non-integral loop bound"
  where
    loop i bound_int loop_v
      | i >= bound_int = pure loop_v
      | otherwise = do
          loop_v' <-
            localEnv (envExtend iv (ValInt i) . envExtend loopparam loop_v) $
              eval body
          loop (succ i) bound_int loop_v'
eval (Lambda var body) = do
  env <- askEnv
  pure $ ValFun env var body
eval (Apply e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValFun f_env var body, arg) ->
      localEnv (const $ envExtend var arg f_env) $ eval body
    (_, _) ->
      failure "Cannot apply non-function"
eval (TryCatch e1 e2) =
  eval e1 `catch` eval e2
```

APL/InterpIO.hs

```
module APL.InterpIO (runEvalIO) where

import APL.Monad

runEvalIO :: EvalM a -> IO (Either Error a)
runEvalIO = runEvalIO' envEmpty stateInitial
  where
    runEvalIO' :: Env -> State -> EvalM a -> IO (Either Error a)
    runEvalIO' _ _ (Pure x) = pure $ pure x
    runEvalIO' r s (Free (ReadOp k)) = runEvalIO' r s $ k r
    runEvalIO' r s (Free (StateGetOp k)) = runEvalIO' r s $ k s
    runEvalIO' r _ (Free (StatePutOp s' m)) = runEvalIO' r s' m
    runEvalIO' r s (Free (PrintOp p m)) = do
      putStrLn p
      runEvalIO' r s m
    runEvalIO' _ _ (Free (ErrorOp e)) = pure $ Left e
```

APL/InterpPure.hs

```
module APL.InterpPure (runEval) where

import APL.Monad

runEval :: EvalM a -> ([String], Either Error a)
runEval = runEval' envEmpty stateInitial
  where
    runEval' :: Env -> State -> EvalM a -> ([String], Either Error a)
    runEval' _ _ (Pure x) = ([], pure x)
    runEval' r s (Free (ReadOp k)) = runEval' r s $ k r
    runEval' r s (Free (StateGetOp k)) = runEval' r s $ k s
    runEval' r _ (Free (StatePutOp s' m)) = runEval' r s' m
    runEval' r s (Free (PrintOp p m)) =
      let (ps, res) = runEval' r s m
       in (p : ps, res)
    runEval' _ _ (Free (ErrorOp e)) = ([], Left e)
```

APL/Interp_Tests.hs

```
module APL.Interp_Tests (tests) where

import APL.AST (Exp (..))
import APL.Eval (eval)
import APL.InterpIO (runEvalIO)
import APL.InterpPure (runEval)
import APL.Monad
import APL.Util (captureIO)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (testCase, (@?=))

eval' :: Exp -> ([String], Either Error Val)
eval' = runEval . eval

evalIO' :: Exp -> IO (Either Error Val)
evalIO' = runEvalIO . eval

tests :: TestTree
tests = testGroup "Free monad interpreters" [pureTests, ioTests]

pureTests :: TestTree
pureTests =
  testGroup
    "Pure interpreter"
    [ testCase "localEnv" $
        runEval
          ( localEnv (const [("x", ValInt 1)]) $
              askEnv
          )
          @?= ([], Right [("x", ValInt 1)]),
      --
      testCase "Let" $
        eval' (Let "x" (Add (CstInt 2) (CstInt 3)) (Var "x"))
          @?= ([], Right (ValInt 5)),
      --
      testCase "Let (shadowing)" $
        eval'
          ( Let
              "x"
              (Add (CstInt 2) (CstInt 3))
              (Let "x" (CstBool True) (Var "x"))
          )
          @?= ([], Right (ValBool True)),
      --
      testCase "State" $
        runEval
          ( do
              putState [(ValInt 0, ValInt 1)]
              modifyState $ map (\(key, _) -> (key, ValInt 5))
              getState
          )
          @?= ([], Right [(ValInt 0, ValInt 5)]),
      --
      testCase "Print" $
        runEval (evalPrint "test")
          @?= (["test"], Right ()),
      --
      testCase "Error" $
        runEval
          ( do
              _ <- failure "Oh no!"
              evalPrint "test"
          )
          @?= ([], Left "Oh no!"),
      --
      testCase "Div0" $
```

```
        eval' (Div (CstInt 7) (CstInt 0))
          @?= ([], Left "Division by zero")
    ]

ioTests :: TestTree
ioTests =
  testGroup
    "IO interpreter"
    [ testCase "print" $ do
        let s1 = "Lalalalala"
            s2 = "Weeeeeeeee"
        (out, res) <-
          captureIO [] $
            runEvalIO $ do
              evalPrint s1
              evalPrint s2
        (out, res) @?= ([s1, s2], Right ())
        -- NOTE: This test will give a runtime error unless you replace the
        -- version of `eval` in `APL.Eval` with a complete version that supports
        -- `Print`-expressions. Uncomment at your own risk.
        -- testCase "print 2" $ do
        --    (out, res) <-
        --      captureIO [] $
        --        evalIO' $
        --          Print "This is also 1" $
        --            Print "This is 1" $
        --              CstInt 1
        --    (out, res) @?= (["This is 1: 1", "This is also 1: 1"], Right $ ValInt 1)
    ]
```

APL/Monad.hs

```
module APL.Monad
  ( envEmpty,
    envExtend,
    envLookup,
    stateInitial,
    askEnv,
    modifyEffects,
    localEnv,
    getState,
    putState,
    modifyState,
    evalKvGet,
    evalKvPut,
    evalPrint,
    failure,
    catch,
    EvalM,
    Val (..),
    EvalOp (..),
    Free (..),
    Error,
    Env,
    State,
  )
where

import APL.AST (Exp (..), VName)
import Control.Monad (ap)

data Val
  = ValInt Integer
  | ValBool Bool
  | ValFun Env VName Exp
  deriving (Eq, Show)

type Error = String

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

type State = [(Val, Val)]

stateInitial :: State
stateInitial = []

data Free e a
  = Pure a
  | Free (e (Free e a))

instance (Functor e) => Functor (Free e) where
  fmap f (Pure x) = Pure $ f x
  fmap f (Free g) = Free $ fmap (fmap f) g

instance (Functor e) => Applicative (Free e) where
  pure = Pure
  (<*>) = ap

instance (Functor e) => Monad (Free e) where
```

```
67    Pure x >>= f = f x
68    Free g >>= f = Free $ h <$> g
69      where
70        h x = x >>= f
71
72  data EvalOp a
73    = ReadOp (Env -> a)
74    | StateGetOp (State -> a)
75    | StatePutOp State a
76    | PrintOp String a
77    | ErrorOp Error
78
79  instance Functor EvalOp where
80    fmap f (ReadOp k) = ReadOp $ f . k
81    fmap f (StateGetOp k) = StateGetOp $ f . k
82    fmap f (StatePutOp s m) = StatePutOp s $ f m
83    fmap f (PrintOp p m) = PrintOp p $ f m
84    fmap _ (ErrorOp e) = ErrorOp e
85
86  type EvalM a = Free EvalOp a
87
88  askEnv :: EvalM Env
89  askEnv = Free $ ReadOp $ \env -> pure env
90
91  modifyEffects :: (Functor e, Functor h) => (e (Free e a) -> h (Free e a)) -> Free e a -> Free h a
92  modifyEffects _ (Pure x) = Pure x
93  modifyEffects g (Free e) = Free $ modifyEffects g <$> g e
94
95  localEnv :: (Env -> Env) -> EvalM a -> EvalM a
96  localEnv f = modifyEffects g
97    where
98      g (ReadOp k) = ReadOp $ k . f
99      g op = op
100
101 getState :: EvalM State
102 getState = Free $ StateGetOp $ \s -> pure s
103
104 putState :: State -> EvalM ()
105 putState s = Free $ StatePutOp s $ pure ()
106
107 modifyState :: (State -> State) -> EvalM ()
108 modifyState f = do
109   s <- getState
110   putState $ f s
111
112 evalPrint :: String -> EvalM ()
113 evalPrint p = Free $ PrintOp p $ pure ()
114
115 failure :: String -> EvalM a
116 failure = Free . ErrorOp
117
118 catch :: EvalM a -> EvalM a -> EvalM a
119 catch = error "To be completed in assignment 4."
120
121 evalKvGet :: Val -> EvalM Val
122 evalKvGet = error "To be completed in assignment 4."
123
124 evalKvPut :: Val -> Val -> EvalM ()
125 evalKvPut = error "To be completed in assignment 4."
```

APL/Util.hs

```
{-# LANGUAGE FlexibleInstances #-}

module APL.Util
  ( Serialize (..),
    newTempDB,
    captureIO,
  )
where

import APL.Monad
import Control.Concurrent (threadDelay)
import Control.Exception (bracket)
import Control.Monad (guard)
import Data.Char (isDigit)
import Data.List (stripPrefix)
import Data.Maybe (mapMaybe)
import GHC.IO.Handle (hDuplicate, hDuplicateTo)
import System.Directory (listDirectory)
import System.IO
import System.Process (createPipe)
import Text.Read (readMaybe)

class Serialize a where
  serialize :: a -> String
  unserialize :: String -> Maybe a

instance Serialize Val where
  serialize (ValInt x) = "ValInt " <> show x
  serialize (ValBool b) = "ValBool " <> show b
  serialize ValFun {} = error "ValFun serialization is not supported."

  unserialize s =
    case words s of
      ["ValInt", rest]
        | all isDigit rest -> ValInt <$> readMaybe rest
      ["ValBool", rest] -> ValBool <$> readMaybe rest
      _ -> Nothing

instance (Serialize a, Serialize b) => Serialize (a, b) where
  serialize (a, b) =
    serialize a ++ "," ++ serialize b
  unserialize s =
    case span (/= ',') s of
      (s_a, _ : s_b) -> (,) <$> unserialize s_a <*> unserialize s_b
      _ -> Nothing

instance Serialize [(Val, Val)] where
  serialize kv =
    unlines $ map serialize kv
  unserialize =
    mapM unserialize . lines

newTempDB :: IO FilePath
newTempDB = do
  files <- listDirectory "."
  let n = maximum (0 : mapMaybe match files) + 1
      tempFile = "temp" ++ show n ++ ".txt"
  writeFile tempFile ""
  pure $ tempFile
  where
    match :: FilePath -> Maybe Int
    match s = do
      s' <- stripPrefix "temp" s
      let (n_s, rest) = (takeWhile isDigit s', dropWhile isDigit s')
      guard $ rest == ".txt"
      readMaybe n_s
```

```
67
68  captureIO :: [String] -> IO a -> IO ([String], a)
69  captureIO inputs m = do
70    hFlush stdout
71    threadDelay 50000 -- Needed to make sure things are actually flushed
72    stdin' <- hDuplicate stdin
73    stdout' <- hDuplicate stdout
74
75    (inR, inW) <- createPipe
76    (outR, outW) <- createPipe
77
78    hSetBuffering inW NoBuffering
79    hSetBuffering outW NoBuffering
80
81    bracket
82      ( do
83          inR `hDuplicateTo` stdin
84          outW `hDuplicateTo` stdout
85      )
86      ( \_ -> do
87          stdin' `hDuplicateTo` stdin
88          stdout' `hDuplicateTo` stdout
89          mapM_ hClose [stdin', stdout', inR, inW, outW]
90      )
91      ( \_ -> do
92          mapM_ (hPutStrLn inW) inputs
93          hFlush inW
94
95          res <- m
96
97          output <- hGetContents outR -- hGetContents closes outR
98          pure (lines output, res)
99      )
```

Advanced Programming 2026

Assignment Four

Free Monads

Due: 2026-10-4

Synopsis: Implementing free monad-based interpreters for a small arithmetic language.

1 Introduction

In this assignment you will implement an interpreter for APL, based on free monads. In particular, you will extend what you developed in the exercises with new effects and new ways of interpreting them. Since the handout for the assignment differs slightly from the solution to the exercises, do **not** base the assignment on your own solutions to the exercises. You **must** use the assignment handout. In particular, the assignment handout removes the `StateGetOp` and `StatePutOp` effects in favour of a more fine grained representation.

For this assignment, you'll be implementing the following:

- Try-catch effects.
- A pure and IO-based interpretation of key-value store effects.
- Interactive missing key recovery.
- A transactional computation effect.
- Breaking out of loops.

For all tasks you are expected to add appropriate tests to the file `Interp_Tests.hs`. For information on how to test the IO-based effects, please refer to the week 4 exercises.[1] Do not rename any of the definitions already present in the handout and do not change their types unless the assignment text explicitly instructs you to do so.

[1] `github.com/diku-dk/ap-e2026/tree/main/week4#testing-runevalio`

To make testing easier, we recommend that you **replace the definition of `eval`** in `APL.Eval` with your version of `eval` from your solution to assignment 2. A complete version of `eval` is not needed to fully solve the assignment; all functionality in this assignment can be tested without using `eval` by constructing appropriate `EvalM` values (directly or by using the interface functions in `APL.Monad`).

2 Tasks

Task 1: The `TryCatchOp` Effect

The `TryCatchOp` effect is for exception handling. A `TryCatchOp m1 m2 k` effect says to interpret `m1`; if it fails (i.e., returns a `Left` value), then interpret `m2`. Finally, pass the value resulting from interpreting `m1` or `m2` to the continuation `k`. For example,

```
> runEval $ Free $ TryCatchOp (failure "Oh no!") (pure $ ValInt 1) pure
([], Right (ValInt 1))
> divZero = CstInt 1 `Div` CstInt 0
> runEval $ eval $ TryCatch (CstInt 5) divZero
([], Right $ ValInt 5)
> badEql = CstInt 0 `Eql` CstBool True
> runEvalIO $ eval $ TryCatch badEql divZero
Left "Division by zero"
```

To start, extend `EvalOp` with a `TryCatchOp` constructor:

```
-- APL.Monad
data EvalOp a
  = ...
  | TryCatchOp (EvalM Val) (EvalM Val) (Val -> a)
```

and extend `EvalOp`'s `Functor` instance appropriately. Also complete the definition of `catch`. You will also need to handle `TryCatchOp` in `localEnv`, by recursively applying `localEnv` to the two `EvalM Val` actions.

Lastly, add support for `TryCatchOp` effects to `runEval'` in `APL.InterpPure` and to `runEvalIO'` in `APL.InterpIO`; `runEval` and `runEvalIO` do not need to treat `TryCatchOp` identically; for example, when interpreting `TryCatchOp m1 m2 k` it's permissible for the effects of `m1` to be visible in `m2` in `runEvalIO` but not visible in `runEval` (or vice versa).

2.1 Task 2: Key-value Store Effects

In this task, you'll add `KvGetOp` and `KvPutOp` operations for reading and storing values from the key-value store, respectively. For example,

```
> put0 m = KvPutOp (ValInt 0) (ValInt 1) m
> get0 = Free $ KvGetOp (ValInt 0) $ \val -> pure val
> runEval $ Free $ put0 get0
([],Right (ValInt 1))
```

Start by extending the `EvalOp` type with the `KvGetOp` and `KvPutOp` constructors:

```
-- APL.Monad
data EvalOp a
  = ...
  | KvGetOp Val (Val -> a)
  | KvPutOp Val Val a
```

then, extend `EvalOp`'s `Functor` instance appropriately. Next, complete the definitions of `evalKvGet` and `evalKvPut` using `KvGetOp` and `KvPutOp`, respectively. Also extend `localEnv` appropriately.

Finally, extend `runEval'` in `APL.InterpPure` to support `KvGetOp` and `KvPutOp` effects as follows:

- On `KvGetOp key k` effects, `runEval'` should lookup the key in the state (the function `lookup` will be useful). If the key is contained in the state with value `val`, continue interpreting on `k val`. Otherwise, fail by returning a `Left` with an appropriate error message.
- On `KvPutOp key val m` effects, `runEval'` should should insert the association `(key, val)` into the state. If the key already exists in the state, it should be replaced by the new association.

Using a Database File for the Key-Value Store

Rather than storing the key-value store in a pure manner and passing state around as a parameter during interpretation (as you did in `runEval`), in `runEvalIO` we can instead write the key-value to a database file on disk.

To keep things simple, the database will just be a simple text file. We need a way to read and write data from the database—the following functions are provided to do so:

```
-- APL.InterpIO
writeDB :: FilePath -> State -> IO ()
readDB :: FilePath -> IO (Either Error State)
```

Note that `readDB` returns an error in the form of a `Left`-expression when trying to read an invalid database; if this happens, your interpreter should simply propagate the error and return the `Left`-expression.

For simplicity, the database can only store `ValInt` and `ValBool` values; function values are **not** supported. Your implementation is **not** expected to handle storing/reading of `ValFun` values to/from the database.

Your task is to modify `runEvalIO'` so that it stores state (i.e., the key-value store) in a database file and to add support for `KvGetOP` and `KvPutOp` effects to `runEvalIO'`.

Specifically, extend `runEvalIO` to support `KvGetOp` and `KvPutOp` effects as follows:

- On `KvGetOp key k` effects, `runEvalIO'` should read the database in from `db` and then lookup the key in the database. As with `runEval`, return `Left` if the key doesn't exist in the database. Otherwise, pass the associated value to `k` and continue.

- On `KvPutOp key val m` effects, `runEvalIO'` should read the database in from the `db` file to get a value of type `State`; let's call this value `dbState`. It should then insert the association `(key, val)` into `dbState` to construct a new state `dbState'`. As with `runEval`, if the key `key` already exists, the corresponding association should be replaced by the new one. Finally, overwrite the `db` database file with `dbState'` by using `writeDB`.[2]

For your testing, remember that the database only supports `ValInt` and `ValBool` values; you do **not** need to add tests for storing/reading `ValFun` values. Also note that `runEvalIO` clears the database on each execution, so database values will **not** persist between invocations of `runEvalIO`.

[2] This isn't a particularly efficient way to insert a key-value pair—we do it this way for simplicity.

Missing keys

Having a database is great and all, but our computations still fail if we look for a key that isn't in the database. Since `runEvalIO` already uses `IO`, instead of failing, we can interactively prompt the user to specify the value of a missing key during interpretation.

Here's an example of how this should work:

```
> runEvalIO $ eval $ KvGet $ CstInt 0
Invalid key: ValInt 0. Enter a replacement: ValInt 5
Right (ValInt 5)
> runEvalIO $ evalKvGet $ ValInt 0
Invalid key: ValInt 0. Enter a replacement: ValBool True
Right (ValBool True)
```

Keys entered this way should *not* be added to the database.

Add support for this functionality to `runEvalIO'`. To do so, you'll need to modify how `runEvalIO'` interprets `KvGetOp key k` effects. When the key `key` doesn't exist in the database file, your implementation must print a message to the terminal saying that the given key is invalid and then prompt the user for a replacement. The entered replacement should either be a `ValInt` or a `ValBool`. All other inputs (notably, `ValFun` values) are **not** supported. To convert the input string into a `Val`, use the provided `readVal :: String -> Maybe Val` function in `APL.InterpIO`. When the input is invalid (i.e., not of the form specified above), `readVal` will return `Nothing`; in this case, your interpreter should fail with an appropriate message:

```
> runEvalIO $ eval $ KvGet $ CstInt 0
Invalid key: ValInt 0. Enter a replacement: lol
Left "Invalid value input: lol"
```

Hint: You can use the `prompt :: String -> IO String` function defined in `APL.InterpIO` to get input from the user.
Hint 2: To test your missing key handling, you can use `captureIO` to simulate input like so:

```
-- APL.Interp_Tests
testCase "Missing key test" $ do
  (_, res) <-
```

```
    captureIO ["ValInt 1"] $
      runEvalIO $
        Free $ KvGetOp (ValInt 0) $ \val -> pure val
  res @?= Right (ValInt 1),
```

Task 3: `TransactionOp` effect

Another thing we might want for our database is to have *transactional* or *atomic* writes to it. First, extend APL with a `Transaction` expression:

```
-- APL.AST
data Exp
  = ...
  | Transaction Exp
```

The semantics of evaluating `Transaction e` is to wrap evaluation of `e` within a transaction as defined below (and using `APL.Monad.transaction`). The result of evaluation is otherwise the result of evaluating `e`.

Next add the `TransactionOp` effect to the `EvalOp` type:

```
-- APL.Monad
data EvalOp a
  = ...
  | TransactionOp (EvalM Val) (Val -> a)
```

Notice that it has an `EvalM Val` payload—when this payload is interpreted, any effects that **change the state (i.e., the key-value store)** should be all-or-nothing: that is, they should only be manifested if the computation succeeds (i.e, it returns result `Right v`). If the computation fails (returns `Left e`), then the **state (i.e., the key-value store)** should be rolled back to the point it was at before the payload was executed. For example,

```
> goodPut = KvPut (CstInt 0) (CstInt 1)
> badPut = Let "_" (KvPut (CstInt 0) (CstBool False)) (Var "die")
> get0 = KvGet (CstInt 0)
> runEval $ eval $ Let "_" (Transaction goodPut) get0
([],Right (ValInt 1))
> runEval $ eval $ TryCatch (Transaction badPut) get0
([],Left "Invalid key: ValInt 0")
```

Note that if the enclosed computation fails, the error should be propagated. For example,

```
> runEval $ eval $ Transaction badPut
([],Left "Unknown variable: die")
```

Extend the `Functor` instance of `EvalOp` to support `TransactionOp` and, using `TransactionOp`, complete the definition of the `transaction` function. Also extend `localEnv` appropriately.

As usual, extend `runEval'` with support for `TransactionOp` effects. To do so, you should only keep changes to the **state (i.e., the key-value store)** from executing the enclosed `EvalM Val` computation if it succeeds; otherwise continue execution with the prior state. You **must** include the result from any `PrintOp` effects that occurred before the failure (in the transactional computation) in the final output, regardless of whether or not the transactional computation succeeded:

```
> runEval $ transaction (evalPrint "weee" >> failure "oh shit")
(["weee"],Left "oh shit")
```

You must also correctly handle **nested transactions**. For example,

```
> runEval $ eval $ Let "_" (Transaction
                             (Let "_" goodPut
                              (TryCatch (Transaction badPut)
                                        (CstBool True))))
                   get0
([],Right (ValInt 1))
> runEval $ eval $ Let "_" (TryCatch (Transaction
                                       (Transaction badPut))
                                     (CstBool True))
                   get0
([],Left "Invalid key: ValInt 0")
```

Next, add support for `TransactionOp` to `runEvalIO'`. You should only manifest writes to the database if the whole transactional computation succeeds. To do this, before execution of the transactional computation, make a temporary copy of the database file. To make temporary databases, use the `withTempDB` function, which creates a fresh temporary database, passes it to a function returning an `IO`-computation, executes

the computation, deletes the temporary database, and finally returns the result of the computation:

```
-- APL.InterpIO
withTempDB :: (FilePath -> IO a) -> IO a
withTempDB m = do
  tempDB <- newTempDB -- Create a new temp database file.
  res <- m tempDB     -- Run the computation with the new file.
  removeFile tempDB   -- Delete the temp database file.
  pure res            -- Return the result of the computation.
```

Note that `newTempDB` ensures that the database `tempDB` is fresh; i.e. that there are no other files named `tempDB`. You **must** use `withTempDB` to create temporary databases; **do not** use `newTempDB`. To copy a database, use the `copyDB` function:

```
-- APL.InterpIO
copyDB :: FilePath -> FilePath -> IO ()
```

During execution of the transactional computation, perform all writes and reads on the temporary database file. If the computation succeeds, subsequently copy the temporary database to the actual database and continue interpreting. If it fails, discard the temporary database and propagate the error. As before, you must also correctly handle nested transactions. **Hint:** The function you pass `withTempDB` should call `copyDB` to copy the database to the temporary database. If the transactional computation succeeds, you will have to call `copyDB` again to copy the temporary database back.

Task 4: Breaking out of loops

In this task you will implement support for breaking out of loops, similar to the `break` statement in C. In contrast with the previous tasks, the main challenge here is that you have to design the representation of the necessary effects yourself.

First, add the following constructor to `Exp`:

```
-- APL.AST
data Exp
  = ...
  | Break Exp
```

The intended semantics of evaluating `Break e` is to first evaluate `e` to a value v, then immediately return from the innermost enclosing loop with the value v.

To do this, extend the evaluation function in `APL.Eval`. You will need to add a new case for `Break` and to modify the case for `ForLoop`. You should make use of the functions `looping` and `breakLoop` from `APL.Monad`, which are intended to be the the entry points for the effects.

Then add corresponding new effects to `EvalOp` and implement `looping` and `breakLoop` to use them. Also extend `localEnv` appropriately.

Finally, modify the interpretation functions in `APL.InterpPure` and `APL.InterpIO` to handle the new effects you have added. These implementations will likely be quite similar.

Hints

The operational behaviour of breaking is quite similar how errors are propagated and and handled by `TryCatch`, and your implementation can be fairly similar.

Examples

```
> runEval $ eval $
  ForLoop ("p", CstInt 0) ("i", CstInt 100) $
  Let "_" (Break (CstBool True)) (Var "i")
([],Right (ValBool True))
> runEval $ eval $ Break (CstBool True)
([],Left "Break␣outside␣loop")
```

3 Code handout

The code handout consists of the following nontrivial files.

- `a4.cabal`: Cabal build file. **Do not modify this file.**
- `runtests.hs`: Test runner. **Do not modify this file.**
- `src/APL/AST.hs`: AST definition, which you will need to extend slightly.

- `src/APL/Eval.hs`: An incomplete evaluator corresponding to the solution to the week 2 exercises. You should **replace the definition of `eval`** with your complete version of `eval` from your solution to assignment 2.
- `src/APL/InterpIO.hs`: Contains the incomplete IO-based `runEvalIO` interpreter.
- `src/APL/InterpPure.hs`: Contains the incomplete pure `runEval` interpreter.
- `src/APL/Interp_Tests.hs`: An interpreter test suite where you will add plentiful tests.
- `src/APL/Monad.hs`: Contains all things related to the evaluation monad. Note that some definitions from assignment 2 have moved from `APL.Eval` to `APL.Monad` in this assignment; e.g. `Val` and definitions related to the environment.
- `src/APL/Util.hs`: Utility functions needed for serialization, testing IO, and making temporary database files. You can safely ignore this file. **Do not modify this file.**

4 Your Report

You are expected to comment on the *interesting* details of your implementation. You are *not* expected to give a line-by-line walkthrough of your code. Most importantly, you are expected to reflect on the *quality* of your code:

- Do you think it is functionally correct? Why or why not?
- Is there some improvement you'd have liked to make, but didn't have the time?

It is more important to be aware of the strengths or shortcomings of your solution, than it is to have a complete solution.

4.1 The structure of your report

Your report must be structured exactly as follows:

Introduction: Briefly mention general concerns, any ambiguities in the problem text and how you resolved them, and your own estimation of the quality of your solution. Briefly mention whether your solution is functional, which test cases cover its functionality, which test cases it fails for (if any), and what you think might be wrong.

A section answering the following numbered questions:

1. Consider interpreting a `TryCatchOp m1 m2 k` effect where `m1` fails after performing some key-value store effects.
   (a) Is there a difference between your pure interpreter and your IO-based interpreter in terms of whether the key-value store effects that `m1` performed before it failed are visible when interpreting `m2`? If so, why?
   (b) Suppose you've implemented your interpreters such that the key-value store effects that `m1` performed before it failed are always **visible** when interpreting `m2`. **Without changing the interpreters**, is it possible to have different behavior where the key-value store effects in `m1` are **invisible** in `m2`? If so, how? If not, why not?
2. Why is `TransactionOp` not defined as follows?

   ```
   data EvalOp a
    = ...
    | TransactionOp (EvalM ()) a
   ```

   What problems might arise with this definition? Would it make `TransactionOp` completely useless?
3. Why is `TryCatchOp` not defined as follows?

   ```
   data EvalOp a
     = ...
     | TryCatchOp a a
   ```

   What problems might arise with this definition?

All else being equal, **a short report is a good report**.

5 Deliverables for This Assignment

You must submit the following items:

- A single PDF file, A4 size, no more than 5 pages, structured as specified in the report section above.
- A single zip/tar.gz file with all code relevant to the implementation, including at least all the files from the handout. For this assignment it is not necessary to add additional files.

Remember to follow the general assignment rules listed on the course homepage.

6 Assessment

You will get written qualitative feedback, and points from zero to four. There are no resubmissions, so please hand in what you managed to do, even if you have not solved the assignment completely.

A4 handout

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  | ForLoop (VName, Exp) (VName, Exp) Exp
  | Lambda VName Exp
  | Apply Exp Exp
  | TryCatch Exp Exp
  | Print String Exp
  | KvPut Exp Exp
  | KvGet Exp
  deriving (Eq, Show)
```

APL/Eval.hs

```
module APL.Eval
  ( eval,
  )
where

import APL.AST (Exp (..))
import APL.Monad

evalIntBinOp :: (Integer -> Integer -> EvalM Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp f e1 e2 = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValInt x, ValInt y) -> ValInt <$> f x y
    (_, _) -> failure "Non-integer operand"

evalIntBinOp' :: (Integer -> Integer -> Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp' f e1 e2 =
  evalIntBinOp f' e1 e2
  where
    f' x y = pure $ f x y

-- Replace with your 'eval' from your solution to assignment 2.
eval :: Exp -> EvalM Val
eval (CstInt x) = pure $ ValInt x
eval (CstBool b) = pure $ ValBool b
eval (Var v) = do
  env <- askEnv
  case envLookup v env of
    Just x -> pure x
    Nothing -> failure $ "Unknown variable: " ++ v
eval (Add e1 e2) = evalIntBinOp' (+) e1 e2
eval (Sub e1 e2) = evalIntBinOp' (-) e1 e2
eval (Mul e1 e2) = evalIntBinOp' (*) e1 e2
eval (Div e1 e2) = evalIntBinOp checkedDiv e1 e2
  where
    checkedDiv _ 0 = failure "Division by zero"
    checkedDiv x y = pure $ x `div` y
eval (Pow e1 e2) = evalIntBinOp checkedPow e1 e2
  where
    checkedPow x y =
      if y < 0
        then failure "Negative exponent"
        else pure $ x ^ y
eval (Eql e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValInt x, ValInt y) -> pure $ ValBool $ x == y
    (ValBool x, ValBool y) -> pure $ ValBool $ x == y
    (_, _) -> failure "Invalid operands to equality"
eval (If cond e1 e2) = do
  cond' <- eval cond
  case cond' of
    ValBool True -> eval e1
    ValBool False -> eval e2
    _ -> failure "Non-boolean conditional."
eval (Let var e1 e2) = do
  v1 <- eval e1
  localEnv (envExtend var v1) $ eval e2
eval (ForLoop (loopparam, initial) (iv, bound) body) = do
  initial_v <- eval initial
  bound_v <- eval bound
  case bound_v of
    ValInt bound_int ->
      loop 0 bound_int initial_v
```

```
    _ ->
      failure "Non-integral loop bound"
  where
    loop i bound_int loop_v
      | i >= bound_int = pure loop_v
      | otherwise = do
          loop_v' <-
            localEnv (envExtend iv (ValInt i) . envExtend loopparam loop_v) $
              eval body
          loop (succ i) bound_int loop_v'
eval (Lambda var body) = do
  env <- askEnv
  pure $ ValFun env var body
eval (Apply e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValFun f_env var body, arg) ->
      localEnv (const $ envExtend var arg f_env) $ eval body
    (_, _) ->
      failure "Cannot apply non-function"
eval (TryCatch e1 e2) =
  eval e1 `catch` eval e2
```

APL/InterpIO.hs

```
module APL.InterpIO (runEvalIO) where

import APL.Monad
import APL.Util
import System.Directory (removeFile)
import System.IO (hFlush, readFile', stdout)

-- Converts a string into a value. Only 'ValInt's and 'ValBool' are supported.
readVal :: String -> Maybe Val
readVal = unserialize

-- 'prompt s' prints 's' to the console and then reads a line from stdin.
prompt :: String -> IO String
prompt s = do
  putStr s
  hFlush stdout
  getLine

-- 'writeDB dbFile s' writes the 'State' 's' to the file 'db'.
writeDB :: FilePath -> State -> IO ()
writeDB db s =
  writeFile db $ serialize s

-- 'readDB db' reads the database stored in 'db'.
readDB :: FilePath -> IO (Either Error State)
readDB db = do
  ms <- readFile' db
  case unserialize ms of
    Just s -> pure $ pure s
    Nothing -> pure $ Left "Invalid DB."

-- 'copyDB db1 db2' copies 'db1' to 'db2'.
copyDB :: FilePath -> FilePath -> IO ()
copyDB db db' = do
  s <- readFile' db
  writeFile db' s

-- Removes all key-value pairs from the database file.
clearDB :: IO ()
clearDB = writeFile dbFile ""

-- The name of the database file.
dbFile :: FilePath
dbFile = "db.txt"

-- Creates a fresh temporary database, passes it to a function returning an
-- IO-computation, executes the computation, deletes the temporary database, and
-- finally returns the result of the computation. The temporary database file is
-- guaranteed fresh and won't have a name conflict with any other files.
withTempDB :: (FilePath -> IO a) -> IO a
withTempDB m = do
  tempDB <- newTempDB -- Create a new temp database file.
  res <- m tempDB -- Run the computation with the new file.
  removeFile tempDB -- Delete the temp database file.
  pure res -- Return the result of the computation.

runEvalIO :: EvalM a -> IO (Either Error a)
runEvalIO evalm = do
  clearDB
  runEvalIO' envEmpty dbFile evalm
  where
    runEvalIO' :: Env -> FilePath -> EvalM a -> IO (Either Error a)
    runEvalIO' _ _ (Pure x) = pure $ pure x
    runEvalIO' r db (Free (ReadOp k)) = runEvalIO' r db $ k r
    runEvalIO' r db (Free (PrintOp p m)) = do
      putStrLn p
```

```
67        runEvalIO' r db m
68      runEvalIO' _ _ (Free (ErrorOp e)) = pure $ Left e
```

APL/InterpPure.hs

```
module APL.InterpPure (runEval) where

import APL.Monad

runEval :: EvalM a -> ([String], Either Error a)
runEval = runEval' envEmpty stateInitial
  where
    runEval' :: Env -> State -> EvalM a -> ([String], Either Error a)
    runEval' _ _ (Pure x) = ([], pure x)
    runEval' r s (Free (ReadOp k)) = runEval' r s $ k r
    runEval' r s (Free (PrintOp p m)) =
      let (ps, res) = runEval' r s m
       in (p : ps, res)
    runEval' _ _ (Free (ErrorOp e)) = ([], Left e)
```

APL/Interp_Tests.hs

```
module APL.Interp_Tests (tests) where

import APL.AST (Exp (..))
import APL.Eval (eval)
import APL.InterpIO (runEvalIO)
import APL.InterpPure (runEval)
import APL.Monad
import APL.Util (captureIO)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (testCase, (@?=))

eval' :: Exp -> ([String], Either Error Val)
eval' = runEval . eval

evalIO' :: Exp -> IO (Either Error Val)
evalIO' = runEvalIO . eval

tests :: TestTree
tests = testGroup "Free monad interpreters" [pureTests, ioTests]

pureTests :: TestTree
pureTests =
  testGroup
    "Pure interpreter"
    [ testCase "localEnv" $
        runEval
          ( localEnv (const [("x", ValInt 1)]) $
              askEnv
          )
          @?= ([], Right [("x", ValInt 1)]),
      --
      testCase "Let" $
        eval' (Let "x" (Add (CstInt 2) (CstInt 3)) (Var "x"))
          @?= ([], Right (ValInt 5)),
      --
      testCase "Let (shadowing)" $
        eval'
          ( Let
              "x"
              (Add (CstInt 2) (CstInt 3))
              (Let "x" (CstBool True) (Var "x"))
          )
          @?= ([], Right (ValBool True)),
      --
      testCase "Print" $
        runEval (evalPrint "test")
          @?= (["test"], Right ()),
      --
      testCase "Error" $
        runEval
          ( do
              _ <- failure "Oh no!"
              evalPrint "test"
          )
          @?= ([], Left "Oh no!"),
      --
      testCase "Div0" $
        eval' (Div (CstInt 7) (CstInt 0))
          @?= ([], Left "Division by zero")
    ]

ioTests :: TestTree
ioTests =
  testGroup
    "IO interpreter"
    [ testCase "print" $ do
```

```
67          let s1 = "Lalalalala"
68              s2 = "Weeeeeeeee"
69          (out, res) <-
70            captureIO [] $
71              runEvalIO $ do
72                evalPrint s1
73                evalPrint s2
74          (out, res) @?= ([s1, s2], Right ())
75          -- NOTE: This test will give a runtime error unless you replace the
76          -- version of `eval` in `APL.Eval` with a complete version that supports
77          -- `Print`-expressions. Uncomment at your own risk.
78          -- testCase "print 2" $ do
79          --    (out, res) <-
80          --      captureIO [] $
81          --        evalIO' $
82          --          Print "This is also 1" $
83          --            Print "This is 1" $
84          --              CstInt 1
85          --    (out, res) @?= (["This is 1: 1", "This is also 1: 1"], Right $ ValInt 1)
86      ]
```

APL/Monad.hs

```
module APL.Monad
  ( envEmpty,
    envExtend,
    envLookup,
    stateInitial,
    askEnv,
    modifyEffects,
    localEnv,
    evalPrint,
    catch,
    failure,
    evalKvGet,
    evalKvPut,
    transaction,
    looping,
    breakLoop,
    EvalM,
    Val (..),
    EvalOp (..),
    Free (..),
    Error,
    Env,
    State,
  )
where

import APL.AST (Exp (..), VName)
import Control.Monad (ap)

data Val
  = ValInt Integer
  | ValBool Bool
  | ValFun Env VName Exp
  deriving (Eq, Show)

type Error = String

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

type State = [(Val, Val)]

stateInitial :: State
stateInitial = []

data Free e a
  = Pure a
  | Free (e (Free e a))

instance (Functor e) => Functor (Free e) where
  fmap f (Pure x) = Pure $ f x
  fmap f (Free g) = Free $ fmap (fmap f) g

instance (Functor e) => Applicative (Free e) where
  pure = Pure
  (<*>) = ap

instance (Functor e) => Monad (Free e) where
```

```
  Pure x >>= f = f x
  Free g >>= f = Free $ h <$> g
    where
      h x = x >>= f

data EvalOp a
  = ReadOp (Env -> a)
  | PrintOp String a
  | ErrorOp Error

instance Functor EvalOp where
  fmap f (ReadOp k) = ReadOp $ f . k
  fmap f (PrintOp p m) = PrintOp p $ f m
  fmap _ (ErrorOp e) = ErrorOp e

type EvalM a = Free EvalOp a

askEnv :: EvalM Env
askEnv = Free $ ReadOp $ \env -> pure env

modifyEffects ::
  (Functor e, Functor h) =>
  (e (Free e a) -> h (Free e a)) ->
  Free e a ->
  Free h a
modifyEffects _ (Pure x) = Pure x
modifyEffects g (Free e) = Free $ modifyEffects g <$> g e

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f = modifyEffects g
  where
    g (ReadOp k) = ReadOp $ k . f
    -- TODO: add cases for TryCatchOp, TransactionOp, and as necessary for the
    -- effects you add for looping.
    g op = op

evalPrint :: String -> EvalM ()
evalPrint p = Free $ PrintOp p $ pure ()

failure :: String -> EvalM a
failure = Free . ErrorOp

catch :: EvalM Val -> EvalM Val -> EvalM Val
catch = error "TODO"

evalKvGet :: Val -> EvalM Val
evalKvGet = error "TODO"

evalKvPut :: Val -> Val -> EvalM ()
evalKvPut = error "TODO"

transaction :: EvalM Val -> EvalM Val
transaction = error "TODO"

-- | Enclose a computation @m@ such that if a 'breakLoop' is executed in @m@,
-- execution will return here.
looping :: EvalM Val -> EvalM Val
looping = error "TODO"

-- | Return the provided value from the most immediately enclosing 'looping'.
breakLoop :: Val -> EvalM a
breakLoop = error "TODO"
```

APL/Util.hs

```
{-# LANGUAGE FlexibleInstances #-}

module APL.Util
  ( Serialize (..),
    newTempDB,
    captureIO,
  )
where

import APL.Monad
import Control.Concurrent (threadDelay)
import Control.Exception (bracket)
import Control.Monad (guard)
import Data.Char (isDigit)
import Data.List (stripPrefix)
import Data.Maybe (mapMaybe)
import GHC.IO.Handle (hDuplicate, hDuplicateTo)
import System.Directory (listDirectory)
import System.IO
import System.Process (createPipe)
import Text.Read (readMaybe)

class Serialize a where
  serialize :: a -> String
  unserialize :: String -> Maybe a

instance Serialize Val where
  serialize (ValInt x) = "ValInt " <> show x
  serialize (ValBool b) = "ValBool " <> show b
  serialize ValFun {} = error "ValFun serialization is not supported."

  unserialize s =
    case words s of
      ["ValInt", rest]
        | all isDigit rest -> ValInt <$> readMaybe rest
      ["ValBool", rest] -> ValBool <$> readMaybe rest
      _ -> Nothing

instance (Serialize a, Serialize b) => Serialize (a, b) where
  serialize (a, b) =
    serialize a ++ "," ++ serialize b
  unserialize s =
    case span (/= ',') s of
      (s_a, _ : s_b) -> (,) <$> unserialize s_a <*> unserialize s_b
      _ -> Nothing

instance Serialize [(Val, Val)] where
  serialize kv =
    unlines $ map serialize kv
  unserialize =
    mapM unserialize . lines

newTempDB :: IO FilePath
newTempDB = do
  files <- listDirectory "."
  let n = maximum (0 : mapMaybe match files) + 1
      tempFile = "temp" ++ show n ++ ".txt"
  writeFile tempFile ""
  pure $ tempFile
  where
    match :: FilePath -> Maybe Int
    match s = do
      s' <- stripPrefix "temp" s
      let (n_s, rest) = (takeWhile isDigit s', dropWhile isDigit s')
      guard $ rest == ".txt"
      readMaybe n_s
```

```
67
68  captureIO :: [String] -> IO a -> IO ([String], a)
69  captureIO inputs m = do
70    hFlush stdout
71    threadDelay 50000 -- Needed to make sure things are actually flushed
72    stdin' <- hDuplicate stdin
73    stdout' <- hDuplicate stdout
74
75    (inR, inW) <- createPipe
76    (outR, outW) <- createPipe
77
78    hSetBuffering inW NoBuffering
79    hSetBuffering outW NoBuffering
80
81    bracket
82      ( do
83          inR `hDuplicateTo` stdin
84          outW `hDuplicateTo` stdout
85      )
86      ( \_ -> do
87          stdin' `hDuplicateTo` stdin
88          stdout' `hDuplicateTo` stdout
89          mapM_ hClose [stdin', stdout', inR, inW, outW]
90      )
91      ( \_ -> do
92          mapM_ (hPutStrLn inW) inputs
93          hFlush inW
94
95          res <- m
96
97          output <- hGetContents outR -- hGetContents closes outR
98          pure (lines output, res)
99      )
```

Week 5 exercises

Week 5 - Property-based testing

This week we will look at testing based on properties. It also serves as an example of using monads.

Slides and Material

- Slides on property-based testing, and the associated Haskell code, will be published Tuesday evening.

Suggested Reading

- QuickCheck Testing for Fun and Profit
- QuickCheck manual
- Course Notes Chapter 5

Going Beyond

- How to Specify it - Guide to writing properties of pure functions
- The sad state of property-based testing libraries

Exercises

In these exercises you will develop QuickCheck generators for expressions and state some simple properties.

Your starting point is found in handout/ and complete solutions are in solution/. For some of the tasks, partial or complete solutions are also included inline below.

Note that the handout evaluator does not satisfy all of the properties you will state below. Part of your task is to discover this by testing, and then to fix it.

Code Overview

The handout code is an amalgam of previous weeks, with some modifications. In particular: * `APL.AST` contains the familiar type of expressions. It has an additional function `subExp` for enumerating the subexpressions of an expression. * `APL.Error` is a new module defining a type of errors, which we will use instead of strings when reporting errors. * `APL.Eval` is the evaluator from week 2, modified to use the new error type. * `APL.Check` is a new module containing a checker for APL, whose `checkExp` returns a *list* of the errors that evaluating the expression might produce. * `APL.Tests` is a new module where you will be using QuickCheck.

A Basic Generator

We will start by making a simple generator that generates a small subset of expressions, namely lambda expressions and variables.

1. Define `genVar :: Gen VName` so that it generates valid variable names (i.e. those accepted by the grammar). You do not need to be able to generate all possible variable names.
2. Define `genExp :: Gen Exp` so that it generates expressions built using the `Lambda` and `Var` constructors.
3. Use `sample` from QuickCheck to see examples of output from `genVar` and `genExp`.

Hint You might find the following QuickCheck functions useful: `elements`, `oneof`, `listOf`.

Solution Open this to see the answer

```
genVar :: Gen VName
genVar = do
    alpha <- elements ['a' .. 'z']
    alphaNums <- listOf $ elements $ ['a' .. 'z'] ++ ['0' .. '9']
    pure (alpha : alphaNums)

genExp :: Gen Exp
genExp = oneof [Var <$> genVar, Lambda <$> genVar <*> genExp]
```

Then run `sample genVar` and `sample genExp` in ghci.

Sized Generation

Extend `genExp` so that it can also produce `Apply` expressions. Try out the generator again using `sample`. What is the problem?

In order to fix this problem `genExp` takes a parameter that specifies the size of its output. Change the type signature to `genExp :: Int -> Gen Exp`. The intention is that `genExp size` must generate expressions with *at most* `size` constructors in them. For example, `Apply (Apply (Var "f") (Var "x")) (Var "y")` has a total of 5 constructors.

Using `sized :: (Int -> Gen a) -> Gen a` from QuickCheck you now have a better expression generator, `sized genExp`. Try sampling again; is the problem solved?

Hints Note that generated expressions are allowed to be smaller than `size`, but ideally expressions with exactly `size` constructors should be generated at least occasionally.

There is more than one reasonable approach to implementing this, so there is no need to worry if your proposed solution does not seem like the "right" one.

Solution Open this to see the answer

```
genExp :: Int -> Gen Exp
genExp size =
  if size <= 1
  then Var <$> genVar
  else
    let half = (size - 1) `div` 2
    in oneof
      [ Var <$> genVar
      , Lambda <$> genVar <*> genExp (size - 1)
      , Apply <$> genExp half <*> genExp half
      ]
```

Then run `sample (sized genExp)` in ghci.

Completing the Generator

Extend `genExp` so that it can produce all possible expressions. Declare `Exp` to be an instance of `Arbitrary` with `arbitrary = sized genExp`.

Associativity

Addition of integers obeys associativity, meaning that `(n1 + n2) + n3 == n1 + (n2 + n3)` for all integers `n1`, `n2` and `n3`. Declare a function `prop_integerAddAssoc :: Integer -> Integer -> Integer -> Bool` in `APL.Tests` which expresses this fact. Then run `quickCheck prop_integerAddAssoc` to test this.

What about addition in APL? Do `Add (Add e1 e2) e3` and `Add e1 (Add e2 e3)` always evaluate to the same result? Declare a function `prop_aplAddAssoc :: Exp -> Exp -> Exp -> Bool` which expresses this hypothesis.

Solution Open this to see the answer

```
prop_integerAddAssoc :: Integer -> Integer -> Integer -> Bool
prop_integerAddAssoc n1 n2 n3 = (n1 + n2) + n3 == n1 + (n2 + n3)

prop_aplAddAssoc :: Exp -> Exp -> Exp -> Bool
prop_aplAddAssoc e1 e2 e3 = runEval (eval (Add (Add e1 e2) e3)) == runEval (eval (Add e1 (Add e2 e3)))
```

Running `quickCheck prop_aplAddAssoc` should generate a counterexample to the property.

Shrinking

Many of the counterexamples for `prop_aplAddAssoc` will be somewhat large, making it hard to spot the issue. In order to address this we will implement *shrinking* for `Exp` by giving a definition for `shrink` in the `Arbitrary` instance for `Exp`.

Given an expression `e` we want `shrink e` to return a list of possible *shrinks* of `e`, i.e. expressions that resemble `e` but are smaller. In general the possible shrinks of an expression should include its direct subexpressions as

well as a version of the expression where exactly one of its arguments has been shrunk. For instance `Add e1 e2` shrinks to any of

- `e1`
- `e2`
- `Add e1' e2` when `e1` shrinks to `e1'`
- `Add e1 e2'` when `e2` shrinks to `e2'`

Arguments of types other than `Exp` simply use their respective `shrink` methods. For instance `Var name` shrinks to `Var name'` when `name` shrinks to `name'`.

However, shrinking should not produce expressions that could not have been generated in the first place, such as `Var ""`.

Implement `shrink` in the `Arbitrary` instance for `Exp` using these rules.

Solution (partial) Open this to see the answer

```
shrink (Add e1 e2) =
  e1 : e2 : [Add e1' e2 | e1' <- shrink e1] ++ [Add e1 e2' | e2' <- shrink e2]
shrink (If cond e1 e2) =
  e1 : e2 : [If cond' e1 e2 | cond' <- shrink cond] ++ [If cond e1' e2 | e1' <- shrink e1] ++ [If cond e1 e2' |
  ↪  e2' <- shrink e2]
shrink (Var x) =
  [Var x' | x' <- shrink x, not (null x')]
shrink (Let x e1 e2) =
  e1 : [Let x e1' e2 | e1' <- shrink e1] ++ [Let x e1 e2' | e2' <- shrink e2]
shrink (Lambda x e) =
  e : [Lambda x e' | e' <- shrink e]
```

Running `quickCheck prop_aplAddAssoc` should now generate a smaller counterexample to the property.

Fixing Associativity

Suppose that we *do* want associativity of addition in APL. Guaranteeing the validity of equational properties such as associativity, commutativity, inlining and the monad laws makes it possible for users, tools and compilers to reason about and optimise programs effectively and efficiently. Alas, many programming languages happen not to be *designed* that way, and neither is ours. But let us pretend for a moment that we do care. Modify the evaluator so that `prop_aplAddAssoc` passes.

Solution (partial) Open this to see the answer

The most straightforward way is to change `evalIntBinOp` to the following:

```
evalIntBinOp f e1 e2 = do
  v1 <- eval e1
  case v1 of
    ValInt x -> do
      v2 <- eval e2
      case v2 of
        ValInt y -> ValInt <$> f x y
        _ -> failure NonInteger
    _ -> failure NonInteger
```

Even better would be to abstract out the notion of extracting the integer from `ValInt` (or failing) into its own function.

Week 5 solutions

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
    printExp,
    subExp,
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  | Lambda VName Exp
  | Apply Exp Exp
  | TryCatch Exp Exp
  deriving (Eq, Show)

parens :: String -> String
parens x = "(" ++ x ++ ")"

printBinOp :: String -> Exp -> Exp -> String
printBinOp op x y = parens $ printExp x ++ " " ++ op ++ " " ++ printExp y

printExp :: Exp -> String
printExp (CstInt x) = show x
printExp (CstBool b) = if b then "true" else "false"
printExp (Add x y) = printBinOp "+" x y
printExp (Sub x y) = printBinOp "-" x y
printExp (Mul x y) = printBinOp "*" x y
printExp (Div x y) = printBinOp "/" x y
printExp (Pow x y) = printBinOp "**" x y
printExp (Eql x y) = printBinOp "==" x y
printExp (If x y z) =
  parens $
    "if "
      ++ printExp x
      ++ " then "
      ++ printExp y
      ++ " else "
      ++ printExp z
printExp (Var v) = v
printExp (Let v e1 e2) =
  parens $
    "let "
      ++ v
      ++ " = "
      ++ printExp e1
      ++ " in "
      ++ printExp e2
printExp (Lambda v body) =
  parens $ "\\" ++ v ++ " -> " ++ printExp body
printExp (Apply x y) =
  printExp x ++ " " ++ printExp y
printExp (TryCatch x y) =
  "try " ++ printExp x ++ " catch " ++ printExp y

```

```
67  subExp :: Exp -> [Exp]
68  subExp e =
69    e : case e of
70      CstInt _ -> []
71      CstBool _ -> []
72      Add e1 e2 -> subExp e1 ++ subExp e2
73      Sub e1 e2 -> subExp e1 ++ subExp e2
74      Mul e1 e2 -> subExp e1 ++ subExp e2
75      Div e1 e2 -> subExp e1 ++ subExp e2
76      Pow e1 e2 -> subExp e1 ++ subExp e2
77      Eql e1 e2 -> subExp e1 ++ subExp e2
78      If e0 e1 e2 -> subExp e0 ++ subExp e1 ++ subExp e2
79      Var _ -> []
80      Let _ e1 e2 -> subExp e1 ++ subExp e2
81      Lambda _ body -> subExp body
82      Apply e1 e2 -> subExp e1 ++ subExp e2
83      TryCatch e1 e2 -> subExp e1 ++ subExp e2
```

APL/Check.hs

```
module APL.Check (checkExp, Error) where

import APL.AST (Exp (..), VName)
import APL.Error (Error (..))
import Control.Monad (ap, liftM, unless)
import Data.List (union)

type Vars = [VName]

newtype CheckM a = CheckM {runCheckM :: Vars -> (a, [Error])}

instance Functor CheckM where
  fmap = liftM

instance Applicative CheckM where
  (<*>) = ap
  pure x = CheckM $ \_ -> (x, [])

instance Monad CheckM where
  CheckM x >>= f = CheckM $ \vars ->
    let (y, errs1) = x vars
        (z, errs2) = runCheckM (f y) vars
     in (z, union errs1 errs2)

askVars :: CheckM Vars
askVars = CheckM $ \vars -> (vars, [])

localVars :: (Vars -> Vars) -> CheckM a -> CheckM a
localVars f m = CheckM $ \vars ->
  runCheckM m (f vars)

failure :: Error -> CheckM ()
failure err = CheckM $ \_ -> ((), [err])

maskErrors :: CheckM a -> CheckM a
maskErrors m = CheckM $ \vars ->
  let (x, _) = runCheckM m vars in (x, [])

check :: Exp -> CheckM ()
check (CstInt _) = pure ()
check (CstBool _) = pure ()
check (Var v) = do
  vars <- askVars
  unless (v `elem` vars) $
    failure $
      UnknownVariable v
check (Add x y) = do
  failure NonInteger
  check x
  check y
check (Sub x y) = do
  failure NonInteger
  check x
  check y
check (Mul x y) = do
  failure NonInteger
  check x
  check y
check (Div x y) = do
  failure NonInteger
  failure DivisionByZero
  check x
  check y
check (Pow x y) = do
  failure NonInteger
  failure NegativeExponent
```

```
67    check x
68    check y
69  check (Eql x y) = do
70    failure InvalidEqual
71    check x
72    check y
73  check (If x y z) = do
74    failure NonBoolean
75    check x
76    check y
77    check z
78  check (Let v e1 e2) = do
79    check e1
80    localVars (v :) $ check e2
81  check (Lambda v e) = do
82    localVars (v :) $ check e
83  check (Apply x y) = do
84    failure NonFunction
85    check x
86    check y
87  check (TryCatch x y) = do
88    maskErrors $ check x
89    check y
90
91  checkExp :: Exp -> [Error]
92  checkExp e = snd $ runCheckM (check e) []
```

APL/Error.hs

```
module APL.Error
  ( Error(..)
  , isVariableError
  , isDomainError
  , isTypeError
  )
where

import APL.AST (VName)

data Error
  = NonInteger
  | UnknownVariable VName
  | DivisionByZero
  | NegativeExponent
  | InvalidEqual
  | NonBoolean
  | NonFunction
  deriving (Show, Eq)

isVariableError :: Error -> Bool
isVariableError (UnknownVariable _) = True
isVariableError _ = False

isDomainError :: Error -> Bool
isDomainError DivisionByZero = True
isDomainError NegativeExponent = True
isDomainError _ = False

isTypeError :: Error -> Bool
isTypeError NonInteger = True
isTypeError InvalidEqual = True
isTypeError NonBoolean = True
isTypeError NonFunction = True
isTypeError _ = False
```

APL/Eval.hs

```
module APL.Eval
  ( Val (..),
    Env,
    eval,
    runEval,
  )
where

import APL.AST (Exp (..), VName)
import APL.Error (Error (..))
import Control.Monad (ap, liftM)

data Val
  = ValInt Integer
  | ValBool Bool
  | ValFun Env VName Exp
  deriving (Eq, Show)

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

newtype EvalM a = EvalM (Env -> Either Error a)

instance Functor EvalM where
  fmap = liftM

instance Applicative EvalM where
  pure x = EvalM $ \_env -> Right x
  (<*>) = ap

instance Monad EvalM where
  EvalM x >>= f = EvalM $ \env ->
    case x env of
      Left err -> Left err
      Right x' ->
        let EvalM y = f x'
         in y env

askEnv :: EvalM Env
askEnv = EvalM $ \env -> Right env

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f (EvalM m) = EvalM $ \env -> m (f env)

failure :: Error -> EvalM a
failure s = EvalM $ \_env -> Left s

catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env ->
  case m1 env of
    Left _ -> m2 env
    Right x -> Right x

runEval :: EvalM a -> Either Error a
runEval (EvalM m) = m envEmpty

evalIntBinOp :: (Integer -> Integer -> EvalM Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp f e1 e2 = do
```

```
  v1 <- eval e1
  case v1 of
    ValInt x -> do
      v2 <- eval e2
      case v2 of
        ValInt y -> ValInt <$> f x y
        _ -> failure NonInteger
    _ -> failure NonInteger

evalIntBinOp' :: (Integer -> Integer -> Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp' f e1 e2 =
  evalIntBinOp f' e1 e2
  where
    f' x y = pure $ f x y

eval :: Exp -> EvalM Val
eval (CstInt x) = pure $ ValInt x
eval (CstBool b) = pure $ ValBool b
eval (Var v) = do
  env <- askEnv
  case envLookup v env of
    Just x -> pure x
    Nothing -> failure $ UnknownVariable v
eval (Add e1 e2) = evalIntBinOp' (+) e1 e2
eval (Sub e1 e2) = evalIntBinOp' (-) e1 e2
eval (Mul e1 e2) = evalIntBinOp' (*) e1 e2
eval (Div e1 e2) = evalIntBinOp checkedDiv e1 e2
  where
    checkedDiv _ 0 = failure DivisionByZero
    checkedDiv x y = pure $ x `div` y
eval (Pow e1 e2) = evalIntBinOp checkedPow e1 e2
  where
    checkedPow x y =
      if y < 0
        then failure NegativeExponent
        else pure $ x ^ y
eval (Eql e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValInt x, ValInt y) -> pure $ ValBool $ x == y
    (ValBool x, ValBool y) -> pure $ ValBool $ x == y
    (_, _) -> failure InvalidEqual
eval (If cond e1 e2) = do
  cond' <- eval cond
  case cond' of
    ValBool True -> eval e1
    ValBool False -> eval e2
    _ -> failure NonBoolean
eval (Let var e1 e2) = do
  v1 <- eval e1
  localEnv (envExtend var v1) $ eval e2
eval (Lambda var body) = do
  env <- askEnv
  pure $ ValFun env var body
eval (Apply e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValFun f_env var body, arg) ->
      localEnv (const $ envExtend var arg f_env) $ eval body
    (_, _) ->
      failure NonFunction
eval (TryCatch e1 e2) =
  eval e1 `catch` eval e2
```

APL/Tests.hs

```
module APL.Tests where

import APL.AST (Exp (..), VName)
import APL.Eval (eval, runEval)
import Test.QuickCheck
  ( Gen
  , Arbitrary (arbitrary, shrink)
  , elements
  , listOf
  , oneof
  , sized
  )

genVar :: Gen VName
genVar = do
    alpha <- elements ['a' .. 'z']
    alphaNums <- listOf $ elements $ ['a' .. 'z'] ++ ['0' .. '9']
    pure (alpha : alphaNums)

genExp :: Int -> Gen Exp
genExp size
  | size <= 1 = oneof [CstInt <$> arbitrary, CstBool <$> arbitrary]
  | otherwise =
    oneof
      [ CstInt <$> arbitrary
      , CstBool <$> arbitrary
      , Add <$> genExp halfSize <*> genExp halfSize
      , Sub <$> genExp halfSize <*> genExp halfSize
      , Mul <$> genExp halfSize <*> genExp halfSize
      , Div <$> genExp halfSize <*> genExp halfSize
      , Pow <$> genExp halfSize <*> genExp halfSize
      , Eql <$> genExp halfSize <*> genExp halfSize
      , If <$> genExp thirdSize <*> genExp thirdSize <*> genExp thirdSize
      , Var <$> genVar
      , Let <$> genVar <*> genExp halfSize <*> genExp halfSize
      , Lambda <$> genVar <*> genExp (size - 1)
      , Apply <$> genExp halfSize <*> genExp halfSize
      , TryCatch <$> genExp halfSize <*> genExp halfSize
      ]
    where
      halfSize = size `div` 2
      thirdSize = size `div` 3

prop_integerAddAssoc :: Integer -> Integer -> Integer -> Bool
prop_integerAddAssoc a b c = (a + b) + c == a + (b + c)

prop_aplAddAssoc :: Exp -> Exp -> Exp -> Bool
prop_aplAddAssoc e1 e2 e3 = runEval (eval (Add (Add e1 e2) e3)) == runEval (eval (Add e1 (Add e2 e3)))

instance Arbitrary Exp where
  arbitrary = sized genExp

  shrink (CstBool _) = []
  shrink (CstInt k) = [CstInt k' | k' <- shrink k]
  shrink (Add e1 e2) =
    e1 : e2 : [Add e1' e2 | e1' <- shrink e1] ++ [Add e1 e2' | e2' <- shrink e2]
  shrink (Sub e1 e2) =
    e1 : e2 : [Sub e1' e2 | e1' <- shrink e1] ++ [Sub e1 e2' | e2' <- shrink e2]
  shrink (Mul e1 e2) =
    e1 : e2 : [Mul e1' e2 | e1' <- shrink e1] ++ [Mul e1 e2' | e2' <- shrink e2]
  shrink (Div e1 e2) =
    e1 : e2 : [Div e1' e2 | e1' <- shrink e1] ++ [Div e1 e2' | e2' <- shrink e2]
  shrink (Pow e1 e2) =
    e1 : e2 : [Pow e1' e2 | e1' <- shrink e1] ++ [Pow e1 e2' | e2' <- shrink e2]
  shrink (Eql e1 e2) =
    e1 : e2 : [Eql e1' e2 | e1' <- shrink e1] ++ [Eql e1 e2' | e2' <- shrink e2]
```

```
67    shrink (If cond e1 e2) =
68      e1 : e2 : [If cond' e1 e2 | cond' <- shrink cond] ++ [If cond e1' e2 | e1' <- shrink e1] ++ [If cond e1 e2' |
        ↪  e2' <- shrink e2]
69    shrink (Var x) =
70      [Var x' | x' <- shrink x, not (null x')]
71    shrink (Let x e1 e2) =
72      e1 : [Let x e1' e2 | e1' <- shrink e1] ++ [Let x e1 e2' | e2' <- shrink e2]
73    shrink (Lambda x e) =
74      e : [Lambda x e' | e' <- shrink e]
75    shrink (Apply e1 e2) =
76      e1 : e2 : [Apply e1' e2 | e1' <- shrink e1] ++ [Apply e1 e2' | e2' <- shrink e2]
77    shrink (TryCatch e1 e2) =
78      e1 : e2 : [TryCatch e1' e2 | e1' <- shrink e1] ++ [TryCatch e1 e2' | e2' <- shrink e2]
```

Advanced Programming 2026

Assignment Five

Property-Based Testing

Due: 2026-10-11

Synopsis: Designing property-based tests for the APL printer, parser, evaluator and checker: a generator with measured coverage, a round-trip property, and a property that exposes a faulty assumption

1 Introduction

In this assignment you will practise the two halves of property-based testing: stating properties, and designing test data with high falsification power.

You will work on the *pure* APL components you built in earlier assignments: the printer, the parser, the evaluator and the checker. You will build a generator for expressions whose distribution is good enough to satisfy a given *coverage specification*, and then use it to test two properties—one that should hold, and one that does not.

Chapter 5 of the course notes covers everything you need. In particular, `cover`, `checkCoverage` and `tabulate` are discussed in the section on test data design.

2 Task: A generator with measured coverage

The handout contains a generator `genExp` for APL expressions that is correct but useless: it produces expressions whose distribution barely exercises any interesting cases. The handout also contains a property `expCoverage`, which states, as a machine-checkable coverage specification, what the distribution ought to look like.

Change the definition of `genExp` so that `expCoverage` passes. In particular this means that

- Between 20% and 80% of expressions should have domain errors (division by zero or negative exponent) reported by `checkExp`.
- Between 20% and 80% of expressions should have type errors reported by `checkExp`.
- Between 5% and 30% of expressions should have variable errors reported by `checkExp`.
- At least 50% of expressions should contain a variable between 2 and 4 characters long.

You *are* permitted to change the signature of `genExp`, but make sure to also change the definition of `arbitrary` for `Exp` accordingly. You must not weaken `expCoverage`.

2.1 Suggested implementation

Use `frequency` instead of `oneof` to control the distribution, and keep track of the variables in scope by adding an additional parameter of type `[VName]` to `genExp`. Use `sample` and `tabulate` while developing the generator: guessing at weights and rerunning `checkCoverage` is much slower than looking at what you are producing.

3 Task: A round-trip property for printing and parsing

Implement the property `parsePrinted`, which checks that printing an expression and parsing the resulting string gives back the original expression.

When QuickCheck finds a counterexample to `parsePrinted` you must decide whether the error is in the *implementation*, the *generator* or the *property*. Then fix the error and continue until the property passes. You will need to fix errors of more than one of these three kinds.

Note that the handout already asks for 10000 tests per property, via `withMaxSuccess` in `properties`. **Do not reduce this number**: some of the properties below fail only rarely, and the number of tests is part of what makes your test suite reproducible for us.

4 Task: A property for checking and evaluating

Implement the property `onlyCheckedErrors`, which checks that evaluating an expression produces only errors that `checkExp` predicted.

`checkExp` returns a *list* of errors. The intention is that this list contains every error that evaluation of the expression *might* produce. So define `onlyCheckedErrors` to evaluate the given expression and, if evaluation returns an error, to check that this error occurs in the list returned by `checkExp`.

This property does *not* hold: `checkExp` rests on an assumption that is not justified. You are *not* asked to repair `checkExp`. You are asked to find a counterexample by testing, and then to explain in your report what the faulty assumption is (see the report questions below).

5 Your Report

You are expected to describe your test design and *interesting* aspects of your implementation. Do *not* give a line-by-line walkthrough of your code—relevant details of code that is not self-explanatory (the ideal) should be documented compactly in the source code. Most importantly, reflect on the *quality* of your code:

- Why is it functionally correct? (Is it?) Provide arguments addressed to a skeptical reader, not yourself.
- Is there an improvement you would have liked to make, but didn't include?

It is more important to reflect on the strengths or shortcomings of your solution than it is to have a complete, but unexplained solution.

5.1 The structure of your report

Your report must be structured as follows:

Introduction: Briefly mention general concerns, any ambiguities in the problem text and how you resolved them, and your own estimation

of the quality of your solution. Briefly mention whether your solution is functional, which test cases cover its functionality, which test cases it fails for (if any), and what you think might be wrong.

A section answering the following numbered questions:

1. Can programs produced by your `genExp` loop infinitely when evaluated? If so, is it possible to avoid this?
2. Which of the four coverage requirements in `expCoverage` was hardest to satisfy, and what did you do to satisfy it?
3. Which counterexamples did `parsePrinted` produce? For each counterexample, which component (implementation, generator or property) did you fix, and why was that the right place to fix it?
4. Which assumption underlying `checkExp` is not justified, and which counterexample did `onlyCheckedErrors` produce to demonstrate this?
5. Your generator satisfies `expCoverage`. Describe a class of expressions that your generator will nevertheless produce essentially never, but that would be worth testing. How would you change either the generator or `expCoverage` to cover it?

All else being equal, **a short report is a good report**.

6 Code handout

The code handout consists of the following nontrivial files.

- `a5.cabal`: Cabal build file. **Do not modify this file.**
- `runtests.hs`: Test execution program. **Do not modify this file.**
- `apl.hs`: A command line interpreter for APL. You will not need it, but it may be convenient for trying out expressions by hand.
- `src/APL/AST.hs`: The APL AST definition. **You may only modify `printExp` in this file.**

- `src/APL/Error.hs`: The APL Error definition. **Do not modify this file.**
- `src/APL/Parser.hs`: The parser.
- `src/APL/Check.hs`: The checker.
- `src/APL/Eval.hs`: The APL evaluation function.
- `src/APL/Tests.hs`: The property tests. This is where most of your work goes.

7 Deliverables for This Assignment

You must submit the following items:

- A single PDF file, A4 size, no more than 5 pages, structured as specified in the report section above.
- A single zip/tar.gz file with all code relevant to the implementation, including at least all the files from the handout. For this assignment it is not necessary to add additional files.

Remember to follow the general assignment rules listed on the course homepage.

8 Assessment

You will get written qualitative feedback, and points from zero to four. There are no resubmissions, so please hand in what you managed to do, even if you have not solved the assignment completely.

A5 handout

APL/AST.hs

```
module APL.AST
  ( VName
  , Exp (..)
  , printExp
  , subExp
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Pow Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  | Lambda VName Exp
  | Apply Exp Exp
  | TryCatch Exp Exp
  deriving (Eq, Show)

parens :: String -> String
parens x = "(" ++ x ++ ")"

printBinOp :: String -> Exp -> Exp -> String
printBinOp op x y = parens $ printExp x ++ " " ++ op ++ " " ++ printExp y

printExp :: Exp -> String
printExp (CstInt x) = show x
printExp (CstBool b) = if b then "true" else "false"
printExp (Add x y) = printBinOp "+" x y
printExp (Sub x y) = printBinOp "-" x y
printExp (Mul x y) = printBinOp "*" x y
printExp (Div x y) = printBinOp "/" x y
printExp (Pow x y) = printBinOp "**" x y
printExp (Eql x y) = printBinOp "==" x y
printExp (If x y z) =
  parens $
    "if "
      ++ printExp x
      ++ " then "
      ++ printExp y
      ++ " else "
      ++ printExp z
printExp (Var v) = v
printExp (Let v e1 e2) =
  parens $
    "let "
      ++ v
      ++ " = "
      ++ printExp e1
      ++ " in "
      ++ printExp e2
printExp (Lambda v body) =
  parens $ "\\" ++ v ++ " -> " ++ printExp body
printExp (Apply x y) =
  printExp x ++ " " ++ printExp y
printExp (TryCatch x y) =
  "try " ++ printExp x ++ " catch " ++ printExp y

```

```
67  subExp :: Exp -> [Exp]
68  subExp e = e : case e of
69    CstInt _ -> []
70    CstBool _ -> []
71    Add e1 e2 -> subExp e1 ++ subExp e2
72    Sub e1 e2 -> subExp e1 ++ subExp e2
73    Mul e1 e2 -> subExp e1 ++ subExp e2
74    Div e1 e2 -> subExp e1 ++ subExp e2
75    Pow e1 e2 -> subExp e1 ++ subExp e2
76    Eql e1 e2 -> subExp e1 ++ subExp e2
77    If e0 e1 e2 -> subExp e0 ++ subExp e1 ++ subExp e2
78    Var _ -> []
79    Let _ e1 e2 -> subExp e1 ++ subExp e2
80    Lambda _ body -> subExp body
81    Apply e1 e2 -> subExp e1 ++ subExp e2
82    TryCatch e1 e2 -> subExp e1 ++ subExp e2
```

APL/Check.hs

```
module APL.Check (checkExp, Error) where

import APL.AST (Exp (..), VName)
import APL.Error (Error (..))
import Control.Monad (ap, liftM, unless)
import Data.List (union)

type Vars = [VName]

newtype CheckM a = CheckM {runCheckM :: Vars -> (a, [Error])}

instance Functor CheckM where
  fmap = liftM

instance Applicative CheckM where
  (<*>) = ap
  pure x = CheckM $ \_ -> (x, [])

instance Monad CheckM where
  CheckM x >>= f = CheckM $ \vars ->
    let (y, errs1) = x vars
        (z, errs2) = runCheckM (f y) vars
    in (z, union errs1 errs2)

askVars :: CheckM Vars
askVars = CheckM $ \vars -> (vars, [])

localVars :: (Vars -> Vars) -> CheckM a -> CheckM a
localVars f m = CheckM $ \vars ->
  runCheckM m (f vars)

failure :: Error -> CheckM ()
failure err = CheckM $ \_ -> ((), [err])

maskErrors :: CheckM a -> CheckM a
maskErrors m = CheckM $ \vars ->
  let (x, _) = runCheckM m vars in (x, [])

check :: Exp -> CheckM ()
check (CstInt _) = pure ()
check (CstBool _) = pure ()
check (Var v) = do
  vars <- askVars
  unless (v `elem` vars) $
    failure $
      UnknownVariable v
check (Add x y) = do
  failure NonInteger
  check x
  check y
check (Sub x y) = do
  failure NonInteger
  check x
  check y
check (Mul x y) = do
  failure NonInteger
  check x
  check y
check (Div x y) = do
  failure NonInteger
  failure DivisionByZero
  check x
  check y
check (Pow x y) = do
  failure NonInteger
  failure NegativeExponent
```

```
67    check x
68    check y
69  check (Eql x y) = do
70    failure InvalidEqual
71    check x
72    check y
73  check (If x y z) = do
74    failure NonBoolean
75    check x
76    check y
77    check z
78  check (Let v e1 e2) = do
79    check e1
80    localVars (v :) $ check e2
81  check (Lambda v e) = do
82    localVars (v :) $ check e
83  check (Apply x y) = do
84    failure NonFunction
85    check x
86    check y
87  check (TryCatch x y) = do
88    maskErrors $ check x
89    check y
90
91  checkExp :: Exp -> [Error]
92  checkExp e = snd $ runCheckM (check e) []
```

APL/Error.hs

```
module APL.Error
  ( Error(..)
  , isVariableError
  , isDomainError
  , isTypeError
  )
where

import APL.AST (VName)

data Error
  = NonInteger
  | UnknownVariable VName
  | DivisionByZero
  | NegativeExponent
  | InvalidEqual
  | NonBoolean
  | NonFunction
  deriving (Show, Eq)

isVariableError :: Error -> Bool
isVariableError (UnknownVariable _) = True
isVariableError _ = False

isDomainError :: Error -> Bool
isDomainError DivisionByZero = True
isDomainError NegativeExponent = True
isDomainError _ = False

isTypeError :: Error -> Bool
isTypeError NonInteger = True
isTypeError InvalidEqual = True
isTypeError NonBoolean = True
isTypeError NonFunction = True
isTypeError _ = False
```

APL/Eval.hs

```
module APL.Eval
  ( Val (..),
    Env,
    eval,
    runEval,
  )
where

import APL.AST (Exp (..), VName)
import APL.Error (Error (..))
import Control.Monad (ap, liftM)

data Val
  = ValInt Integer
  | ValBool Bool
  | ValFun Env VName Exp
  deriving (Eq, Show)

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

newtype EvalM a = EvalM (Env -> Either Error a)

instance Functor EvalM where
  fmap = liftM

instance Applicative EvalM where
  pure x = EvalM $ \_env -> Right x
  (<*>) = ap

instance Monad EvalM where
  EvalM x >>= f = EvalM $ \env ->
    case x env of
      Left err -> Left err
      Right x' ->
        let EvalM y = f x'
         in y env

askEnv :: EvalM Env
askEnv = EvalM $ \env -> Right env

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv f (EvalM m) = EvalM $ \env -> m (f env)

failure :: Error -> EvalM a
failure s = EvalM $ \_env -> Left s

catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env ->
  case m1 env of
    Left _ -> m2 env
    Right x -> Right x

runEval :: EvalM a -> Either Error a
runEval (EvalM m) = m envEmpty

evalIntBinOp :: (Integer -> Integer -> EvalM Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp f e1 e2 = do
```

```
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValInt x, ValInt y) -> ValInt <$> f x y
    (_, _) -> failure NonInteger

evalIntBinOp' :: (Integer -> Integer -> Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp' f e1 e2 =
  evalIntBinOp f' e1 e2
  where
    f' x y = pure $ f x y

eval :: Exp -> EvalM Val
eval (CstInt x) = pure $ ValInt x
eval (CstBool b) = pure $ ValBool b
eval (Var v) = do
  env <- askEnv
  case envLookup v env of
    Just x -> pure x
    Nothing -> failure $ UnknownVariable v
eval (Add e1 e2) = evalIntBinOp' (+) e1 e2
eval (Sub e1 e2) = evalIntBinOp' (-) e1 e2
eval (Mul e1 e2) = evalIntBinOp' (*) e1 e2
eval (Div e1 e2) = evalIntBinOp checkedDiv e1 e2
  where
    checkedDiv _ 0 = failure DivisionByZero
    checkedDiv x y = pure $ x `div` y
eval (Pow e1 e2) = evalIntBinOp checkedPow e1 e2
  where
    checkedPow x y =
      if y < 0
        then failure NegativeExponent
        else pure $ x ^ y
eval (Eql e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValInt x, ValInt y) -> pure $ ValBool $ x == y
    (ValBool x, ValBool y) -> pure $ ValBool $ x == y
    (_, _) -> failure InvalidEqual
eval (If cond e1 e2) = do
  cond' <- eval cond
  case cond' of
    ValBool True -> eval e1
    ValBool False -> eval e2
    _ -> failure NonBoolean
eval (Let var e1 e2) = do
  v1 <- eval e1
  localEnv (envExtend var v1) $ eval e2
eval (Lambda var body) = do
  env <- askEnv
  pure $ ValFun env var body
eval (Apply e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValFun f_env var body, arg) ->
      localEnv (const $ envExtend var arg f_env) $ eval body
    (_, _) ->
      failure NonFunction
eval (TryCatch e1 e2) =
  eval e1 `catch` eval e2
```

APL/Parser.hs

```
module APL.Parser (parseAPL) where

import APL.AST (Exp (..), VName)
import Control.Monad (void)
import Data.Char (isAlpha, isAlphaNum, isDigit)
import Data.Void (Void)
import Text.Megaparsec
  ( Parsec,
    choice,
    chunk,
    eof,
    errorBundlePretty,
    many,
    notFollowedBy,
    parse,
    satisfy,
    some,
    try,
  )
import Text.Megaparsec.Char (space)

type Parser = Parsec Void String

lexeme :: Parser a -> Parser a
lexeme p = p <* space

keywords :: [String]
keywords =
  [ "if",
    "then",
    "else",
    "true",
    "false",
    "let",
    "in",
    "try",
    "catch"
  ]

lVName :: Parser VName
lVName = lexeme $ try $ do
  c <- satisfy isAlpha
  cs <- many $ satisfy isAlphaNum
  let v = c : cs
  if v `elem` keywords
    then fail "Unexpected keyword"
    else pure v

lInteger :: Parser Integer
lInteger =
  lexeme $ read <$> some (satisfy isDigit) <* notFollowedBy (satisfy isAlphaNum)

lString :: String -> Parser ()
lString s = lexeme $ void $ chunk s

lKeyword :: String -> Parser ()
lKeyword s = lexeme $ void $ try $ chunk s <* notFollowedBy (satisfy isAlphaNum)

lBool :: Parser Bool
lBool =
  lexeme . try . choice $
    [ const True <$> lKeyword "true",
      const False <$> lKeyword "false"
    ]

pAtom :: Parser Exp
```

```
67  pAtom =
68    choice
69      [ CstInt <$> lInteger,
70        CstBool <$> lBool,
71        Var <$> lVName,
72        lString "(" *> pExp <* lString ")"
73      ]
74
75  pFExp :: Parser Exp
76  pFExp = chain =<< pAtom
77    where
78      chain x =
79        choice
80          [ do
81              y <- pAtom
82              chain $ Apply x y,
83            pure x
84          ]
85
86  pLExp :: Parser Exp
87  pLExp =
88    choice
89      [ If
90          <$> (lKeyword "if" *> pExp)
91          <*> (lKeyword "then" *> pExp)
92          <*> (lKeyword "else" *> pExp),
93        Lambda
94          <$> (lString "\\" *> lVName)
95          <*> (lString "->" *> pExp),
96        TryCatch
97          <$> (lKeyword "try" *> pExp)
98          <*> (lKeyword "catch" *> pExp),
99        Let
100         <$> (lKeyword "let" *> lVName)
101         <*> (lString "=" *> pExp)
102         <*> (lKeyword "in" *> pExp),
103       pFExp
104     ]
105
106 pExp4 :: Parser Exp
107 pExp4 = pLExp >>= chain
108   where
109     chain x =
110       choice
111         [ do
112             lString "**"
113             y <- pLExp
114             Pow x <$> chain y,
115           pure x
116         ]
117
118 pExp3 :: Parser Exp
119 pExp3 = pExp4 >>= chain
120   where
121     chain x =
122       choice
123         [ do
124             lString "*"
125             y <- pExp4
126             chain $ Mul x y,
127           do
128             lString "/"
129             y <- pExp4
130             chain $ Div x y,
131           pure x
132         ]
133
134 pExp2 :: Parser Exp
```

```
135 pExp2 = pExp3 >>= chain
136   where
137     chain x =
138       choice
139         [ do
140             lString "+"
141             y <- pExp3
142             chain $ Add x y,
143           do
144             lString "-"
145             y <- pExp3
146             chain $ Sub x y,
147           pure x
148         ]
149
150 pExp1 :: Parser Exp
151 pExp1 = pExp2 >>= chain
152   where
153     chain x =
154       choice
155         [ do
156             lString "=="
157             y <- pExp2
158             chain $ Eql x y,
159           pure x
160         ]
161
162 pExp :: Parser Exp
163 pExp = pExp1
164
165 parseAPL :: FilePath -> String -> Either String Exp
166 parseAPL fname s = case parse (space *> pExp <* eof) fname s of
167   Left err -> Left $ errorBundlePretty err
168   Right x -> Right x
```

APL/Tests.hs

```
module APL.Tests
  ( properties
  )
where

import APL.AST (Exp (..), subExp)
import APL.Error (isVariableError, isDomainError, isTypeError)
import APL.Check (checkExp)
import Test.QuickCheck
  ( Property
  , Gen
  , Arbitrary (arbitrary, shrink)
  , property
  , cover
  , checkCoverage
  , oneof
  , sized
  , withMaxSuccess
  )

instance Arbitrary Exp where
  arbitrary = sized genExp

  shrink (Add e1 e2) =
    e1 : e2 : [Add e1' e2 | e1' <- shrink e1] ++ [Add e1 e2' | e2' <- shrink e2]
  shrink (Sub e1 e2) =
    e1 : e2 : [Sub e1' e2 | e1' <- shrink e1] ++ [Sub e1 e2' | e2' <- shrink e2]
  shrink (Mul e1 e2) =
    e1 : e2 : [Mul e1' e2 | e1' <- shrink e1] ++ [Mul e1 e2' | e2' <- shrink e2]
  shrink (Div e1 e2) =
    e1 : e2 : [Div e1' e2 | e1' <- shrink e1] ++ [Div e1 e2' | e2' <- shrink e2]
  shrink (Pow e1 e2) =
    e1 : e2 : [Pow e1' e2 | e1' <- shrink e1] ++ [Pow e1 e2' | e2' <- shrink e2]
  shrink (Eql e1 e2) =
    e1 : e2 : [Eql e1' e2 | e1' <- shrink e1] ++ [Eql e1 e2' | e2' <- shrink e2]
  shrink (If cond e1 e2) =
    e1 : e2 : [If cond' e1 e2 | cond' <- shrink cond] ++ [If cond e1' e2 | e1' <- shrink e1] ++ [If cond e1 e2' |
    ↪  e2' <- shrink e2]
  shrink (Let x e1 e2) =
    e1 : [Let x e1' e2 | e1' <- shrink e1] ++ [Let x e1 e2' | e2' <- shrink e2]
  shrink (Lambda x e) =
    [Lambda x e' | e' <- shrink e]
  shrink (Apply e1 e2) =
    e1 : e2 : [Apply e1' e2 | e1' <- shrink e1] ++ [Apply e1 e2' | e2' <- shrink e2]
  shrink (TryCatch e1 e2) =
    e1 : e2 : [TryCatch e1' e2 | e1' <- shrink e1] ++ [TryCatch e1 e2' | e2' <- shrink e2]
  shrink _ = []

genExp :: Int -> Gen Exp
genExp 0 = oneof [CstInt <$> arbitrary, CstBool <$> arbitrary]
genExp size =
  oneof
    [ CstInt <$> arbitrary
    , CstBool <$> arbitrary
    , Add <$> genExp halfSize <*> genExp halfSize
    , Sub <$> genExp halfSize <*> genExp halfSize
    , Mul <$> genExp halfSize <*> genExp halfSize
    , Div <$> genExp halfSize <*> genExp halfSize
    , Pow <$> genExp halfSize <*> genExp halfSize
    , Eql <$> genExp halfSize <*> genExp halfSize
    , If <$> genExp thirdSize <*> genExp thirdSize <*> genExp thirdSize
    , Var <$> arbitrary
    , Let <$> arbitrary <*> genExp halfSize <*> genExp halfSize
    , Lambda <$> arbitrary <*> genExp (size - 1)
    , Apply <$> genExp halfSize <*> genExp halfSize
    , TryCatch <$> genExp halfSize <*> genExp halfSize
```

```
    ]
  where
    halfSize = size `div` 2
    thirdSize = size `div` 3

expCoverage :: Exp -> Property
expCoverage e = checkCoverage
  . cover 20 (any isDomainError (checkExp e)) "domain error"
  . cover 20 (not $ any isDomainError (checkExp e)) "no domain error"
  . cover 20 (any isTypeError (checkExp e)) "type error"
  . cover 20 (not $ any isTypeError (checkExp e)) "no type error"
  . cover 5 (any isVariableError (checkExp e)) "variable error"
  . cover 70 (not $ any isVariableError (checkExp e)) "no variable error"
  . cover 50 (or [2 <= n && n <= 4 | Var v <- subExp e, let n = length v]) "non-trivial variable"
  $ ()

parsePrinted :: Exp -> Bool
parsePrinted _ = undefined

onlyCheckedErrors :: Exp -> Bool
onlyCheckedErrors _ = undefined

-- The number of tests is part of the specification of this test suite: some of
-- these properties fail only rarely.  Do not reduce it.
properties :: [(String, Property)]
properties =
  [ ("expCoverage", property $ withMaxSuccess 10000 expCoverage)
  , ("parsePrinted", property $ withMaxSuccess 10000 parsePrinted)
  , ("onlyCheckedErrors", property $ withMaxSuccess 10000 onlyCheckedErrors)
  ]
```

Week 6 exercises

Week 6 - Concurrency

Suggested Reading

- Course Notes Chapter 6
- Concurrent Haskell

Going Beyond

- The three kinds of Haskell exceptions and how to use them
- `exitFailure` doesn't exit

Exercises

In these exercises, and the associated assignment, you will be implementing the *Stateful Planning Committee* (SPC), a *job scheduler* for managing the employment of workers and allocation of resources. A job is any Haskell computation in the `IO` monad. A job does not return any value, but is executed solely for its side effects. After a job is *enqueued* in SPC, we can ask for the status of the job. At some point, SPC will decide to actually execute the job.

We can imagine using a job scheduler such as SPC to enqueue thousands of jobs for downloading various files from the Internet, with the scheduler taking care of ensuring only a limited number of jobs run concurrently, handling timeouts, performing logging, and so on. Our scheduler will be fairly simple, but will have a lot of the machinery you also need in a real scheduler.

SPC will take the form of various Haskell threads that communicate through message passing. The main challenge is that the jobs are *untrusted*, and may signal errors and go into infinite loops, which must be handled without any damage to the system as a whole. In particular, there will be a central *SPC thread* which we must protect by ensuring evaluation of untrusted code happens in separate threads, and with appropriate handlers.

There are some limits to the antagonism we will assume of enqueued work: we cannot cope with work that uses functions from System.Exit to shut down the entire process, or maliciously try to subvert SPC itself. Since jobs are (for now) arbitrary `IO` actions, there is not any true security in SPC.

This exercise permits you somewhat more freedom than most of the other exercises. In particular, you are expected to largely design the message types (the "protocol") yourself.

You must use the abstractions provided by `GenServer`, as discussed in the course notes. However, for some functionality, you will need to move beyond this abstraction, and directly launch threads with `forkIO`.

Creating the Event Loop

You will find a skeletal code handout in handout/. The module `SPC` contains two utility functions that will become useful later (`getSeconds` and `removeAssoc`). It also contains two types (`SPCMsg`, `SPC`) and one value definition (`startSPC`). It also contains a bunch of imports that will become necessary, but adds a lot of warning clutter - feel free to comment them until you need them.

The `SPC` type is our handle to the SPC, and it is complete - you do not need to modify it.

The `SPCMsg` type represents messages that will be sent *to* the SPC thread. We will extend this type with new messages as we go along.

The first thing you must do is to extend `startSPC` such that it launches a new thread that runs a loop that reads messages from the created channel and acts on them. To do this:

1. Add a constructor `MsgPing (ReplyChan Int)` to `SPCMsg`. We will remove this later, but it is useful for testing that we got the event loop right.
2. Modify `startSPC` such that it creates a thread that runs in an infinite loop. This thread runs in an infinite loop and reads `SPCMsg` messages. When it receives a `MsgPing` message, it should send back an integer (your choice) on the channel in the `MsgPing` message.
3. Write a function `pingSPC :: SPC -> IO Int` that sends a `MsgPing` message to SPC, waits for a response, then returns the integer in the response. Use `requestReply`. You must also add `pingSPC` to the module export list.
4. Add a test to `SPC_Tests` that exercises `pingSPC`.

Hints

Use `forever` from `Control.Monad` to write infinite monadic loops.

Solution

Open this to see the implementation

```
data SPCMsg = MsgPing (ReplyChan Int)

startSPC :: IO SPC
startSPC = do
  server <- spawn $ \c -> forever $ handle c
  pure $ SPC server
  where
    handle c = do
      msg <- receive c
      case msg of
        MsgPing rsvp ->
          reply rsvp 1337

pingSPC :: SPC -> IO Int
pingSPC (SPC c) =
  requestReply c MsgPing

-- And in SPC_Tests:

tests :: TestTree
tests =
  localOption (mkTimeout 3000000) $
    testGroup
      "SPC"
      [ testCase "ping" $ do
          spc <- startSPC
          x <- pingSPC spc
          x @?= 1337
      ]
```

A Stateful Monadic Event Loop

Most servers maintain some kind of state that is modified by the messages they receive. Eventually SPC will have a rather complicated notion of state, but to start out, we will simply have it track how many ping messages it has received, and reply to pings with that number.

To make things interesting (and ultimately, more manageable), we will maintain the state using a monad. First, add a type repesenting the SPC state:

```
-- | The central state. Must be protected from the bourgeoisie.
data SPCState = SPCState
  { spcPingCounter :: Int
  }
```

Then we define an SPC monad that is a state monad with a state of type `SPCState`, but which also supports `IO`:

```
newtype SPCM a = SPCM (SPCState -> IO (a, SPCState))
```

- Implement the `Functor`, `Applicative`, and `Monad` instances for `SPCM`.

Open this to see the implementation

```
instance Functor SPCM where
  fmap = liftM

instance Applicative SPCM where
  pure x = SPCM $ \state -> pure (x, state)
  (<*>) = ap

instance Monad SPCM where
  SPCM m >>= f = SPCM $ \state -> do
```

```
    (x, state') <- m state
    let SPCM f' = f x
    f' state'
```

We will also need the usual utility functions for managing the state:

```
get :: SPCM SPCState
put :: SPCState -> SPCM ()
```

- Implement `get` and `put`.

Open this to see the implementation

```
-- | Retrieve the state.
get :: SPCM SPCState
get = SPCM $ \state -> pure (state, state)

-- | Overwrite the state.
put :: SPCState -> SPCM ()
put state = SPCM $ \_ -> pure ((), state)
```

And we also need a function for lifting an arbitrary IO action into `SPCM`:

```
io :: IO a -> SPCM a
```

- Implement `io`.

Open this to see the implementation

```
io m = SPCM $ \state -> do
  x <- m
  pure (x, state)
```

Finally, we need a function for running an `SPCM` operation, which necessarily must be done in the `IO` monad:

```
runSPCM :: SPCState -> SPCM a -> IO a
```

- Implement `runSPCM`.

Open this to see the implementation

```
runSPCM state (SPCM f) = fst <$> f state
```

Now we are ready to use `SPCM`.

- Modify `startSPC` such that the loop runs inside the `SPCM` monad.
- Modify the handling of `MsgPing` such that it replies with the `spcPingCounter` field from the state, and then increments the field by one.
- Move the handling of a message into a separate function with signature `handleMsg :: Chan SPCMsg -> SPCM ()`.
- Add an appropriate test.

Hints

- To read a message from a channel `c` inside `SPCM`: `io $ receive c`.
- `succ x` produces `x+1`.

Solution

Open this to see the implementation

```
handleMsg :: Chan SPCMsg -> SPCM ()
handleMsg c = do
  msg <- io $ receive c
  case msg of
    MsgPing rsvp -> do
      state <- get
      io $ reply rsvp $ spcPingCounter state
      put $ state {spcPingCounter = succ $ spcPingCounter state}

startSPC :: IO SPC
```

```
startSPC = do
  let initial_state =
        SPCState
          { spcPingCounter = 0
          }
  server <- spawn $ \c -> runSPCM initial_state $ forever $ handleMsg c
  pure $ SPC server

-- And a test:

tests :: TestTree
tests =
  localOption (mkTimeout 3000000) $
    testGroup
      "SPC"
      [ testCase "ping" $ do
          spc <- startSPC
          x <- pingSPC spc
          x @?= 0
          y <- pingSPC spc
          y @?= 1
          z <- pingSPC spc
          z @?= 2
      ]
```

Adding Jobs

We have now constructed the skeleton of a stateful message passing server. Essentially all servers can start out this way, although the monadic approach to state handling may be overkill for very simple ones. We will now extend this skeleton with the actual job handling functionality that is the purpose of our server. Feel free to remove the `MsgPing` message and the related functions once you add more interesting messages.

First we will implement a way to add jobs. Jobs are described by this type:

```
-- | A job that is to be enqueued in the glorious SPC.
data Job = Job
  { -- | The IO action that comprises the actual action of the job.
    jobAction :: IO (),
    -- | The maximum allowed runtime of the job, counting from when
    -- the job begins executing (not when it is enqueued).
    jobMaxSeconds :: Int
  }
```

And are added by this function:

```
-- | A unique identifier of a job that has been enqueued.
newtype JobId = JobId Int
  deriving (Eq, Ord, Show)

-- | Add a job for scheduling.
jobAdd :: SPC -> Job -> IO JobId
```

The `Job` type (and its definition) should be exported, but the `JobId` should not have its constructor exported. It must be an abstract type from the point of view of users. This is achieved by putting respectively `Job(..)` and `JobId` in the module export list.

When a job is enqueued, it is not executed immediately. Rather, it is added to a list of *pending* jobs. Each job is associated with a unique `JobId`, which is used to reference it later.

Hints

- Extend `SPCState` with a field of type `[(JobId,Job)]` which tracks pending jobs, and a field of type `JobId` which contains the next available job identifier. Update other parts of the code as necessary.
- Add a synchronous message to `SPCMsg` for adding a new job. You must decide what payload this message should contain, but it must at least provide a channel for the reply, which is a `JobId`. When handling this message, you must add the job to the list that you added to the state.
- Implement `jobAdd` by using the message you added above.

- Add an appropriate test.

Solution

Open this to see the implementation

```
data SPCMsg
  = ...
    -- | Add the job, and reply with the job ID.
    MsgJobAdd Job (ReplyChan JobId)

data SPCState = SPCState
  { spcJobsPending :: [(JobId, Job)],
    spcJobCounter :: JobId
  }

handleMsg :: Chan SPCMsg -> SPCM ()
handleMsg c = do
  msg <- io $ receive c
  case msg of
    MsgJobAdd job rsvp -> do
      state <- get
      let JobId jobid = spcJobCounter state
      put $
        state
          { spcJobsPending =
              (spcJobCounter state, job) : spcJobsPending state,
            spcJobCounter = JobId $ succ jobid
          }
      io $ reply rsvp $ JobId jobid

startSPC :: IO SPC
startSPC = do
  let initial_state =
        SPCState
          { spcJobCounter = JobId 0,
            spcJobsPending = []
          }
  server <- spawn $ \c -> runSPCM initial_state $ forever $ handleMsg c
  pure $ SPC server

-- | Add a job for scheduling.
jobAdd :: SPC -> Job -> IO JobId
jobAdd (SPC c) job =
  requestReply c $ MsgJobAdd job

-- And a test:

tests :: TestTree
tests =
  localOption (mkTimeout 3000000) $
    testGroup
      "SPC"
      [ testCase "adding job" $ do
          spc <- startSPC
          _ <- jobAdd spc $ Job (pure ()) 1
          pure ()
      ]
```

Job Status

We will now add functionality for querying the status of a job. The status of the job is expressed by the following two types:

```
-- | How a job finished.
data JobDoneReason
  = -- | Normal termination.
    Done
```

```
  | -- | The job was killed because it ran for too long.
    DoneTimeout
  | -- | The job was explicitly cancelled.
    DoneCancelled
  | -- | The job crashed due to an exception.
    DoneCrashed
  deriving (Eq, Ord, Show)

-- | The status of a job.
data JobStatus
  = -- | The job is done and this is why.
    JobDone JobDoneReason
  | -- | The job is still running.
    JobRunning
  | -- | The job is enqueued, but is waiting for an idle worker.
    JobPending
  deriving (Eq, Ord, Show)
```

- Add these type definitions to `SPC` and also add them to the module export list.

Initially a job has status `JobPending`. In fact, since we have yet to implement actual execution of jobs, that is the only constructor we will make use of.

Your task is now to implement the function with the following signature:

```
-- | Query the job status.
jobStatus :: SPC -> JobId -> IO (Maybe JobStatus)
```

Hints

- Add a constructor to `SPCMsg` for requesting the status of a job.
- Handle the new message in `handleMsg`. If the provided `JobId` is found in the list of pending jobs, then the response should be `Just JobPending`, and otherwise `Nothing`.
- Implement the `jobStatus` function and add it to the module export list.
- Add an appropriate test, or modify an existing one to make use of `jobStatus`.

Solution

Open this to see the implementation

```
data SPCMsg
  = ...
  | -- | Immediately reply the status of the job.
    MsgJobStatus JobId (ReplyChan (Maybe JobStatus))

handleMsg :: Chan SPCMsg -> SPCM ()
handleMsg c = do
  msg <- io $ receive c
  case msg of
    ...
    MsgJobStatus jobid rsvp -> do
      state <- get
      io $ reply rsvp $ case lookup jobid $ spcJobsPending state of
        Just _ -> Just JobPending
        _ -> Nothing

-- | Query the job status. Returns 'Nothing' if job is not known to
-- this SPC instance.
jobStatus :: SPC -> JobId -> IO (Maybe JobStatus)
jobStatus (SPC c) jobid =
  requestReply c $ MsgJobStatus jobid

-- And a test

tests :: TestTree
tests =
```

```
  localOption (mkTimeout 3000000) $
    testGroup
      "SPC"
      [ testCase "adding job" $ do
          spc <- startSPC
          j <- jobAdd spc $ Job (pure ()) 1
          r <- jobStatus spc j
          r @?= Just JobPending
      ]
```

Job Cancellation

This task is about implementing the following function, which cancels a job and receives no response (even if the job was invalid).

```
-- | Asynchronously cancel a job.
jobCancel :: SPC -> JobId -> IO ()
```

When a pending job is cancelled, it should be moved to a separate list of done jobs, which should associate a `JobId` with a `JobDoneReason`. After a job has been cancelled, performing asking for its status should return `DoneCancelled`.

Hints

1. Extend `SPCState` with a list of done jobs.
2. Extend `SPCMsg` with an asynchronous message for cancelling a given job.
3. Handle the new message in `handleMsg`.
4. Implement `jobCancel` by moving the job (if it exists) from the list of pending jobs to the list of done jobs.
5. Modify the handling of the `jobStatus` message such that it replies `JobDone DoneCancelled` for cancelled jobs.
6. Add a test.

Solution

Open this to see the implementation

```
data SPCMsg
  = ...
  | MsgJobCancel JobId

data SPCState = SPCState
  { ...
    spcJobsDone :: [(JobId, JobDoneReason)]
  }

handleMsg :: Chan SPCMsg -> SPCM ()
handleMsg c = do
  msg <- io $ receive c
  case msg of
    ...
    MsgJobStatus jobid rsvp -> do
      state <- get
      io $ reply rsvp $ case ( lookup jobid $ spcJobsPending state,
                               lookup jobid $ spcJobsDone state
                             ) of
        (Just _, _) -> Just $ JobPending
        (_, Just _) -> Just $ JobDone DoneCancelled
        _ -> Nothing
    MsgJobCancel jobid -> do
      state <- get
      case lookup jobid $ spcJobsPending state of
        Nothing -> pure ()
        Just _ ->
          put $
```

```
            state
              { spcJobsPending = removeAssoc jobid $ spcJobsPending state,
                spcJobsDone = (jobid, DoneCancelled) : spcJobsDone state
              }

jobCancel :: SPC -> JobId -> IO ()
jobCancel (SPC c) jobid =
  sendTo c $ MsgJobCancel jobid

testCase "canceling job" $ do
  spc <- startSPC
  j <- jobAdd spc $ Job (pure ()) 1
  jobCancel spc j
  r <- jobStatus spc j
  r @?= Just (JobDone DoneCancelled)
```

Job Waiting

This task adds support for synchronously waiting for a job to finish, and receiving the `JobDoneReason` in response. Currently we can only wait for jobs to be cancelled, but this will be quite useful functionality once we implement actual job execution.

```
-- | Synchronously block until job is done and return the reason.
-- Returns 'Nothing' if job is not known to this SPC instance.
jobWait :: SPC -> JobId -> IO (Maybe JobDoneReason)
jobWait (SPC c) jobid =
  requestReply c $ MsgJobWait jobid
```

This is more tricky to handle than the other messages. We must send a reply *eventually*, but we cannot do so until the job is done, which it may not be yet. This means we must maintain a list pairing `JobId`s with channels to send a message to once that job finishes. Then, when a job finishes (currently only possible when it is cancelled), we must send a response to all relevant channels.

Hints

1. Extend `SPCMsg` with a new message for waiting on a given job to finish.
2. Extend `SPCState` with with a field of type `[(JobId, ReplyChan (Maybe JobDoneReason))]`.
3. Extend `handleMsg` to handle the new message added in step 1.
4. Modify `handleMsg`'s logic for job cancellation to see if anyone is waiting for the job to be done (by inspecting the state list added in step 2). If so, send `DoneCancelled` on the corresponding channels and remove the waiters from the list.

Solution

Open this to see the implementation

```
data SPCMsg
  = ...
  | MsgJobWait JobId (ReplyChan (Maybe JobDoneReason))

data SPCState = SPCState
  { ...
    spcWaiting :: [(JobId, ReplyChan (Maybe JobDoneReason))]
  }

handleMsg :: Chan SPCMsg -> SPCM ()
handleMsg c = do
  msg <- io $ receive c
  case msg of
    ...
    MsgJobCancel jobid -> do
      state <- get
      case lookup jobid $ spcJobsPending state of
        Nothing -> pure ()
        Just _ -> do
          let (waiting_for_jobid, rest) =
```

```
              partition ((== jobid) . fst) $ spcWaiting state
          forM_ waiting_for_jobid $ \(_, rsvp) ->
            io $ reply rsvp $ Just DoneCancelled
          put $
            state
              { spcJobsPending = removeAssoc jobid $ spcJobsPending state,
                spcJobsDone = (jobid, DoneCancelled) : spcJobsDone state,
                spcWaiting = rest
              }
    MsgJobWait jobid rsvp -> do
      state <- get
      case lookup jobid $ spcJobsDone state of
        Just reason -> do
          io $ reply rsvp $ Just reason
        Nothing ->
          if jobid >= spcJobCounter state
            then io $ reply rsvp Nothing
            else put $ state {spcWaiting = (jobid, rsvp) : spcWaiting state}

jobWait :: SPC -> JobId -> IO (Maybe JobDoneReason)
jobWait (SPC c) jobid =
  requestReply c $ MsgJobWait jobid

startSPC :: IO SPC
startSPC = do
  let initialState c =
        SPCState
         { ...
         , spcChan = c
         , spcJobRunning = Nothing
         }
  server <- spawn $ \c -> runSPCM (initialState c) $ forever $ handleMsg c
  pure $ SPC server
```

Job Execution

In this task you will add support for actually executing a job. We will initially ignore the possibility of timeouts and crashes and return to these concerns in later tasks.

The semantics of job execution are simple: when a job is enqueued, it will eventually be executed. Only a single job is under execution at any given time. If the scheduler is idle, and there are pending jobs, a pending job should be chosen and begin execution.

Operationally, when SPC executes a job, it spawns a new thread (with `forkIO`) which runs the `jobAction`. We call this a *job thread* When the action is done, the job thread will send a message to SPC. Only one job thread should be running at any given time. To support `jobCancel` and `jobStatus`, SPC must track the currently running job (if any).

Hints

0. We will now have multiple ways that a job can finish, so add a function `jobDone :: JobId -> JobDoneReason -> SPCM ()` that contains the book-keeping code from in your handling of the cancellation message. Modify `handleEvent` to use this function. Ensure that your existing tests still work.
1. Extend `SPCState` with a field tracking the currently running job, which must contain a `JobId` and a `ThreadId`.
2. Extend `SPCMsg` with a message that is sent by a job thread when its job is done.
3. Define a function with signature `schedule :: SPCM ()`. When `schedule` is executed, it checks whether a job is *not* currently running, and if there are any *pending* jobs. If so, it launches a thread as discussed above and updates the SPC state appropriately.
4. Insert a call to `schedule` at an appropriate location (such as the beginning of `handleMsg`).
5. Handle the message you added in step (2) in `handleMsg`. Use the `jobDone` function you added above.

6. Write tests. Since jobs are executed only for side effects and do not return any values, we need to write test jobs with observable side effects. The easiest way is for the test to create an `IORef` whose value is changed by the job.

Solution

Open this to see the implementation

```
data SPCMsg
  = ...
  | MsgJobDone JobId

data SPCState = SPCState
  { ...
    spcChan :: Chan SPCMsg,
    spcJobRunning :: Maybe (JobId, ThreadId)
  }

schedule :: SPCM ()
schedule = do
  state <- get
  case (spcJobRunning state, spcJobsPending state) of
    (Nothing, (jobid, job) : jobs) -> do
      t <- io $ forkIO $ do
        jobAction job
        send (spcChan state) $ MsgJobDone jobid
      put $
        state
          { spcJobRunning = Just (jobid, t),
            spcJobsPending = jobs
          }
    _ -> pure ()

jobDone :: JobId -> JobDoneReason -> SPCM ()
jobDone jobid reason = do
  state <- get
  case lookup jobid $ spcJobsDone state of
    Just _ ->
      -- We already know this job is done.
      pure ()
    Nothing -> do
      let (waiting_for_job, not_waiting_for_job) =
            partition ((== jobid) . fst) (spcWaiting state)
      forM_ waiting_for_job $ \(_, rsvp) ->
        io $ reply rsvp $ Just reason
      put $
        state
          { spcWaiting = not_waiting_for_job,
            spcJobsDone = (jobid, reason) : spcJobsDone state,
            spcJobRunning = Nothing,
            spcJobsPending = removeAssoc jobid $ spcJobsPending state
          }

handleMsg :: Chan SPCMsg -> SPCM ()
handleMsg c = do
  schedule
  msg <- io $ receive c
  case msg of
    ...
    MsgJobStatus jobid rsvp -> do
      state <- get
      io $ reply rsvp $ case ( lookup jobid $ spcJobsPending state,
                               spcJobRunning state,
                               lookup jobid $ spcJobsDone state
                             ) of
        (Just _, _, _) -> Just JobPending
        (_, Just (running_job, _), _)
          | running_job == jobid ->
```

```
              Just $ JobRunning
        (_, _, Just r) -> Just $ JobDone r
        _ -> Nothing
    MsgJobDone done_jobid -> do
      state <- get
      case spcJobRunning state of
        Just (jobid, _)
          | jobid == done_jobid ->
              jobDone jobid Done
        _ -> pure ()
    MsgJobCancel cancel_jobid -> do
      state <- get
      case spcJobRunning state of
        Just (jobid, tid) | jobid == cancel_jobid -> do
          io $ killThread tid
          jobDone jobid DoneCancelled
        _ -> pure ()
```

Test case.

```
testCase "running job" $ do
  ref <- newIORef False
  spc <- startSPC
  j <- jobAdd spc $ Job (writeIORef ref True) 1
  r <- jobWait spc j
  r @?= Just Done
  x <- readIORef ref
  x @?= True
```

Handling Crashing Jobs

In this task we will handle the situation where a job throws an exception during execution. When this occurs, the job should be considered done, with a `JobDoneReason` of `DoneCrashed`.

Solution

Open this to see the implementation

```
data SPCMsg
  = ...
  | MsgJobDone JobId
  | MsgJobCrashed JobId

schedule :: SPCM ()
schedule = do
  state <- get
  case (spcJobRunning state, spcJobsPending state) of
    (Nothing, (jobid, job) : jobs) -> do
      t <- io $ forkIO $ do
        let doJob = do
              jobAction job
              send (spcChan state) $ MsgJobDone jobid
            onException :: SomeException -> IO ()
            onException _ =
              send (spcChan state) $ MsgJobCrashed jobid
        doJob `catch` onException
      now <- io $ getSeconds
      put $
        state
          { spcJobRunning = Just (jobid, t),
            spcJobsPending = jobs
          }
    _ -> pure ()

handleMsg :: Chan SPCMsg -> SPCM ()
handleMsg c = do
  schedule
```

```
  msg <- io $ receive c
  case msg of
    ...
    MsgJobCrashed crashed_jobid -> do
      state <- get
      case spcJobRunning state of
        Just (jobid, tid) | jobid == crashed_jobid ->
          jobDone jobid DoneCrashed
        _ -> pure ()
```

Test case:

```
testCase "crash" $ do
  spc <- startSPC
  j1 <- jobAdd spc $ Job (error "boom") 1
  r1 <- jobWait spc j1
  r1 @?= Just DoneCrashed
  -- Ensure new jobs can still work.
  ref <- newIORef False
  j2 <- jobAdd spc $ Job (writeIORef ref True) 1
  r2 <- jobWait spc j2
  r2 @?= Just Done
  v <- readIORef ref
  v @?= True
```

Handling Timeouts

In this task we will enforce the timeout contained in the `Job` datatype. If a job runtime exceeds its timeout, it will be terminated, and its `JobDoneReason` set to `DoneTimeout`.

The key trick is based on the observation that SPC is only able to take action in response to a message, but a task stuck in an infinite loop sends no messages. Therefore we will add a trivial message with no payload (called a "tick") and arrange for a thread to exist that does nothing but infinitely send these tick messages to SPC every second or so. The specific timing does not matter, as the tick messages are not for keeping track of time (we use `getSeconds` for that), but merely to give the server a chance to act.

We further augment the state field tracking the currently running job to contain a *deadline*. When the current time (as retrieved by `getSeconds`) exceeds the deadline, the task must be terminated.

Hints

0. Extend `SPCMsg` with a tick message type.
1. Modify `SPCState` such that a running job is associated with a deadline of type `Seconds`.
2. Modify `schedule` such that when a job begins execution, its deadline is set to the current time plus `jobMaxSeconds` of the job.
3. Add a function with signature `checkTimeouts :: SPCM ()` that checks whether the current job has exceeded its deadlne. If so, destroy the job thread with `killThread` and use `jobDone` to register the fact that the job is now done.
4. Extend `handleMsg` to call `checkTimeouts` as appropriate and handle the new tick message type.
5. Modify `startSPC` to launch a thread that sends a tick message to SPC every second.

Solution

Open this to see the implementation

```
data SPCMsg =
  ...
  | MsgTick

data SPCState = SPCState
  { ...
    spcJobRunning :: Maybe (JobId, Seconds, ThreadId)
  }
```

```
schedule :: SPCM ()
schedule = do
  state <- get
  case (spcJobRunning state, spcJobsPending state) of
    (Nothing, (jobid, job) : jobs) -> do
      t <- io $ forkIO $ do
        let doJob = do
              jobAction job
              send (spcChan state) $ MsgJobDone jobid
            onException :: SomeException -> IO ()
            onException _ =
              send (spcChan state) $ MsgJobCrashed jobid
        doJob `catch` onException
      now <- io $ getSeconds
      let deadline = now + fromIntegral (jobMaxSeconds job)
      put $
        state
          { spcJobRunning = Just (jobid, deadline, t),
            spcJobsPending = jobs
          }
    _ -> pure ()

checkTimeouts :: SPCM ()
checkTimeouts = do
  state <- get
  now <- io getSeconds
  case spcJobRunning state of
    Just (jobid, deadline, tid)
      | now >= deadline -> do
          io $ killThread tid
          jobDone jobid DoneTimeout
    _ -> pure ()

handleMsg :: Chan SPCMsg -> SPCM ()
handleMsg c = do
  checkTimeouts
  ...
  case msg of
    ...
    MsgTick ->
      pure ()

startSPC :: IO SPC
startSPC = do
  ...
  server <- spawn $ \c -> runSPCM (initial_state c) $ forever $ handleMsg c
  void $ spawn $ timer server
  pure $ SPC server
  where
    timer server _ = forever $ do
      threadDelay 1000000 -- 1 second
      sendTo server MsgTick
```

Test case:

```
testCase "timeout" $ do
  spc <- startSPC
  ref <- newIORef False
  j <- jobAdd spc $ Job (threadDelay 2000000 >> writeIORef ref True) 1
  r1 <- jobStatus spc j
  r1 @?= Just JobRunning
  r2 <- jobWait spc j
  r2 @?= Just DoneTimeout,
```

Week 6 solutions

GenServer.hs

```
module GenServer
  ( Chan,
    Server,
    receive,
    send,
    sendTo,
    spawn,
    ReplyChan,
    requestReply,
    reply,
  )
where

import Control.Concurrent (Chan)
import qualified Control.Concurrent as CC

data Server msg = Server CC.ThreadId (Chan msg)

data ReplyChan a = ReplyChan (Chan a)

send :: Chan a -> a -> IO ()
send chan msg =
  CC.writeChan chan msg

sendTo :: Server a -> a -> IO ()
sendTo (Server _tid input) msg =
  send input msg

receive :: Chan a -> IO a
receive = CC.readChan

spawn :: (Chan a -> IO ()) -> IO (Server a)
spawn server = do
  input <- CC.newChan
  tid <- CC.forkIO $ server input
  pure $ Server tid input

requestReply :: Server a -> (ReplyChan b -> a) -> IO b
requestReply serv con = do
  reply_chan <- CC.newChan
  sendTo serv $ con $ ReplyChan reply_chan
  receive reply_chan

reply :: ReplyChan a -> a -> IO ()
reply (ReplyChan chan) x = send chan x
```

SPC.hs

```
module SPC
  ( -- * SPC startup
    SPC,
    startSPC,

    -- * Job functions
    Job (..),
    JobId,
    JobStatus (..),
    JobDoneReason (..),
    jobAdd,
    jobStatus,
    jobWait,
    jobCancel,
  )
where

import Control.Concurrent
  ( ThreadId,
    forkIO,
    killThread,
    threadDelay,
  )
import Control.Exception (SomeException, catch)
import Control.Monad (ap, forM_, forever, liftM, void)
import Data.List (partition)
import GenServer
import System.Clock.Seconds (Clock (Monotonic), Seconds, getTime)

-- First some general utility functions.

-- | Retrieve Unix time using a monotonic clock. You cannot use this
-- to measure the actual world time, but you can use it to measure
-- elapsed time.
getSeconds :: IO Seconds
getSeconds = getTime Monotonic

-- | Remove mapping from association list.
removeAssoc :: (Eq k) => k -> [(k, v)] -> [(k, v)]
removeAssoc needle ((k, v) : kvs) =
  if k == needle
    then kvs
    else (k, v) : removeAssoc needle kvs
removeAssoc _ [] = []

-- Then the definition of the glorious SPC.

-- | A job that is to be enqueued in the glorious SPC.
data Job = Job
  { -- | The IO action that comprises the actual action of the job.
    jobAction :: IO (),
    -- | The maximum allowed runtime of the job, counting from when
    -- the job begins executing (not when it is enqueued).
    jobMaxSeconds :: Int
  }

-- | A unique identifier of a job that has been enqueued.
newtype JobId = JobId Int
  deriving (Eq, Ord, Show)

-- | How a job finished.
data JobDoneReason
  = -- | Normal termination.
    Done
  | -- | The job was killed because it ran for too long.
    DoneTimeout
```

```
  | -- | The job was explicitly cancelled.
    DoneCancelled
  | -- | The job crashed due to an exception.
    DoneCrashed
  deriving (Eq, Ord, Show)

-- | The status of a job.
data JobStatus
  = -- | The job is done and this is why.
    JobDone JobDoneReason
  | -- | The job is still running.
    JobRunning
  | -- | The job is enqueued, but is waiting for an idle worker.
    JobPending
  deriving (Eq, Ord, Show)

-- Messages sent to SPC.
data SPCMsg
  = -- | Add the job, and reply with the job ID.
    MsgJobAdd Job (ReplyChan JobId)
  | -- | Cancel the given job.
    MsgJobCancel JobId
  | -- | Immediately reply the status of the job.
    MsgJobStatus JobId (ReplyChan (Maybe JobStatus))
  | -- | Reply when the job is done.
    MsgJobWait JobId (ReplyChan (Maybe JobDoneReason))
  | -- | Job has finished.
    MsgJobDone JobId
  | -- | Job crashed.
    MsgJobCrashed JobId
  | -- | Some time has passed.
    MsgTick

-- | A Handle to the SPC instance.
data SPC = SPC (Server SPCMsg)

-- | The central state. Must be protected from the bourgeoisie.
data SPCState = SPCState
  { spcChan :: Chan SPCMsg,
    spcJobsPending :: [(JobId, Job)],
    spcJobRunning :: Maybe (JobId, Seconds, ThreadId),
    spcJobsDone :: [(JobId, JobDoneReason)],
    -- | These are waiting for this job to terminate.
    spcWaiting :: [(JobId, ReplyChan (Maybe JobDoneReason))],
    spcJobCounter :: JobId
  }

-- | The monad in which the main SPC thread runs. This is a state
-- monad with support for IO.
newtype SPCM a = SPCM (SPCState -> IO (a, SPCState))

instance Functor SPCM where
  fmap = liftM

instance Applicative SPCM where
  pure x = SPCM $ \state -> pure (x, state)
  (<*>) = ap

instance Monad SPCM where
  SPCM m >>= f = SPCM $ \state -> do
    (x, state') <- m state
    let SPCM f' = f x
    f' state'

-- | Retrieve the state.
get :: SPCM SPCState
get = SPCM $ \state -> pure (state, state)

```

```
-- | Overwrite the state.
put :: SPCState -> SPCM ()
put state = SPCM $ \_ -> pure ((), state)

-- | Lift an 'IO' action into 'SPCM'.
io :: IO a -> SPCM a
io m = SPCM $ \state -> do
  x <- m
  pure (x, state)

-- | Run the SPCM monad.
runSPCM :: SPCState -> SPCM a -> IO a
runSPCM state (SPCM f) = fst <$> f state

schedule :: SPCM ()
schedule = do
  state <- get
  case (spcJobRunning state, spcJobsPending state) of
    (Nothing, (jobid, job) : jobs) -> do
      t <- io $ forkIO $ do
        let doJob = do
              jobAction job
              send (spcChan state) $ MsgJobDone jobid
            onException :: SomeException -> IO ()
            onException _ =
              send (spcChan state) $ MsgJobCrashed jobid
        doJob `catch` onException
      now <- io $ getSeconds
      let deadline = now + fromIntegral (jobMaxSeconds job)
      put $
        state
          { spcJobRunning = Just (jobid, deadline, t),
            spcJobsPending = jobs
          }
    _ -> pure ()

-- Precondition: 'jobid' is currently running.
jobDone :: JobId -> JobDoneReason -> SPCM ()
jobDone jobid reason = do
  state <- get
  case lookup jobid $ spcJobsDone state of
    Just _ ->
      -- We already know this job is done.
      pure ()
    Nothing -> do
      let (waiting_for_job, not_waiting_for_job) =
            partition ((== jobid) . fst) (spcWaiting state)
      forM_ waiting_for_job $ \(_, rsvp) ->
        io $ reply rsvp $ Just reason
      put $
        state
          { spcWaiting = not_waiting_for_job,
            spcJobsDone = (jobid, reason) : spcJobsDone state,
            spcJobRunning = Nothing,
            spcJobsPending = removeAssoc jobid $ spcJobsPending state
          }

checkTimeouts :: SPCM ()
checkTimeouts = do
  state <- get
  now <- io getSeconds
  case spcJobRunning state of
    Just (jobid, deadline, tid)
      | now >= deadline -> do
          io $ killThread tid
          jobDone jobid DoneTimeout
    _ -> pure ()

```

```
203 handleMsg :: Chan SPCMsg -> SPCM ()
204 handleMsg c = do
205   checkTimeouts
206   schedule
207   msg <- io $ receive c
208   case msg of
209     MsgJobAdd job rsvp -> do
210       state <- get
211       let JobId jobid = spcJobCounter state
212       put $
213         state
214           { spcJobsPending =
215               (spcJobCounter state, job) : spcJobsPending state,
216             spcJobCounter = JobId $ succ jobid
217           }
218       io $ reply rsvp $ JobId jobid
219     MsgJobStatus jobid rsvp -> do
220       state <- get
221       io $ reply rsvp $ case ( lookup jobid $ spcJobsPending state,
222                                spcJobRunning state,
223                                lookup jobid $ spcJobsDone state
224                              ) of
225         (Just _, _, _) -> Just JobPending
226         (_, Just (running_job, _, _), _)
227           | running_job == jobid ->
228               Just $ JobRunning
229         (_, _, Just r) -> Just $ JobDone r
230         _ -> Nothing
231     MsgJobWait jobid rsvp -> do
232       state <- get
233       case lookup jobid $ spcJobsDone state of
234         Just reason -> do
235           io $ reply rsvp $ Just reason
236         Nothing ->
237           if jobid >= spcJobCounter state
238             then io $ reply rsvp Nothing
239             else put $ state {spcWaiting = (jobid, rsvp) : spcWaiting state}
240     MsgJobDone done_jobid -> do
241       state <- get
242       case spcJobRunning state of
243         Just (jobid, _, _)
244           | jobid == done_jobid ->
245               jobDone jobid Done
246         _ -> pure ()
247     MsgJobCancel cancel_jobid -> do
248       state <- get
249       case spcJobRunning state of
250         Just (jobid, _, tid) | jobid == cancel_jobid -> do
251           io $ killThread tid
252           jobDone jobid DoneCancelled
253         _ -> pure ()
254     MsgJobCrashed crashed_jobid -> do
255       state <- get
256       case spcJobRunning state of
257         Just (jobid, _, tid) | jobid == crashed_jobid -> do
258           io $ killThread tid
259           jobDone jobid DoneCrashed
260         _ -> pure ()
261     MsgTick ->
262       pure ()
263
264 startSPC :: IO SPC
265 startSPC = do
266   let initial_state c =
267         SPCState
268           { spcJobCounter = JobId 0,
269             spcJobsPending = [],
270             spcJobRunning = Nothing,
```

```
            spcJobsDone = [],
            spcWaiting = [],
            spcChan = c
          }
  server <- spawn $ \c -> runSPCM (initial_state c) $ forever $ handleMsg c
  void $ spawn $ timer server
  pure $ SPC server
  where
    timer server _ = forever $ do
      threadDelay 1000000 -- 1 second
      sendTo server MsgTick

-- | Add a job for scheduling.
jobAdd :: SPC -> Job -> IO JobId
jobAdd (SPC c) job =
  requestReply c $ MsgJobAdd job

-- | Query the job status. Returns 'Nothing' if job is not known to
-- this SPC instance.
jobStatus :: SPC -> JobId -> IO (Maybe JobStatus)
jobStatus (SPC c) jobid =
  requestReply c $ MsgJobStatus jobid

-- | Synchronously block until job is done and return the reason.
-- Returns 'Nothing' if job is not known to this SPC instance.
jobWait :: SPC -> JobId -> IO (Maybe JobDoneReason)
jobWait (SPC c) jobid =
  requestReply c $ MsgJobWait jobid

-- | Asynchronously cancel a job.
jobCancel :: SPC -> JobId -> IO ()
jobCancel (SPC c) jobid =
  sendTo c $ MsgJobCancel jobid
```

SPC_Tests.hs

```
module SPC_Tests (tests) where

import Control.Concurrent (threadDelay)
import Control.Monad (replicateM)
import Data.IORef
import SPC
import Test.Tasty (TestTree, localOption, mkTimeout, testGroup)
import Test.Tasty.HUnit (testCase, (@?=))

tests :: TestTree
tests =
  localOption (mkTimeout 3000000) $
    testGroup
      "SPC"
      [ testCase "simple-job" $ do
          spc <- startSPC
          ref <- newIORef False
          j <- jobAdd spc $ Job (writeIORef ref True) 1
          r1 <- jobStatus spc j
          r1 @?= Just JobRunning
          r2 <- jobWait spc j
          r2 @?= Just Done
          v <- readIORef ref
          v @?= True,
        testCase "multiple-jobs" $ do
          spc <- startSPC
          let num_jobs = 10
          ref <- newIORef (0 :: Int)
          js <- replicateM num_jobs $ jobAdd spc $ Job (modifyIORef ref (+ 1)) 1
          rs <- mapM (jobWait spc) js
          rs @?= replicate num_jobs (Just Done)
          v <- readIORef ref
          v @?= 10,
        testCase "timeout" $ do
          spc <- startSPC
          ref <- newIORef False
          j <- jobAdd spc $ Job (threadDelay 2000000 >> writeIORef ref True) 1
          r1 <- jobStatus spc j
          r1 @?= Just JobRunning
          r2 <- jobWait spc j
          r2 @?= Just DoneTimeout,
        testCase "cancel" $ do
          spc <- startSPC
          ref <- newIORef False
          j <- jobAdd spc $ Job (threadDelay 20000000 >> writeIORef ref True) 1
          r1 <- jobStatus spc j
          r1 @?= Just JobRunning
          jobCancel spc j
          r2 <- jobWait spc j
          r2 @?= Just DoneCancelled,
        testCase "crash" $ do
          spc <- startSPC
          j1 <- jobAdd spc $ Job (error "boom") 1
          r1 <- jobWait spc j1
          r1 @?= Just DoneCrashed
          -- Ensure new jobs can still work.
          ref <- newIORef False
          j2 <- jobAdd spc $ Job (writeIORef ref True) 1
          r2 <- jobWait spc j2
          r2 @?= Just Done
          v <- readIORef ref
          v @?= True,
        testCase "unknown job" $ do
          spc1 <- startSPC
          j1 <- jobAdd spc1 $ Job (pure ()) 1
          r1 <- jobWait spc1 j1
```

```
67            r1 @?= Just Done
68            spc2 <- startSPC
69            r2 <- jobWait spc2 j1
70            r2 @?= Nothing
71        ]
```

Advanced Programming 2026

Assignment Six

Concurrent Programming

Due: 2026-10-22

Synopsis: Implementing a concurrent job scheduling system with support for timeouts and error handling.

1 Introduction

The main limitation of the Stateful Planning Committee (SPC), developed during the exercises, is that it supports only a single job at any given time, meaning there is no true concurrency. In this assignment we will extend SPC with a notion of *workers*, which are persistent threads that receive work orders from the central SPC thread. The number of workers can vary during the lifetime of an SPC session. Workers are explicitly created and destroyed through API functions.

The starting point of your work is the SPC developed during the exercises, except that the existing scheduling/execution machinery has been removed. You will be gradually reconstructing the removed functionality throughout the assignment, including support for timeouts and exceptions. The main challenge in this assignment is designing the protocol used for communication between SPC and its workers.

You may not modify the types of any functions exported from the `SPC` module, but you may (and are expected) to modify any internal definitions as you see fit.

1.1 Concepts

A worker can be either *idle*, if it is not currently executing a job, or *busy*, which means it is executing a job. Each worker is represented by a *worker thread*, which communicates with SPC through messages. A worker may launch further threads as necessary.

Initially, an SPC instance has no workers, and any submitted jobs will be pending. At any time, if there is at least one pending job and one idle

worker, a pending job and an idle worker will be picked, and that worker asked to execute that job. The worker is now considered busy.

When a worker finishes with a job, it notifies SPC. The worker is now considered idle. SPC then notifies anyone waiting for completion of the job and performs other necessary bookkeeping.

All workers have a name of type `String`. These have no semantic meaning, but can be used for debugging and logging. One invariant that must hold is that no two workers may have the same name.

An SPC instance with k workers can at any given time be executing at most k jobs concurrently.

2 Task: Adding Workers

A fresh SPC instance has zero workers. A worker can be added with the following function:

```
workerAdd
  :: SPC -> WorkerName -> IO (Either String Worker)
```

The return value is `Left` with an error message if anything goes wrong. One thing that can go wrong is that a worker with the specified name already exists. Otherwise, the return value is `Right` alongside a handle to a worker, of type `Worker`. This handle is used by API clients for interacting with the worker in later tasks.

A worker starts out idle and can immediately be used by SPC for executing jobs.

You should also add tests.

At the conclusion of this task, `jobAdd`, `jobStatus`, and `jobWait` should work for the cases where a job neither times out nor crashes.

2.1 Hints

A worker is essentially a kind of server, just like SPC itself. A worker will need to have access to the `Chan SPCMsg` used for sending messages to SPC. The handout contains a message type, `WorkerMsg`, that you should add messages to, just as you did with `SPCMsg`. Similarly, you will need to add new messages to `SPCMsg` for the messages that workers will send to SPC.

A minimal solution will involve extending `WorkerMsg` with a message that contains the job to be done (of type `IO ()`), and `SPCMsg` with a message that a worker sends when it is done with a job. This message should contain the worker name so SPC can know from which worker it received it.

You do not need to worry about timeouts or exceptions for now—we will return to that in later tasks.

It is probably overkill to define a monad for workers the way you did for SPC with `SPCM`, as their state is much simpler.

3 Task: Job Cancellation

In this task you will restore support for job cancellation:

```
jobCancel :: SPC -> JobId -> IO ()
```

The semantics are identical to the version that you implemented in the exercises, but now SPC must communicate the cancellation to the worker (if any) executing the job.

4 Task: Timeouts

In this task you will restore support for timeouts (the `jobMaxSeconds` field of the `Job` type). The semantics are the same as in the exercises: if a job exceeds its time limit, it will be terminated. Further, the worker executing the job must survive this ordeal, and become idle again.

As before, when a job is terminated due to timeout, its final status will be `JobDone DoneTimeout`.

4.1 Hints

The major design decision is to decide whether to enforce the timeouts centrally in the SPC thread, or decentrally in the worker threads. Specifically, either SPC can tell a busy worker that it should abort its current job, or a worker can itself abort after a timeout and inform SPC that the job took too long. Which of these approaches you implement is up to you.

The operational way you notice the timeouts is similar to in the exercises: have some sort of thread deliver regular "ping" messages, and check whether some job has exceeded its deadline.

Depending on your design, you will probably need to extend both `WorkerMsg` and `SPCMsg`.

5 Task: Exceptions

In this task you will restore support for robust handling of exceptions. The most important property is that a job that crashes should not harm the worker. Instead, SPC should be notified, and the worker once again become idle.

As before, when a job terminates with an exception, its final status will be `JobDone DoneCrashed`.

5.1 Hints

You will need to catch exceptions in the workers, just as in the exercises. The logic is essentially the same.

6 Task: Removing Workers

In this task you will add support for this function:

```
workerStop :: Worker -> IO ()
```

The function `workerStop` stops (i.e., shuts down) a worker. It has no effect if the worker is already stopped. If the worker is busy with a job when the worker is stopped, then it is equivalent to first cancelling the job, then stopping the worker.

7 Code handout

The code handout consists of the following nontrivial files.

- `a6.cabal`: Cabal build file. **Do not modify this file.**
- `runtests.hs`: Test execution program. **Do not modify this file.**

- `src/GenServer.hs`: The GenServer library. **Do not modify this file.**
- `src/SPC.hs`: Implementation of the glorious SPC.
- `src/SPC_Tests.hs`: Test suite, where you will add plentiful tests.

8 Your Report

You are expected to comment on the *interesting* details of your implementation. You are *not* expected to give a line-by-line walkthrough of your code. Most importantly, you are expected to reflect on the *quality* of your code:

- Do you think it is functionally correct? Why or why not?
- Is there some improvement you'd have liked to make, but didn't have the time?

It is more important to be aware of the strengths or shortcomings of your solution, than it is to have a complete solution.

8.1 The structure of your report

Your report must be structured exactly as follows:

Introduction: Briefly mention general concerns, any ambiguities in the problem text and how you resolved them, and your own estimation of the quality of your solution. Briefly mention whether your solution is functional, which test cases cover its functionality, which test cases it fails for (if any), and what you think might be wrong.

A section answering the following numbered questions:

1. What happens when a job is enqueued in an SPC instance with no workers, and a worker is added later? Which of your tests demonstrate this?
2. Did you decide to implement timeouts centrally in SPC, or decentrally in the worker threads? What are the pros and cons of your approach? Is there any observable effect?

3. Which of your tests verify that if a worker executes a job that is cancelled, times out, or crashes, then that worker can still be used to execute further jobs?
4. If a worker thread were somehow to crash (either due to a bug in the worker thread logic, or because a scoundrel `killThread`s it), how would that impact the functionality of the rest of SPC?
5. Suppose a client calls `jobCancel` (which is asynchronous) and then immediately `jobStatus`. Can we expect this always produces `DoneCancelled`, or is there a risk of race condition?

All else being equal, **a short report is a good report**.

9 Deliverables for This Assignment

You must submit the following items:

- A single PDF file, A4 size, no more than 5 pages, structured as specified in the report section above.
- A single zip/tar.gz file with all code relevant to the implementation, including at least all the files from the handout. For this assignment it is not necessary to add additional files.

Remember to follow the general assignment rules listed on the course homepage.

10 Assessment

You will get written qualitative feedback, and points from zero to four. There are no resubmissions, so please hand in what you managed to do, even if you have not solved the assignment completely.

A6 handout

GenServer.hs

```
module GenServer
  ( Chan,
    Server,
    receive,
    send,
    sendTo,
    spawn,
    ReplyChan,
    requestReply,
    reply,
  )
where

import Control.Concurrent (Chan)
import qualified Control.Concurrent as CC

data Server msg = Server CC.ThreadId (Chan msg)

data ReplyChan a = ReplyChan (Chan a)

send :: Chan a -> a -> IO ()
send chan msg =
  CC.writeChan chan msg

sendTo :: Server a -> a -> IO ()
sendTo (Server _tid input) msg =
  send input msg

receive :: Chan a -> IO a
receive = CC.readChan

spawn :: (Chan a -> IO ()) -> IO (Server a)
spawn server = do
  input <- CC.newChan
  tid <- CC.forkIO $ server input
  pure $ Server tid input

requestReply :: Server a -> (ReplyChan b -> a) -> IO b
requestReply serv con = do
  reply_chan <- CC.newChan
  sendTo serv $ con $ ReplyChan reply_chan
  receive reply_chan

reply :: ReplyChan a -> a -> IO ()
reply (ReplyChan chan) x = send chan x
```

SPC.hs

```
module SPC
  ( -- * SPC startup
    SPC,
    startSPC,

    -- * Job functions
    Job (..),
    JobId,
    JobStatus (..),
    JobDoneReason (..),
    jobAdd,
    jobStatus,
    jobWait,
    jobCancel,

    -- * Worker functions
    WorkerName,
    workerAdd,
    workerStop,
  )
where

import Control.Concurrent
  ( forkIO,
    killThread,
    threadDelay,
  )
import Control.Monad (ap, forever, liftM, void)
import GenServer
import System.Clock.Seconds (Clock (Monotonic), Seconds, getTime)

-- First some general utility functions.

-- | Retrieve Unix time using a monotonic clock. You cannot use this
-- to measure the actual world time, but you can use it to measure
-- elapsed time.
getSeconds :: IO Seconds
getSeconds = getTime Monotonic

-- | Remove mapping from association list.
removeAssoc :: (Eq k) => k -> [(k, v)] -> [(k, v)]
removeAssoc needle ((k, v) : kvs) =
  if k == needle
    then kvs
    else (k, v) : removeAssoc needle kvs
removeAssoc _ [] = []

-- Then the definition of the glorious SPC.

-- | A job that is to be enqueued in the glorious SPC.
data Job = Job
  { -- | The IO action that comprises the actual action of the job.
    jobAction :: IO (),
    -- | The maximum allowed runtime of the job, counting from when
    -- the job begins executing (not when it is enqueued).
    jobMaxSeconds :: Int
  }

-- | A unique identifier of a job that has been enqueued.
newtype JobId = JobId Int
  deriving (Eq, Ord, Show)

-- | How a job finished.
data JobDoneReason
  = -- | Normal termination.
    Done
```

```
67    | -- | The job was killed because it ran for too long.
68      DoneTimeout
69    | -- | The job was explicitly cancelled, or the worker
70      -- it was running on was stopped.
71      DoneCancelled
72    | -- | The job crashed due to an exception.
73      DoneCrashed
74    deriving (Eq, Ord, Show)
75
76  -- | The status of a job.
77  data JobStatus
78    = -- | The job is done and this is why.
79      JobDone JobDoneReason
80    | -- | The job is still running.
81      JobRunning
82    | -- | The job is enqueued, but is waiting for an idle worker.
83      JobPending
84    deriving (Eq, Ord, Show)
85
86  -- | A worker decides its own human-readable name. This is useful for
87  -- debugging.
88  type WorkerName = String
89
90  -- | Messages sent to workers. These are sent both by SPC and by
91  -- processes spawned by the workes.
92  data WorkerMsg -- TODO: add messages.
93
94  -- Messages sent to SPC.
95  data SPCMsg
96    = -- | Add the job, and reply with the job ID.
97      MsgJobAdd Job (ReplyChan JobId)
98    | -- | Cancel the given job.
99      MsgJobCancel JobId
100   | -- | Immediately reply the status of the job.
101     MsgJobStatus JobId (ReplyChan (Maybe JobStatus))
102   | -- | Reply when the job is done.
103     MsgJobWait JobId (ReplyChan (Maybe JobDoneReason))
104   | -- | Some time has passed.
105     MsgTick
106
107 -- | A handle to the SPC instance.
108 data SPC = SPC (Server SPCMsg)
109
110 -- | A handle to a worker.
111 data Worker = Worker (Server WorkerMsg)
112
113 -- | The central state. Must be protected from the bourgeoisie.
114 data SPCState = SPCState
115   { spcJobsPending :: [(JobId, Job)],
116     spcJobsRunning :: [(JobId, Job)],
117     spcJobsDone :: [(JobId, JobDoneReason)],
118     spcJobCounter :: JobId
119     -- TODO: you will need to add more fields.
120   }
121
122 -- | The monad in which the main SPC thread runs. This is a state
123 -- monad with support for IO.
124 newtype SPCM a = SPCM (SPCState -> IO (a, SPCState))
125
126 instance Functor SPCM where
127   fmap = liftM
128
129 instance Applicative SPCM where
130   pure x = SPCM $ \state -> pure (x, state)
131   (<*>) = ap
132
133 instance Monad SPCM where
134   SPCM m >>= f = SPCM $ \state -> do
```

```
135     (x, state') <- m state
136     let SPCM f' = f x
137     f' state'
138
139 -- | Retrieve the state.
140 get :: SPCM SPCState
141 get = SPCM $ \state -> pure (state, state)
142
143 -- | Overwrite the state.
144 put :: SPCState -> SPCM ()
145 put state = SPCM $ \_ -> pure ((), state)
146
147 -- | Modify the state.
148 modify :: (SPCState -> SPCState) -> SPCM ()
149 modify f = do
150   state <- get
151   put $ f state
152
153 -- | Lift an 'IO' action into 'SPCM'.
154 io :: IO a -> SPCM a
155 io m = SPCM $ \state -> do
156   x <- m
157   pure (x, state)
158
159 -- | Run the SPCM monad.
160 runSPCM :: SPCState -> SPCM a -> IO a
161 runSPCM state (SPCM f) = fst <$> f state
162
163 schedule :: SPCM ()
164 schedule = undefined
165
166 jobDone :: JobId -> JobDoneReason -> SPCM ()
167 jobDone = undefined
168
169 workerIsIdle :: WorkerName -> Worker -> SPCM ()
170 workerIsIdle = undefined
171
172 workerIsGone :: WorkerName -> SPCM ()
173 workerIsGone = undefined
174
175 checkTimeouts :: SPCM ()
176 checkTimeouts = pure () -- change in Task 4
177
178 workerExists :: WorkerName -> SPCM Bool
179 workerExists = undefined
180
181 handleMsg :: Chan SPCMsg -> SPCM ()
182 handleMsg c = do
183   checkTimeouts
184   schedule
185   msg <- io $ receive c
186   case msg of
187     MsgJobAdd job rsvp -> do
188       state <- get
189       let JobId jobid = spcJobCounter state
190       put $
191         state
192           { spcJobsPending =
193               (spcJobCounter state, job) : spcJobsPending state,
194             spcJobCounter = JobId $ succ jobid
195           }
196       io $ reply rsvp $ JobId jobid
197     MsgJobStatus jobid rsvp -> do
198       state <- get
199       io $ reply rsvp $ case ( lookup jobid $ spcJobsPending state,
200                                lookup jobid $ spcJobsRunning state,
201                                lookup jobid $ spcJobsDone state
202                              ) of
```

```
        (Just _, _, _) -> Just JobPending
        (_, Just _, _) -> Just JobRunning
        (_, _, Just r) -> Just $ JobDone r
        _ -> Nothing

startSPC :: IO SPC
startSPC = do
  let initial_state =
        SPCState
          { spcJobCounter = JobId 0,
            spcJobsPending = [],
            spcJobsRunning = [],
            spcJobsDone = []
          }
  c <- spawn $ \c -> runSPCM initial_state $ forever $ handleMsg c
  void $ spawn $ timer c
  pure $ SPC c
  where
    timer c _ = forever $ do
      threadDelay 1000000 -- 1 second
      sendTo c MsgTick

-- | Add a job for scheduling.
jobAdd :: SPC -> Job -> IO JobId
jobAdd (SPC c) job =
  requestReply c $ MsgJobAdd job

-- | Asynchronously query the job status.
jobStatus :: SPC -> JobId -> IO (Maybe JobStatus)
jobStatus (SPC c) jobid =
  requestReply c $ MsgJobStatus jobid

-- | Synchronously block until job is done and return the reason.
-- Returns 'Nothing' if job is not known to this SPC instance.
jobWait :: SPC -> JobId -> IO (Maybe JobDoneReason)
jobWait (SPC c) jobid =
  requestReply c $ MsgJobWait jobid

-- | Asynchronously cancel a job.
jobCancel :: SPC -> JobId -> IO ()
jobCancel (SPC c) jobid =
  sendTo c $ MsgJobCancel jobid

-- | Add a new worker with this name. Fails with 'Left' if a worker
-- with that name already exists.
workerAdd :: SPC -> WorkerName -> IO (Either String Worker)
workerAdd = undefined

-- | Shut down a running worker. No effect if the worker is already
-- terminated.
workerStop :: Worker -> IO ()
workerStop = undefined
```

SPC_Tests.hs

```
module SPC_Tests (tests) where

import Control.Concurrent (threadDelay)
import Control.Monad (forM, forM_, replicateM)
import Data.IORef
import SPC
import Test.Tasty (TestTree, localOption, mkTimeout, testGroup)
import Test.Tasty.HUnit (assertFailure, testCase, (@?=))

tests :: TestTree
tests =
  localOption (mkTimeout 3000000) $
    testGroup
      "SPC (core)"
      []
```

Week 7 exercises

Exam preparation exercises

Like all exercises, the following exercises are part of the curriculum for Advanced Programming in 2026, and questions at the exam may reference them. It is expected that an attendee at the exam has familiarised themselves with them. In contrast to previous exercises, these ones do not introduce new concepts, but merely emphasize concepts previously seen. Naturally, they build on previous exercises and assignments. There is no corresponding assignment to be handed, and therefore they are much larger than previous weekly exercises.

Introduction

The starting point for these exercises will be a slightly cut down variant of the *AP Language* (APL) that was the topic of several of your assignments. The most significant simplification is that printing is not supported. The `localEnv` function is also implemented in a more direct way.

Your overall goal is to extend APL with various new features focussed on support for concurrency. This is partitioned into several tasks. Some of the tasks have dependencies on each other, which we will note explicitly. If you find yourself struggling with a task, consider attempting another (non-dependent) task. You do not need to solve (or even attempt) all tasks in order to pass, although a good performance across all tasks is necessary for a top grade.

The full ambiguous grammar, including features to be introduced later in this text, is shown below. The new language constructs you will implement are these: tuples, projection, `&&`, and `||`. The semantics of these language constructs will be discussed in the related tasks.

You are given a code handout with a complete AST definition and a partial implementation of a parser and an evaluator. The code handout corresponds roughly to the features developed during the course exercises. The evaluator expresses evaluation through the free monad `EvalM`, for which you will write three interpreters:

- `APL.InterpPure`: the *pure interpreter*, which simply executes the given program sequentially and straightforwardly. Most of this interpreter is already complete in the code handout, although you are asked to make some extensions.
- `APL.InterpSim`: the *simulated concurrent interpreter*, which simulates concurrency without using `IO`. You are asked to implement this interpreter in task D.
- `APL.InterpConcurrent`: the *concurrent interpreter*, which uses true `IO`-based concurrency, based on the SPC job scheduler.

You do not need to define new effect types, as these are already included in the handout. The code handout also includes a complete definition of a slightly simplified version of the SPC job scheduler (which you must not modify), as well as some other skeleton files that you will extend as part of the exam tasks.

You should read the code handout carefully. It contains helpful comments. The handout contains a small collection of tests, of which some will initially fail. A comment connected to each test will mention after which task the test is supposed to work. You are *strongly* advised to add more tests of your own.

When the semantics for a language construct states that something "is an error", it means that you must report an appropriate runtime error if that situation occurs (similar to division by zero). No specific error message is required.

Grammar

APL syntax in EBNF, including all extensions described later in the text. This grammar is ambiguous and contains left recursion, which you will be asked to address. Operator priority is specified in a table below. The new language constructs are described in the corresponding tasks. Whitespace is permitted between all terminals.

```
Atom ::= var
       | int
       | bool
       | "(" ")"
       | "(" Exp ")"
       | "(" Exp Exps ")"
       | "put" Atom Atom
       | "get" Atom
       | Atom "." int
```

```
Exps ::= "," Exp
       | "," Exp Exps

FExp ::= FExp FExp
       | Atom

LExp ::= "if" Exp "then" Exp "else" Exp
       | "\" var "->" Exp
       | "let" var "=" Exp "in" Exp
       | "loop" var "=" Exp "for" var "<" Exp "do" Exp
       | "loop" var "=" Exp "while" Exp "do" Exp
       | FExp

Exp ::= LExp
      | Exp "==" Exp
      | Exp "+" Exp
      | Exp "-" Exp
      | Exp "*" Exp
      | Exp "/" Exp
      | Exp "&&" Exp
      | Exp "||" Exp
```

Operator precedence

Operators	Associativity
Application	Left
`* /`	Left
`+ -`	Left
`==`	Left
`&&`	Left
`||`	Left

Examples

Examples of APL parsing and evaluation follow. Consult these if you find the semantics of some of the language constructs unclear, but consider skipping this section until you reach the relevant tasks.

Tuples

- Syntax: `let x = (1,2) in x.0`
- AST: `Let "x" (Tuple [CstInt 1,CstInt 2]) (Project (Var "x") 0)`
- Evaluation: `ValInt 1`

- Syntax: `(1,2).1`
- AST: Project (Tuple [CstInt 1,CstInt 2]) 1‘
- Evaluation: `ValInt 2`

Concurrency operators

Due to nondeterminism, expressions may produce different results in the pure and simulated/concurrent interpreters. This is noted where relevant. The concurrent interpreter may further produce results that differ from the simulated interpreter, due to nondeterminism—only a single possible result is listed below.

- Syntax: `(1+2) && (3+4)`
- AST: `BothOf (Add (CstInt 1) (CstInt 2)) (Add (CstInt 3) (CstInt 4))`
- Evaluation: `ValTuple [ValInt 3,ValInt 7]`

- Syntax: `(1+2) || (3+4+5+6)`

- AST: `OneOf (Add (CstInt 1) (CstInt 2)) (Add (Add (Add (CstInt 3) (CstInt 4)) (CstInt 5)) (CstInt 6))`
- Evaluation (pure): `ValInt 3`
- Evaluation (simulated): `ValInt 3`

Concurrent `put/get`

The evaluation results for the following examples apply only* for the simulated and concurrent interpretation functions.*

- Syntax: `get 0 && put 0 true`
- AST: `BothOf (KvGet (CstInt 0)) (KvPut (CstInt 0) (CstBool True))`
- Evaluation: `ValTuple [ValBool True,ValBool True]`

- Syntax: `get 0 + 1 && put 0 2`
- AST: `BothOf (Add (KvGet (CstInt 0)) (CstInt 1)) (KvPut (CstInt 0) (CstInt 2))`
- Evaluation: `ValTuple [ValInt 3,ValInt 2]`

- Syntax: `put (get 0) 1 && let x = put 0 2 in get 2`
- AST: `BothOf (KvPut (KvGet (CstInt 0)) (CstInt 1)) (Let "x" (KvPut (CstInt 0) (CstInt 2)) (KvGet (CstInt 2)))`
- Evaluation: `ValTuple [ValInt 1,ValInt 1]`

Tasks

Task A: Implement tuples

For this task you must implement *tuples* in APL. A tuple is a value that comprises zero or more values. They are defined as follows in `APL.Monad`:

```
data Val
  = ...
  | ValTuple [Val]
```

Tuples are constructed by the rule

```
"(" ")"
```

corresponding to a tuple with no elements, and by the rule

```
"(" Exp "," Exps ")"
```

corresponding to a tuple with two or more elements. Single-element tuples are thus not syntactically legal (just as in Haskell), although the AST representation can express them. Note that the production `"(" Exp ")"` is a parenthesized expression and not a single-element tuple.

The corresponding AST constructor is defined in `APL.AST` as follows:

```
data Exp
  = ...
  | Tuple [Exp]
```

For example, the input `(a, b)` corresponds to the following `Exp`:

```
Tuple [Var "a", Var "b"]
```

A tuple expression `(e1, ..., eN)` is evaluated by evaluating the components from left to right, then constructing a `ValTuple` with the resulting values in the same order as their corresponding expressions. If the evaluation of any expression fails, the result of the evaluation of the tuple also fails and the first error encountered should be reported.

The elements of a tuple `x` can be projected (accessed) with the syntax `x.i`, where `i` is a literal integer, such as in `x.0`, which projects element 0 of the tuple. For example, `(e1, ..., eN) == e1`. If `x` has N elements, then `i` must

be between *0* and *N - 1*; using an index outside of this range is an error. The corresponding AST constructor is `Project`. It is an error to try to project an element from a non-tuple.

Your task: Implement parsing of tuple construction and projection in `APL.Parser` and evaluation of tuples and projections in `APL.Eval`.

Solution Open this to see the implementation

Tuples are parsed in `APL.Parser.pAtom0`, with some care (using `try`) to disambiguate them from parenthesized expressions. That is the only subtle part of this task.

Their evaluation is straightforward:

```
eval (Tuple es) =
  ValTuple <$> mapM eval es
```

Since `mapM` traverses the list left-to-right, this ensures the component expressions are evaluated in the desired order.

Task B: Implement `while` loops and stepping

For this task you must extend APL with a notion of `while` loops, as well as a "step" effect that must be emitted for every iteration of *any* loop (including `for` loops)

A `while`-loop expression

```
loop p = init while cond do body
```

is represented with the `WhileLoop` AST constructor and is evaluated as follows:

1. Evaluate expression `init` to a value v.
2. Bind variable `p` to v.
3. Evaluate expression `cond` to a boolean c. It is an error if c does not evaluate to a boolean. If c is false, the loop stops and returns the value of `p`. Otherwise, if c is true, evaluate expression `body` and bind `p` to the result.

Note that in a `while`-loop `p` is in scope when evaluating `cond`.

Stepping is an effect represented by the `StepOp` constructor of `EvalOp`. In later tasks we will use it to interrupt computations that do not otherwise have any effects. For `APL.InterpPure`, interpretation of `StepOp` is simply by recursively executing its continuation. For now, you must modify `APL.Eval` to use the `StepOp` effect (via `evalStep`) in the following cases: just before a loop body is executed (for both `for`- and `while`-loops), and just before a function value is applied.

Your task: Implement parsing of `while`-loops in `APL.Parser` and evaluation in `APL.Eval`. Treat `while` as keywords. Add uses of `evalStep` as stated above, including in the existing case for `ForLoop`. Extend `APL.InterpPure` to handle the `StepOp` effect.

Task C: Implement `&&` and `||`

For this task you must implement operators intended for concurrent programming.

The expression `e1 && e2` denotes concurrent execution of two expressions, returning the result of both. To evaluate the expression `e1 && e2` we evaluate `e1` and `e2` in unspecified order, then return a pair of their results. If either subexpression fails, then the overall expression also fails. If both fail, either error may be reported. Using the pure interpreter, `e1 && e2` is similar to `(e1,e2)` (except for evaluation order), but it will behave differently when using the concurrent interpreters.

The expression `e1 || e2` denotes concurrent execution of two expressions, returning the result of the one that finishes "first" (the meaning of which depends on the interpreter). To evaluate the expression `e1 || e2` we evaluate either `e1` or `e2` and return one of their results. The pure interpreter must start by evaluating `e1`, returning its value if evaluation succeeds. If `e1` fails, the pure interpreter should return the result of evaluating `e2`.

Your task: Transform the grammar of APL to eliminate left recursion and ambiguity. Then implement parsing of `&&` and `||` in `APL.Parser` and evaluation in `APL.Eval`. Evaluation of `&&` is with the `BothOfOp` effect (`evalBothOf`), and evaluation of `||` is with `OneOfOp` (`evalOneOf`).

Extend `APL.InterpPure` to handle the `BothOfOp` and `OneOfOp` effects. For `OneOfOp`, execute the first computation and return its value. If execution of the first computation results in an error, execute the second computation and return its value. If the second computation also results in an error, then the overall computation fails with either the first or second error message.

Task D: Implement a simulated concurrent interpreter

For this task you must implement an interpretation function for `EvalM` that simulates concurrent execution. The main feature will be that `e1 || e2` can be implemented such that the expression that finishes *first* (i.e., in the shortest number of steps) has its value returned. As a special case, it means that `e1 || e2` can terminate even if one of `e1` or `e2` is an infinite loop (but not both).

The key idea is writing a function `step` that evaluates a `EvalM` computation up to (and including) the next `StepOp` effect if possible *but no further*. By repeatedly "stepping", we can progress a computation arbitrarily. But importantly, we can interleave steppings of different concurrent computations.

Further, we also refine how the `KvGetOp`/`KvPutOp` effects work in a concurrent setting. When a `KvGetOp` effect requests the value of a key that does not exist in the state, then that is not an error. Rather, execution is just stuck until some other conconcurrent computation (if any) performs a `KvPutOp` with the desired key. It does, however, remain an error in the pure interpreter.

Your task: Finish the implementation of the simulated concurrent interpreter in `APL.InterpSim`.

Hints: Consider postponing the treatment of `KvGetOp`/`KvPutOp` until the basics work.

Task E: Implement a key-value database

For this task you must implement a simple server for managing a concurrent key-value database *KVDB*. Your implementation must be in the `KVDB` module and implement the following API:

```
-- | A reference to a KVDB instance that stores keys
-- of type 'k' and corresponding values of type 'v'.
data KVDB k v

-- | Start a new KVDB instance.
startKVDB :: (Ord k) => IO (KVDB k v)

-- | Retrieve the value corresponding to a given key.
-- If that key does not exist in the store, then this
-- function blocks until another thread writes the desired
-- key with 'kvPut', after which this function returns
-- the now available value.
kvGet :: KVDB k v -> k -> IO v

-- | Write a key-value mapping to the database.
-- Replaces any prior mapping of the key.
kvPut :: KVDB k v -> k -> v -> IO ()
```

Note that the API is polymorphic, as KVDB can store keys of any type that is an instance of `Ord`, and values of any type.

Your task: Implement the KVDB interface. You must make use of the `GenServer` module. It is up to you to design the state and protocol used to implement KVDB. You must also define appropriate tests in `KVDB_Tests`.

Task F: Implement a concurrent interpreter

In this task you will implement a truly concurrent interpreter that potentially uses multiple Haskell threads to execute the `EvalM` monad. In particular, execution of the `BothOfOp` and `OneOfOp` involve actual IO-level concurrency. You must implement this concurrency by enqueuing jobs in `SPC`, *not* by using `forkIO` directly.

You must implement the same semantics for `KvGetOp`/`KvPutOp` as in Task D, meaning that trying to retrieve a value for a key that does not exist causes the thread to block until the key is available. Interpretation of `StepOp` is simply by recursively executing its continuation.

Your task: Finish the implementation of the concurrent interpreter in `APL.InterpConcurrent`. For handling `BothOfOp` and `OneOfOp`, you must make use of `SPC`. For handling `KvPutOp` and `KVGetOp` you must make use of KVDB.

Hints: Since SPC jobs are executed purely for side effects and have no return value as such, you will need to use `IORef`s to actually store the results of executing tasks. As in Task D, consider postponing the treatment of `KvGetOp`/`KvPutOp` until the basics work.

Your interpreter may crash with an error message "thread blocked indefinitely in an MVar operation", signalled by the Haskell runtime system. This is not necessarily an error in your implementation, but can be due to the APL program you are testing being invalid.

Week 7 solutions

APL/AST.hs

```
module APL.AST
  ( VName,
    Exp (..),
  )
where

type VName = String

data Exp
  = CstInt Integer
  | CstBool Bool
  | Add Exp Exp
  | Sub Exp Exp
  | Mul Exp Exp
  | Div Exp Exp
  | Eql Exp Exp
  | If Exp Exp Exp
  | Var VName
  | Let VName Exp Exp
  | Lambda VName Exp
  | Apply Exp Exp
  | -- | Although the AST can represent 1-element tuples, that is
    -- still not allowed according to the surface grammar.
    Tuple [Exp]
  | Project Exp Integer
  | ForLoop (VName, Exp) (VName, Exp) Exp
  | WhileLoop (VName, Exp) Exp Exp
  | BothOf Exp Exp
  | OneOf Exp Exp
  | KvPut Exp Exp
  | KvGet Exp
  deriving (Eq, Ord, Show)
```

APL/Eval.hs

```
module APL.Eval
  ( eval,
  )
where

import APL.AST (Exp (..))
import APL.Monad
import Data.List (genericDrop)

evalIntBinOp :: (Integer -> Integer -> EvalM Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp f e1 e2 = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValInt x, ValInt y) -> ValInt <$> f x y
    (_, _) -> failure "Non-integer operand"

evalIntBinOp' :: (Integer -> Integer -> Integer) -> Exp -> Exp -> EvalM Val
evalIntBinOp' f e1 e2 =
  evalIntBinOp f' e1 e2
  where
    f' x y = pure $ f x y

eval :: Exp -> EvalM Val
eval (CstInt x) = pure $ ValInt x
eval (CstBool b) = pure $ ValBool b
eval (Var v) = do
  env <- askEnv
  case envLookup v env of
    Just x -> pure x
    Nothing -> failure $ "Unknown variable: " ++ v
eval (Add e1 e2) = evalIntBinOp' (+) e1 e2
eval (Sub e1 e2) = evalIntBinOp' (-) e1 e2
eval (Mul e1 e2) = evalIntBinOp' (*) e1 e2
eval (Div e1 e2) = evalIntBinOp checkedDiv e1 e2
  where
    checkedDiv _ 0 = failure "Division by zero"
    checkedDiv x y = pure $ x `div` y
eval (Eql e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValInt x, ValInt y) -> pure $ ValBool $ x == y
    (ValBool x, ValBool y) -> pure $ ValBool $ x == y
    (_, _) -> failure "Invalid operands to equality"
eval (If cond e1 e2) = do
  cond' <- eval cond
  case cond' of
    ValBool True -> eval e1
    ValBool False -> eval e2
    _ -> failure "if: non-boolean conditional"
eval (Let var e1 e2) = do
  v1 <- eval e1
  localEnv (envExtend var v1) $ eval e2
eval (Lambda var body) = do
  env <- askEnv
  pure $ ValFun env var body
eval (Apply e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  case (v1, v2) of
    (ValFun f_env var body, arg) ->
      localEnv (const $ envExtend var arg f_env) $ evalStep $ eval body
    (_, _) ->
      failure "Cannot apply non-function"
eval (KvPut key_e val_e) = do
```

```
  key <- eval key_e
  val <- eval val_e
  evalKvPut key val
  pure val
eval (KvGet key_e) = do
  key <- eval key_e
  evalKvGet key
eval (ForLoop (loopparam, initial) (iv, bound) body) = do
  initial_v <- eval initial
  bound_v <- eval bound
  case bound_v of
    ValInt bound_int ->
      loop 0 bound_int initial_v
    _ ->
      failure "for-loop: non-integer bound"
  where
    loop i bound_int loop_v
      | i >= bound_int = pure loop_v
      | otherwise = do
          loop_v' <-
            evalStep $
              localEnv (envExtend iv (ValInt i) . envExtend loopparam loop_v) $
                eval body
          loop (succ i) bound_int loop_v'
eval (Tuple es) =
  ValTuple <$> mapM eval es
eval (Project e i) = do
  v <- eval e
  case v of
    ValTuple vs ->
      case genericDrop i vs of
        [] -> failure "project: invalid index"
        v' : _ -> pure v'
    _ -> failure "project: non-pair"
eval (WhileLoop (loopparam, initial) cond body) = do
  initial_v <- eval initial
  loop initial_v
  where
    loop loop_v = do
      cond_v <- localEnv (envExtend loopparam loop_v) $ eval cond
      case cond_v of
        ValBool False -> pure loop_v
        ValBool True -> do
          loop_v' <- evalStep $ localEnv (envExtend loopparam loop_v) $ eval body
          loop loop_v'
        _ -> failure "while-loop: non-boolean condition"
eval (BothOf e1 e2) =
  evalBothOf (eval e1) (eval e2)
eval (OneOf e1 e2) =
  evalOneOf (eval e1) (eval e2)
```

APL/InterpConcurrent.hs

```
module APL.InterpConcurrent (runEval) where

import APL.Monad
import Data.IORef
import KVDB
import SPC

runEval :: EvalM a -> IO (Either Error a)
runEval computation = do
  kvdb <- startKVDB
  spc <- startSPC
  let runEval' :: Env -> EvalM a -> IO (Either Error a)
      runEval' _ (Pure x) = pure (Right x)
      runEval' r (Free (ReadOp k)) = runEval' r $ k r
      runEval' r (Free (KvGetOp key k)) = do
        val <- kvGet kvdb key
        runEval' r $ k val
      runEval' r (Free (KvPutOp key val m)) = do
        kvPut kvdb key val
        runEval' r m
      runEval' _ (Free (ErrorOp e)) = pure (Left e)
      runEval' r (Free (StepOp c)) = runEval' r c
      runEval' r (Free (BothOfOp x y c)) = do
        x_ref <- newIORef Nothing
        x_job <- jobAdd spc $ Job $ do
          writeIORef x_ref . Just =<< runEval' r x
        y_ref <- newIORef Nothing
        y_job <- jobAdd spc $ Job $ do
          writeIORef y_ref . Just =<< runEval' r y
        _ <- jobWaitAny spc [x_job]
        _ <- jobWaitAny spc [y_job]
        x' <- readIORef x_ref
        y' <- readIORef y_ref
        case (x', y') of
          (Just (Right x''), Just (Right y'')) ->
            runEval' r $ c $ ValTuple [x'', y'']
          (Just (Left err), _) ->
            pure (Left err)
          (_, Just (Left err)) ->
            pure (Left err)
          _ ->
            error "Unexpected failure"
      runEval' r (Free (OneOfOp x y c)) = do
        x_ref <- newIORef Nothing
        x_job <- jobAdd spc $ Job $ do
          writeIORef x_ref . Just =<< runEval' r x
        y_ref <- newIORef Nothing
        y_job <- jobAdd spc $ Job $ do
          writeIORef y_ref . Just =<< runEval' r y
        (which, Done) <- jobWaitAny spc [x_job, y_job]
        if which == x_job
          then do
            Just x' <- readIORef x_ref
            case x' of
              Right x'' -> do
                jobCancel spc y_job
                runEval' r $ c x''
              Left _ -> do
                _ <- jobWaitAny spc [y_job]
                Just y' <- readIORef y_ref
                case y' of
                  Right y'' -> runEval' r $ c y''
                  Left err -> pure (Left err)
          else do
            Just y' <- readIORef y_ref
            case y' of
```

```
              Right y'' -> do
                jobCancel spc x_job
                runEval' r $ c y''
              Left _ -> do
                _ <- jobWaitAny spc [x_job]
                Just x' <- readIORef x_ref
                case x' of
                  Right x'' -> runEval' r $ c x''
                  Left err -> pure (Left err)
  runEval' envEmpty computation
```

APL/InterpPure.hs

```
module APL.InterpPure (runEval) where

import APL.Monad

type State = [(Val, Val)]

stateInitial :: State
stateInitial = []

runEval :: EvalM a -> Either Error a
runEval = fst . runEval' envEmpty stateInitial
  where
    runEval' :: Env -> State -> EvalM a -> (Either Error a, State)
    runEval' _ s (Pure x) = (pure x, s)
    runEval' r s (Free (ReadOp k)) = runEval' r s $ k r
    runEval' r s (Free (KvGetOp key k)) =
      case lookup key s of
        Nothing -> (Left $ "Invalid key: " ++ show key, s)
        Just val -> runEval' r s $ k val
    runEval' r s (Free (KvPutOp key val m)) =
      let s' = (key, val) : filter ((/= key) . fst) s
       in runEval' r s' m
    runEval' _ s (Free (ErrorOp e)) = (Left e, s)
    runEval' r s (Free (StepOp c)) = runEval' r s c
    runEval' r s (Free (BothOfOp x y c)) =
      case runEval' r s x of
        (Left err, s') -> (Left err, s')
        (Right x', s') ->
          case runEval' r s' y of
            (Left err, s'') -> (Left err, s'')
            (Right y', s'') ->
              runEval' r s'' $ c $ ValTuple [x', y']
    runEval' r s (Free (OneOfOp x y c)) =
      case runEval' r s x of
        (Left _, s') ->
          case runEval' r s' y of
            (Left err, s'') -> (Left err, s'')
            (Right y', s'') ->
              runEval' r s'' $ c y'
        (Right x', s') ->
          runEval' r s' $ c x'
```

APL/InterpSim.hs

```
module APL.InterpSim (runEval) where

import APL.Monad

type State = [(Val, Val)]

stateInitial :: State
stateInitial = []

step :: Env -> State -> EvalM a -> (EvalM a, State)
step _ s (Pure x) = (Pure x, s)
step r s (Free (ReadOp c)) = step r s $ c r
step r s (Free (KvGetOp key k)) =
  case lookup key s of
    Nothing -> (Free (KvGetOp key k), s)
    Just val -> step r s $ k val
step r s (Free (KvPutOp key val m)) =
  let s' = (key, val) : filter ((/= key) . fst) s
   in step r s' m
step _ s (Free (ErrorOp e)) = (Free (ErrorOp e), s)
step _ s (Free (StepOp c)) = (c, s)
step _ s (Free (BothOfOp (Free (ErrorOp e)) _ _)) =
  (Free (ErrorOp e), s)
step _ s (Free (BothOfOp _ (Free (ErrorOp e)) _)) =
  (Free (ErrorOp e), s)
step r s (Free (BothOfOp (Pure x) (Pure y) c)) =
  step r s $ c $ ValTuple [x, y]
step r s (Free (BothOfOp x y c)) =
  let (x', s') = step r s x
      (y', s'') = step r s' y
   in (Free (BothOfOp x' y' c), s'')
step _ s (Free (OneOfOp (Free (ErrorOp e)) (Free (ErrorOp _)) _)) =
  (Free (ErrorOp e), s)
step r s (Free (OneOfOp (Pure x) _ c)) =
  step r s $ c x
step r s (Free (OneOfOp _ (Pure y) c)) =
  step r s $ c y
step r s (Free (OneOfOp x y c)) =
  let (x', s') = step r s x
      (y', s'') = step r s' y
   in (Free (OneOfOp x' y' c), s'')

runEval :: EvalM a -> Either Error a
runEval initial = fst $ runEval' envEmpty stateInitial initial
  where
    runEval' :: Env -> State -> EvalM a -> (Either Error a, State)
    runEval' r s m =
      case step r s m of
        (Pure x, s') -> (Right x, s')
        (Free (ErrorOp e), s') -> (Left e, s')
        (m', s') -> runEval' r s' m'
```

APL/InterpStep.hs

```
-- Like the pure interpreter, but returns the number of step effects.
-- Only counts the successful execution for ||.
module APL.InterpStep (runEval) where

import APL.Monad

type State = [(Val, Val)]

stateInitial :: State
stateInitial = []

runEval :: EvalM a -> Int
runEval = fst . fst . runEval' envEmpty stateInitial
  where
    runEval' :: Env -> State -> EvalM a -> ((Int, Either Error a), State)
    runEval' _ s (Pure x) = ((0, pure x), s)
    runEval' r s (Free (ReadOp k)) = runEval' r s $ k r
    runEval' r s (Free (KvGetOp key k)) =
      case lookup key s of
        Nothing -> ((0, Left $ "Invalid key: " ++ show key), s)
        Just val -> runEval' r s $ k val
    runEval' r s (Free (KvPutOp key val m)) =
      let s' = (key, val) : filter ((/= key) . fst) s
       in runEval' r s' m
    runEval' _ s (Free (ErrorOp e)) = ((0, Left e), s)
    runEval' r s (Free (StepOp c)) =
      let ((n, x), s') = runEval' r s c
       in ((n + 1, x), s')
    runEval' r s (Free (BothOfOp x y c)) =
      case runEval' r s x of
        ((n, Left err), s') -> ((n, Left err), s')
        ((n, Right x'), s') ->
          case runEval' r s' y of
            ((m, Left err), s'') -> ((n + m, Left err), s'')
            ((m, Right y'), s'') ->
              let ((k, c'), s''') = runEval' r s'' $ c $ ValTuple [x', y']
               in ((n + m + k, c'), s''')
    runEval' r s (Free (OneOfOp x y c)) =
      case runEval' r s x of
        ((n, Left err), s') ->
          case runEval' r s' y of
            ((_, Left _), s'') -> ((n, Left err), s'')
            ((m, Right y'), s'') ->
              let ((k, c'), s''') = runEval' r s'' $ c y'
               in ((m + k, c'), s''')
        ((n, Right x'), s') ->
          let ((m, c'), s'') = runEval' r s' $ c x'
           in ((n + m, c'), s'')
```

APL/Monad.hs

```
module APL.Monad
  ( envEmpty,
    envExtend,
    envLookup,
    askEnv,
    localEnv,
    failure,
    evalStep,
    evalKvGet,
    evalKvPut,
    evalBothOf,
    evalOneOf,
    EvalM,
    Val (..),
    EvalOp (..),
    Free (..),
    Error,
    Env,
  )
where

import APL.AST (Exp (..), VName)
import Control.Monad (ap)

data Val
  = ValInt Integer
  | ValBool Bool
  | ValFun Env VName Exp
  | -- | Although this value representation allows 1-element tuples,
    -- that is still not allowed according to the surface grammar.
    ValTuple [Val]
  deriving (Eq, Ord, Show)

type Error = String

type Env = [(VName, Val)]

envEmpty :: Env
envEmpty = []

envExtend :: VName -> Val -> Env -> Env
envExtend v val env = (v, val) : env

envLookup :: VName -> Env -> Maybe Val
envLookup v env = lookup v env

data Free e a
  = Pure a
  | Free (e (Free e a))

instance (Functor e) => Functor (Free e) where
  fmap f (Pure x) = Pure $ f x
  fmap f (Free g) = Free $ fmap (fmap f) g

instance (Functor e) => Applicative (Free e) where
  pure = Pure
  (<*>) = ap

instance (Functor e) => Monad (Free e) where
  Pure x >>= f = f x
  Free g >>= f = Free $ h <$> g
    where
      h x = x >>= f

data EvalOp a
  = ReadOp (Env -> a)
```

```
  | ErrorOp Error
  | KvGetOp Val (Val -> a)
  | KvPutOp Val Val a
  | -- | A vacuous effect that has no influence on APL as such, but
    -- allows an interpretation function to take back control.
    StepOp a
  | -- | This effect corresponds to the '&&' operator. The
    -- continuation must be invoked with a pair ('ValTuple') of the
    -- results of the two 'EvalM Val' computations.
    BothOfOp (EvalM Val) (EvalM Val) (Val -> a)
  | -- | This effect corresponds to the '||' operator. The
    -- continuation must be invoked with the result of either of the
    -- 'EvalM Val' computations.
    OneOfOp (EvalM Val) (EvalM Val) (Val -> a)

instance Functor EvalOp where
  fmap f (ReadOp c) = ReadOp $ f . c
  fmap f (KvGetOp key k) = KvGetOp key $ f . k
  fmap f (KvPutOp key val m) = KvPutOp key val $ f m
  fmap _ (ErrorOp e) = ErrorOp e
  fmap f (StepOp c) = StepOp $ f c
  fmap f (BothOfOp x y c) = BothOfOp x y $ f . c
  fmap f (OneOfOp x y c) = OneOfOp x y $ f . c

type EvalM a = Free EvalOp a

askEnv :: EvalM Env
askEnv = Free $ ReadOp $ \env -> pure env

localEnv :: (Env -> Env) -> EvalM a -> EvalM a
localEnv _ (Pure x) = Pure x
localEnv _ (Free (ErrorOp e)) = Free (ErrorOp e)
localEnv f (Free (ReadOp c)) = Free $ ReadOp $ localEnv f . c . f
localEnv f (Free (KvGetOp key c)) = Free $ KvGetOp key $ localEnv f . c
localEnv f (Free (KvPutOp key val c)) = Free $ KvPutOp key val $ localEnv f c
localEnv f (Free (StepOp c)) = Free $ StepOp $ localEnv f c
localEnv f (Free (BothOfOp x y c)) =
  Free $ BothOfOp (localEnv f x) (localEnv f y) $ localEnv f . c
localEnv f (Free (OneOfOp x y c)) =
  Free $ OneOfOp (localEnv f x) (localEnv f y) $ localEnv f . c

failure :: String -> EvalM a
failure = Free . ErrorOp

-- | Perform a StepOp effect, then perform the provided computation.
evalStep :: EvalM a -> EvalM a
evalStep m = Free $ StepOp m

evalKvGet :: Val -> EvalM Val
evalKvGet key = Free $ KvGetOp key $ \val -> pure val

evalKvPut :: Val -> Val -> EvalM ()
evalKvPut key val = Free $ KvPutOp key val $ pure ()

-- | Produce a BothOfOp effect.
evalBothOf :: EvalM Val -> EvalM Val -> EvalM Val
evalBothOf a b = Free $ BothOfOp a b pure

-- | Produce a OneOfOp effect.
evalOneOf :: EvalM Val -> EvalM Val -> EvalM Val
evalOneOf a b = Free $ OneOfOp a b pure
```

APL/Parser.hs

```
module APL.Parser (parseAPL) where

import APL.AST (Exp (..), VName)
import Control.Monad (void)
import Data.Char (isAlpha, isAlphaNum, isDigit)
import Data.Void (Void)
import Text.Megaparsec
  ( Parsec,
    choice,
    chunk,
    eof,
    errorBundlePretty,
    many,
    notFollowedBy,
    parse,
    satisfy,
    some,
    try,
  )
import Text.Megaparsec.Char (space)

type Parser = Parsec Void String

lexeme :: Parser a -> Parser a
lexeme p = p <* space

keywords :: [String]
keywords =
  [ "if",
    "then",
    "else",
    "true",
    "false",
    "let",
    "in",
    "put",
    "get",
    "loop",
    "for",
    "while",
    "do"
  ]

lVName :: Parser VName
lVName = lexeme $ try $ do
  c <- satisfy isAlpha
  cs <- many $ satisfy isAlphaNum
  let v = c : cs
  if v `elem` keywords
    then fail "Unexpected keyword"
    else pure v

lInteger :: Parser Integer
lInteger =
  lexeme $ read <$> some (satisfy isDigit) <* notFollowedBy (satisfy isAlphaNum)

lString :: String -> Parser ()
lString s = lexeme $ void $ chunk s

lKeyword :: String -> Parser ()
lKeyword s = lexeme $ void $ try $ chunk s <* notFollowedBy (satisfy isAlphaNum)

pBool :: Parser Bool
pBool =
  choice
    [ const True <$> lKeyword "true",
```

```
67        const False <$> lKeyword "false"
68      ]
69
70  pAtom0 :: Parser Exp
71  pAtom0 =
72    choice
73      [ CstInt <$> lInteger,
74        CstBool <$> pBool,
75        Var <$> lVName,
76        try $ Tuple <$> (lString "(" *> lString ")" *> pure []),
77        try $
78          Tuple
79            <$> ( lString "("
80                    *> ((:) <$> pExp <*> some (lString "," *> pExp))
81                    <* lString ")"
82                ),
83        lString "(" *> pExp <* lString ")",
84        KvPut <$> (lKeyword "put" *> pAtom) <*> pAtom,
85        KvGet <$> (lKeyword "get" *> pAtom)
86      ]
87
88  pAtom1 :: Parser Exp
89  pAtom1 = pAtom0 >>= chain
90    where
91      chain x =
92        choice
93          [ do
94              lString "."
95              i <- lInteger
96              chain $ Project x i,
97            pure x
98          ]
99
100 pAtom :: Parser Exp
101 pAtom = pAtom1
102
103 pFExp :: Parser Exp
104 pFExp = chain =<< pAtom
105   where
106     chain x =
107       choice
108         [ do
109             y <- pAtom
110             chain $ Apply x y,
111           pure x
112         ]
113
114 pLExp :: Parser Exp
115 pLExp =
116   choice
117     [ If
118         <$> (lKeyword "if" *> pExp)
119         <*> (lKeyword "then" *> pExp)
120         <*> (lKeyword "else" *> pExp),
121       Lambda
122         <$> (lString "\\" *> lVName)
123         <*> (lString "->" *> pExp),
124       Let
125         <$> (lKeyword "let" *> lVName)
126         <*> (lString "=" *> pExp)
127         <*> (lKeyword "in" *> pExp),
128       try $
129         ForLoop
130           <$> (lKeyword "loop" *> ((,) <$> lVName <* lString "=" <*> pAtom))
131           <*> (lKeyword "for" *> ((,) <$> lVName <* lString "<" <*> pExp))
132           <*> (lKeyword "do" *> pExp),
133       try $
134         WhileLoop
```

```
          <$> (lKeyword "loop" *> ((,) <$> lVName <* lString "=" <*> pAtom))
          <*> (lKeyword "while" *> pExp)
          <*> (lKeyword "do" *> pExp),
      pFExp
    ]

pExp4 :: Parser Exp
pExp4 = pLExp >>= chain
  where
    chain x =
      choice
        [ do
            lString "*"
            y <- pLExp
            chain $ Mul x y,
          do
            lString "/"
            y <- pLExp
            chain $ Div x y,
          pure x
        ]

pExp3 :: Parser Exp
pExp3 = pExp4 >>= chain
  where
    chain x =
      choice
        [ do
            lString "+"
            y <- pExp4
            chain $ Add x y,
          do
            lString "-"
            y <- pExp4
            chain $ Sub x y,
          pure x
        ]

pExp2 :: Parser Exp
pExp2 = pExp3 >>= chain
  where
    chain x =
      choice
        [ do
            lString "=="
            y <- pExp3
            chain $ Eql x y,
          pure x
        ]

pExp1 :: Parser Exp
pExp1 = pExp2 >>= chain
  where
    chain x =
      choice
        [ do
            lString "&&"
            y <- pExp2
            chain $ BothOf x y,
          pure x
        ]

pExp0 :: Parser Exp
pExp0 = pExp1 >>= chain
  where
    chain x =
      choice
        [ do
```

```
            lString "||"
            y <- pExp1
            chain $ OneOf x y,
          pure x
        ]

pExp :: Parser Exp
pExp = pExp0

parseAPL :: FilePath -> String -> Either String Exp
parseAPL fname s = case parse (space *> pExp <* eof) fname s of
  Left err -> Left $ errorBundlePretty err
  Right x -> Right x
```

GenServer.hs

```
module GenServer
  ( Chan,
    Server,
    receive,
    send,
    sendTo,
    spawn,
    ReplyChan,
    requestReply,
    reply,
    actionWithTimeout,
    actionWithTimeoutKill,
  )
where

-- ANCHOR: Setup
import Control.Concurrent
  ( Chan,
    ThreadId,
    forkIO,
    killThread,
    newChan,
    readChan,
    threadDelay,
    writeChan,
  )
-- ANCHOR_END: Setup

-- ANCHOR: Server
type Server msg = (ThreadId, Chan msg)
-- ANCHOR_END: Server

-- ANCHOR: SendReceive
send :: Chan a -> a -> IO ()
send chan msg =
  writeChan chan msg

sendTo :: Server a -> a -> IO ()
sendTo (_tid, input) msg =
  send input msg

receive :: Chan a -> IO a
receive = readChan
-- ANCHOR_END: SendReceive

-- ANCHOR: Spawn
spawn :: (Chan a -> IO ()) -> IO (Server a)
spawn serverLoop = do
  input <- newChan
  tid <- forkIO $ serverLoop input
  return (tid, input)
-- ANCHOR_END: Spawn

-- ANCHOR: RequestReply
newtype ReplyChan a = ReplyChan (Chan a)

requestReply :: Server a -> (ReplyChan b -> a) -> IO b
requestReply serv con = do
  reply_chan <- ReplyChan <$> newChan
  sendTo serv $ con reply_chan
  receiveReply reply_chan

reply :: ReplyChan a -> a -> IO ()
reply (ReplyChan chan) x = send chan x

receiveReply :: ReplyChan a -> IO a
```

```haskell
receiveReply (ReplyChan chan) = receive chan
-- ANCHOR_END: RequestReply

data Timeout = Timeout

-- ANCHOR: ActionWithTimeout
actionWithTimeout :: Int -> IO a -> IO (Either Timeout a)
actionWithTimeout seconds action = do
  chan <- ReplyChan <$> newChan
  _ <- forkIO $ do
    -- worker thread
    x <- action
    reply chan $ Right x
  _ <- forkIO $ do
    -- timeout thread
    threadDelay (seconds * 1000000)
    reply chan $ Left Timeout
  receiveReply chan
-- ANCHOR_END: ActionWithTimeout

-- ANCHOR: ActionWithTimeoutKill
actionWithTimeoutKill :: Int -> IO a -> IO (Either Timeout a)
actionWithTimeoutKill seconds action = do
  chan <- ReplyChan <$> newChan
  worker_tid <- forkIO $ do
    -- worker thread
    x <- action
    reply chan $ Right x
  _ <- forkIO $ do
    -- timeout thread
    threadDelay (seconds * 1000000)
    killThread worker_tid
    reply chan $ Left Timeout
  receiveReply chan
-- ANCHOR_END: ActionWithTimeoutKill
```

KVDB.hs

```haskell
-- | Key-value database.
module KVDB
  ( KVDB,
    startKVDB,
    kvGet,
    kvPut,
  )
where

import Control.Monad (forM_)
import Data.List (partition)
import GenServer

data Msg k v
  = MsgPut k v
  | MsgGet k (ReplyChan v)

newtype KVDB k v = KVDB (Server (Msg k v))

data State k v = State
  { stateKVs :: [(k, v)],
    stateWaiters :: [(k, ReplyChan v)]
  }

kvdbEvent :: (Ord k) => Msg k v -> State k v -> IO (State k v)
kvdbEvent (MsgPut k v) state = do
  let (waiting, not_waiting) =
        partition ((== k) . fst) $ stateWaiters state
  forM_ waiting $ \(_, from) -> reply from v
  pure
    state
      { stateKVs = (k, v) : filter ((/= k) . fst) (stateKVs state),
        stateWaiters = not_waiting
      }
kvdbEvent (MsgGet k from) state =
  case lookup k $ stateKVs state of
    Nothing ->
      pure state {stateWaiters = (k, from) : stateWaiters state}
    Just v -> do
      reply from v
      pure state

startKVDB :: (Ord k) => IO (KVDB k v)
startKVDB =
  KVDB <$> spawn (loop (State [] []))
  where
    loop state c = do
      msg <- receive c
      state' <- kvdbEvent msg state
      loop state' c

kvGet :: KVDB k v -> k -> IO v
kvGet (KVDB s) k = requestReply s $ MsgGet k

kvPut :: KVDB k v -> k -> v -> IO ()
kvPut (KVDB s) k v = sendTo s $ MsgPut k v
```

SPC.hs

```
-- | Simplified variant of SPC.
--
-- Always starts a new worker thread when a job is added (so nothing
-- is ever "pending"), removes support for timeouts, and adds a
-- function 'jobWaitAny' that allows us to wait for several jobs
-- concurrently.
module SPC
  ( -- * SPC startup
    SPC,
    startSPC,

    -- * Job functions
    Job (..),
    JobId,
    JobStatus (..),
    JobDoneReason (..),
    jobAdd,
    jobStatus,
    jobWaitAny,
    jobCancel,
  )
where

import Control.Concurrent
  ( ThreadId,
    forkIO,
    killThread,
  )
import Control.Exception (SomeException, catch)
import Control.Monad (ap, forM_, forever, liftM)
import Data.List (partition)
import GenServer

-- First some general utility functions.

-- Then the definition of the glorious SPC.

-- | A job that is to be enqueued in the glorious SPC.
newtype Job = Job
  { -- | The IO action that comprises the actual action of the job.
    jobAction :: IO ()
  }

-- | A unique identifier of a job that has been enqueued.
newtype JobId = JobId Int
  deriving (Eq, Ord, Show)

-- | How a job finished.
data JobDoneReason
  = -- | Normal termination.
    Done
  | -- | The job was killed because it ran for too long.
    DoneTimeout
  | -- | The job was explicitly cancelled.
    DoneCancelled
  | -- | The job crashed due to an exception.
    DoneCrashed
  deriving (Eq, Ord, Show)

-- | The status of a job.
data JobStatus
  = -- | The job is done and this is why.
    JobDone JobDoneReason
  | -- | The job is still running.
    JobsRunning
  | -- | A job with this ID is not known to this SPC instance.
```

```
    JobUnknown
  deriving (Eq, Ord, Show)

-- Messages sent to SPC.
data SPCMsg
  = -- | Add the job, and reply with the job ID.
    MsgJobAdd Job (ReplyChan JobId)
  | -- | Cancel the given job.
    MsgJobCancel JobId
  | -- | Immediately reply the status of the job.
    MsgJobStatus JobId (ReplyChan JobStatus)
  | -- | Reply when the job is done.
    MsgJobWait [JobId] (ReplyChan (JobId, JobDoneReason))
  | -- | Job has finished.
    MsgJobDone JobId
  | -- | Job crashed.
    MsgJobCrashed JobId

-- | A Handle to the SPC instance.
newtype SPC = SPC (Server SPCMsg)

-- | The central state. Must be protected from the bourgeoisie.
data SPCState = SPCState
  { spcChan :: Chan SPCMsg,
    spcJobsRunning :: [(JobId, ThreadId)],
    spcJobsDone :: [(JobId, JobDoneReason)],
    -- | These are waiting for this job to terminate.
    spcWaiting :: [([JobId], ReplyChan (JobId, JobDoneReason))],
    spcJobCounter :: Int
  }

-- | The monad in which the main SPC thread runs. This is a state
-- monad with support for IO.
newtype SPCM a = SPCM (SPCState -> IO (a, SPCState))

instance Functor SPCM where
  fmap = liftM

instance Applicative SPCM where
  pure x = SPCM $ \state -> pure (x, state)
  (<*>) = ap

instance Monad SPCM where
  SPCM m >>= f = SPCM $ \state -> do
    (x, state') <- m state
    let SPCM f' = f x
    f' state'

-- | Retrieve the state.
get :: SPCM SPCState
get = SPCM $ \state -> pure (state, state)

-- | Overwrite the state.
put :: SPCState -> SPCM ()
put state = SPCM $ \_ -> pure ((), state)

-- | Lift an 'IO' action into 'SPCM'.
io :: IO a -> SPCM a
io m = SPCM $ \state -> do
  x <- m
  pure (x, state)

-- | Run the SPCM monad.
runSPCM :: SPCState -> SPCM a -> IO a
runSPCM state (SPCM f) = fst <$> f state

jobDone :: JobId -> JobDoneReason -> SPCM ()
jobDone jobid reason = do
```

```
  state <- get
  case lookup jobid $ spcJobsDone state of
    Just _ ->
      -- We already know this job is done.
      pure ()
    Nothing -> do
      let (waiting_for_job, not_waiting_for_job) =
            partition (elem jobid . fst) (spcWaiting state)
      forM_ waiting_for_job $ \(_, rsvp) ->
        io $ reply rsvp (jobid, reason)
      put $
        state
          { spcWaiting = not_waiting_for_job,
            spcJobsDone = (jobid, reason) : spcJobsDone state
          }

addJob :: Job -> SPCM JobId
addJob job = do
  state <- get
  let jobid = JobId $ spcJobCounter state
  tid <- io $ forkIO $ do
    let doJob = do
          jobAction job
          send (spcChan state) $ MsgJobDone jobid
        onException :: SomeException -> IO ()
        onException _ =
          send (spcChan state) $ MsgJobCrashed jobid
    doJob `catch` onException
  put
    state
      { spcJobCounter = succ $ spcJobCounter state,
        spcJobsRunning = (jobid, tid) : spcJobsRunning state
      }
  pure jobid

handleMsg :: Chan SPCMsg -> SPCM ()
handleMsg c = do
  msg <- io $ receive c
  case msg of
    MsgJobAdd job rsvp -> do
      jobid <- addJob job
      io $ reply rsvp jobid
    MsgJobStatus jobid rsvp -> do
      state <- get
      io $ reply rsvp $ case ( lookup jobid $ spcJobsRunning state,
                               lookup jobid $ spcJobsDone state
                             ) of
        (Just _, _) -> JobsRunning
        (_, Just r) -> JobDone r
        _ -> JobUnknown
    MsgJobWait jobids rsvp -> do
      state <- get
      case filter (flip elem jobids . fst) $ spcJobsDone state of
        (jobid, reason) : _ -> do
          io $ reply rsvp (jobid, reason)
        _ ->
          put $ state {spcWaiting = (jobids, rsvp) : spcWaiting state}
    MsgJobDone jobid -> do
      state <- get
      case lookup jobid $ spcJobsRunning state of
        Just _ ->
          jobDone jobid Done
        _ -> pure ()
    MsgJobCancel jobid -> do
      state <- get
      case lookup jobid $ spcJobsRunning state of
        Just tid -> do
          io $ killThread tid
```

```
          jobDone jobid DoneCancelled
        _ -> pure ()
    MsgJobCrashed jobid -> do
      state <- get
      case lookup jobid $ spcJobsRunning state of
        Just tid -> do
          io $ killThread tid
          jobDone jobid DoneCrashed
        _ -> pure ()

startSPC :: IO SPC
startSPC = do
  let initial_state c =
        SPCState
          { spcJobCounter = 0,
            spcJobsRunning = [],
            spcJobsDone = [],
            spcWaiting = [],
            spcChan = c
          }
  server <- spawn $ \c -> runSPCM (initial_state c) $ forever $ handleMsg c
  pure $ SPC server

-- | Add a job for scheduling.
jobAdd :: SPC -> Job -> IO JobId
jobAdd (SPC c) job =
  requestReply c $ MsgJobAdd job

-- | Query the job status.
jobStatus :: SPC -> JobId -> IO JobStatus
jobStatus (SPC c) jobid =
  requestReply c $ MsgJobStatus jobid

-- | Synchronously block until one of the given jobs is done and
-- return the reason.
jobWaitAny :: SPC -> [JobId] -> IO (JobId, JobDoneReason)
jobWaitAny (SPC c) jobids =
  requestReply c $ MsgJobWait jobids

-- | Asynchronously cancel a job.
jobCancel :: SPC -> JobId -> IO ()
jobCancel (SPC c) jobid =
  sendTo c $ MsgJobCancel jobid
```

Tests.hs

```haskell
-- | Omnibus test suite grouped by which task each test applies to.
-- This is supposed to ease grading.
module Tests (tests) where

import APL.AST (Exp (..))
import APL.Eval (eval)
import APL.InterpConcurrent as Concurrent
import APL.InterpPure as Pure
import APL.InterpSim as Sim
import APL.InterpStep as Step
import APL.Monad (Val (..))
import APL.Parser (parseAPL)
import Control.Concurrent (forkIO, newChan, readChan, threadDelay, writeChan)
import KVDB (kvGet, kvPut, startKVDB)
import Test.Tasty (TestTree, testGroup)
import Test.Tasty.HUnit (assertFailure, testCase, (@?=))

infiniteLoop :: Exp
infiniteLoop =
  WhileLoop ("x", CstInt 0) (CstBool True) (Var "x")

finiteLoop :: Exp
finiteLoop =
  ForLoop
    ("x", CstInt 1)
    ("i", CstInt 10)
    (Mul (Var "x") (CstInt 2))

parserTest :: String -> Exp -> TestTree
parserTest s e =
  testCase s $
    case parseAPL "input" s of
      Left err -> assertFailure err
      Right e' -> e' @?= e

parserTestFail :: String -> TestTree
parserTestFail s =
  testCase s $
    case parseAPL "input" s of
      Left _ -> pure ()
      Right e ->
        assertFailure $
          "Expected parse error but received this AST:\n" ++ show e

pureEvalTest :: String -> Exp -> Val -> TestTree
pureEvalTest desc e v =
  testCase desc $
    Pure.runEval (eval e) @?= Right v

pureEvalTestFail :: String -> Exp -> TestTree
pureEvalTestFail desc e =
  testCase desc $
    case Pure.runEval (eval e) of
      Left _ -> pure ()
      Right v ->
        assertFailure $
          "Expected error but received this value:\n" ++ show v

concurrentEvalTest :: String -> Exp -> Val -> TestTree
concurrentEvalTest desc e v = testCase desc $ do
  res <- Concurrent.runEval $ eval e
  res @?= Right v

concurrentEvalTestFail :: String -> Exp -> TestTree
concurrentEvalTestFail desc e = testCase desc $ do
  res <- Concurrent.runEval $ eval e
```

```
  case res of
    Right _ -> assertFailure "Expected failure."
    Left _ -> pure ()

simEvalTest :: String -> Exp -> Val -> TestTree
simEvalTest desc e v = testCase desc $ do
  Sim.runEval (eval e) @?= Right v

simEvalTestFail :: String -> Exp -> TestTree
simEvalTestFail desc e = testCase desc $
  case Sim.runEval (eval e) of
    Right _ -> assertFailure "Expected failure."
    Left _ -> pure ()

taskA :: TestTree
taskA =
  testGroup
    "Task A: tuples"
    [ testGroup
        "Parsing"
        [ testGroup
            "Tuples"
            [ parserTest "(x,y)" $ Tuple [Var "x", Var "y"],
              parserTest "(x,y,z)" $ Tuple [Var "x", Var "y", Var "z"],
              parserTest "()" $ Tuple [],
              parserTestFail "(a,",
              parserTestFail "(a,b,",
              parserTest "(a)" $ Var "a"
            ],
          testGroup
            "Projection"
            [ parserTest "x.0" $ Project (Var "x") 0,
              parserTest "x.0.1" $ Project (Project (Var "x") 0) 1,
              parserTest "(x.0).1" $ Project (Project (Var "x") 0) 1,
              parserTestFail "x.(y.0)",
              parserTest "x .0" $ Project (Var "x") 0,
              parserTest "x. 0" $ Project (Var "x") 0,
              parserTest "x . 0" $ Project (Var "x") 0,
              parserTest "x . 0 . 1" $ Project (Project (Var "x") 0) 1,
              parserTest "((\\x -> x) x).0" $ Project (Apply (Lambda "x" (Var "x")) (Var "x")) 0
            ]
        ],
      testGroup
        "Evaluation"
        [ testGroup
            "Tuples"
            [ evalTest
                "(e1,e2)"
                (Tuple [CstInt 1, CstInt 2])
                (ValTuple [ValInt 1, ValInt 2]),
              evalTest
                "(put 0 1, get 0)"
                (Tuple [KvPut (CstInt 0) (CstInt 1), KvGet (CstInt 0)])
                (ValTuple [ValInt 1, ValInt 1]),
              evalTestFail
                "(get 0, put 0 1)"
                (Tuple [KvGet (CstInt 0), KvPut (CstInt 0) (CstInt 1)])
            ],
          testGroup
            "Projection"
            [ evalTest
                "(e1,e2).0"
                (Project (Tuple [CstInt 1, CstInt 2]) 0)
                (ValInt 1),
              evalTest
                "(e1,e2).1"
                (Project (Tuple [CstInt 1, CstInt 2]) 1)
                (ValInt 2),
```

```
135               evalTestFail
136                 "(e1,e2).2"
137                 (Project (Tuple [CstInt 1, CstInt 2]) 2),
138               evalTestFail
139                 "1.2"
140                 (Project (CstInt 1) 2)
141             ]
142         ]
143     ]
144   where
145     evalTest = pureEvalTest
146     evalTestFail = pureEvalTestFail
147
148 taskB :: TestTree
149 taskB =
150   testGroup
151     "Task B: loops and stepping"
152     [ testGroup
153         "Parsing"
154         [ parserTestFail "loop",
155           parserTestFail "do",
156           parserTestFail "for",
157           parserTestFail "while",
158           parserTest "loop x=y for i<n do x" $
159             ForLoop ("x", Var "y") ("i", Var "n") (Var "x"),
160           parserTest "loop x=y for i < n do x" $
161             ForLoop ("x", Var "y") ("i", Var "n") (Var "x"),
162           parserTestFail "loop x=y for i<n dox",
163           parserTest "loop x=y while x do x" $
164             WhileLoop ("x", Var "y") (Var "x") (Var "x")
165         ],
166       testGroup
167         "Stepping"
168         [ testCase "for-loop" $
169             Step.runEval
170               (eval $ ForLoop ("x", CstInt 0) ("i", CstInt 4) (Var "x"))
171               @?= 4,
172           testCase "for-loop (use iterator)" $
173             Step.runEval
174               (eval $ ForLoop ("x", CstInt 0) ("i", CstInt 4) (Add (Var "x") (Var "i")))
175               @?= 4,
176           testCase "while-loop" $
177             Step.runEval
178               ( eval $
179                   ( WhileLoop
180                       ("x", Tuple [CstInt 1, CstInt 10])
181                       (If (Eql (Project (Var "x") 1) (CstInt 0)) (CstBool False) (CstBool True))
182                       (Tuple [Mul (Project (Var "x") 0) (CstInt 2), Sub (Project (Var "x") 1) (CstInt 1)])
183                   )
184               )
185               @?= 10,
186           testCase "application" $
187             Step.runEval
188               ( eval $
189                   Let
190                     "f"
191                     (Lambda "x" (Var "x"))
192                     ( Add
193                         (Apply (Var "f") (CstInt 2))
194                         (Apply (Var "f") (CstInt 2))
195                     )
196               )
197               @?= 2
198         ],
199       testGroup
200         "Evaluation"
201         [ evalTest
202             "For loop"
```

```
203             (ForLoop ("x", CstInt 1) ("i", CstInt 10) (Mul (Var "x") (CstInt 2)))
204             (ValInt 1024),
205           --
206           evalTest
207             "For loop zero"
208             (ForLoop ("x", CstInt 42) ("i", CstInt 0) (Add (Var "x") (Var "x")))
209             (ValInt 42),
210           --
211           evalTest
212             "For loop negative"
213             (ForLoop ("x", CstInt 42) ("i", CstInt (-1)) (Add (Var "x") (Var "x")))
214             (ValInt 42),
215           --
216           evalTest
217             "For loop depend on i"
218             (ForLoop ("acc", CstInt 0) ("i", CstInt 101) (Add (Var "acc") (Var "i")))
219             (ValInt 5050),
220           --
221           evalTestFail
222             "For loop: p not in scope when evaluating the bound"
223             (ForLoop ("p", CstInt 100) ("i", Var "p") (Add (Var "p") (Var "i"))),
224           --
225           evalTestFail
226             "For loop noninteger"
227             (ForLoop ("x", CstInt 42) ("i", CstBool True) (Add (Var "x") (Var "x"))),
228           --
229           evalTest
230             "While loop"
231             (WhileLoop ("x", Tuple [CstInt 1, CstInt 10]) (If (Eql (Project (Var "x") 1) (CstInt 0)) (CstBool
                ↪  False) (CstBool True)) (Tuple [Mul (Project (Var "x") 0) (CstInt 2), Sub (Project (Var "x") 1)
                ↪  (CstInt 1)]))
232             (ValTuple [ValInt 1024, ValInt 0]),
233           --
234           evalTest
235             "While loop false"
236             (WhileLoop ("x", CstInt 42) (CstBool False) (Add (Var "x") (Var "x")))
237             (ValInt 42),
238           --
239           evalTestFail
240             "While loop nonbool"
241             (WhileLoop ("x", CstInt 42) (CstInt 0) (Add (Var "x") (Var "x")))
242         ]
243     ]
244   where
245     evalTest = pureEvalTest
246     evalTestFail = pureEvalTestFail
247
248 taskC :: TestTree
249 taskC =
250   testGroup
251     "Task C: parsing && and ||, InterpPure"
252     [ testGroup
253         "Parsing"
254         [ parserTest "x&&y&&z" $
255             BothOf (BothOf (Var "x") (Var "y")) (Var "z"),
256           parserTest "x||y||z" $
257             OneOf (OneOf (Var "x") (Var "y")) (Var "z"),
258           parserTest "x||y&&z" $
259             OneOf (Var "x") (BothOf (Var "y") (Var "z")),
260           parserTest "x+y&&y*z" $
261             BothOf (Add (Var "x") (Var "y")) (Mul (Var "y") (Var "z")),
262           parserTest "x+y||y*z" $
263             OneOf (Add (Var "x") (Var "y")) (Mul (Var "y") (Var "z"))
264         ],
265       testGroup
266         "Interpretation (pure)"
267         [ evalTest
268             "e1 && e2"
```

```
            (BothOf (CstInt 0) (CstInt 1))
            (ValTuple [ValInt 0, ValInt 1]),
          --
          evalTestFail
            "e1 && e2 (first fails)"
            (BothOf (KvGet (CstInt 0)) (CstInt 1)),
          --
          evalTestFail
            "e1 && e2 (second fails)"
            (BothOf (KvGet (CstInt 0)) (CstInt 1)),
          --
          evalTest
            "e1 || e2"
            (OneOf (CstInt 0) (CstInt 1))
            (ValInt 0),
          --
          evalTest
            "e1 || e2 (first fails)"
            (OneOf (KvGet (CstInt 0)) (CstInt 1))
            (ValInt 1),
          --
          evalTestFail
            "e1 || e2 (both fails)"
            (OneOf (KvGet (CstInt 0)) (KvGet (CstInt 0))),
          --
          -- Strictly speaking we don't require any particular error
          -- messages, but the division-by-zero one is defined in the
          -- handout, so hopefully it is stable.
          testCase "e1 || e2 (both fails, check error is second)" $
            Pure.runEval
              (eval (OneOf (KvGet (CstInt 0)) (Div (CstInt 0) (CstInt 0))))
              @?= Left "Division by zero"
        ]
    ]
  where
    evalTest = pureEvalTest
    evalTestFail = pureEvalTestFail

-- Concurrent expressions.
expectingVal :: [(String, Exp, Val)]
expectingVal =
  [ ( "Infinite and finite loop",
      (OneOf infiniteLoop (BothOf (CstInt 0) finiteLoop)),
      (ValTuple [ValInt 0, ValInt 1024])
    ),
    ( "Finite and infinite loop",
      (OneOf (BothOf (CstInt 0) finiteLoop) infiniteLoop),
      (ValTuple [ValInt 0, ValInt 1024])
    ),
    ( "Concurrent put/get",
      BothOf
        (KvGet (CstInt 42))
        (KvPut (CstInt 42) finiteLoop),
      (ValTuple [ValInt 1024, ValInt 1024])
    ),
    -- Note that it is OK for students to pick the error if they
    -- explicitly state so in their report.
    ( "OneOf error left",
      (OneOf (Div (CstInt 42) (CstInt 0)) finiteLoop),
      (ValInt 1024)
    ),
    ( "OneOf error right",
      (OneOf finiteLoop (Div (CstInt 42) (CstInt 0))),
      (ValInt 1024)
    ),
    ( "client/server (may timeout if eval of for-loops doesn't bind the iterator in the body)",
      Let
        "requestTok"
```

```
337         (Lambda "request" (Var "request"))
338         ( Let
339             "respondTok"
340             (Lambda "respond" (Var "respond"))
341             ( Let
342                 "client"
343                 (Lambda "ignore" (ForLoop ("acc", CstInt 1) ("i", CstInt 10) (Let "ignore" (KvPut (Tuple [Var
                    ↪  "requestTok", Var "i"]) (Var "acc")) (Add (Var "acc") (KvGet (Tuple [Var "respondTok", Var
                    ↪  "i"])))))))
344                 ( Let
345                     "server"
346                     (Lambda "ignore" (WhileLoop ("clock", CstInt 0) (CstBool True) (Let "r" (KvGet (Tuple [Var
                        ↪  "requestTok", Var "clock"])) (Let "ignore" (KvPut (Tuple [Var "respondTok", Var "clock"])
                        ↪  (Mul (Var "r") (CstInt 2))) (Add (Var "clock") (CstInt 1))))))
347                     (OneOf (Apply (Var "client") (CstBool True)) (Apply (Var "server") (CstBool True)))
348                 )
349             )
350         ),
351       ValInt 59049
352     ),
353     ( "OneOf <failure> <long computation> (warning: may time out on slow impls!)",
354       OneOf
355         (CstInt 1 `Div` CstInt 0)
356         ( ForLoop
357             ("x", CstInt 0)
358             ("i", CstInt 999999)
359             (Var "x" `Add` CstInt 1)
360         ),
361       ValInt 999999
362     ),
363     ( "OneOf <failure> <medium computation>",
364       OneOf
365         (CstInt 1 `Div` CstInt 0)
366         ( ForLoop
367             ("x", CstInt 0)
368             ("i", CstInt 9999)
369             (Var "x" `Add` CstInt 1)
370         ),
371       ValInt 9999
372     ),
373     ( "OneOf <failure> <short computation>",
374       OneOf
375         (CstInt 1 `Div` CstInt 0)
376         ( ForLoop
377             ("x", CstInt 0)
378             ("i", CstInt 99)
379             (Var "x" `Add` CstInt 1)
380         ),
381       ValInt 99
382     ),
383     ( "Blocking get",
384       OneOf
385         (KvGet (CstInt 0))
386         ( ForLoop
387             ("x", CstInt 0)
388             ("i", CstInt 3)
389             ( If
390                 (Eql (Var "i") (CstInt 1))
391                 (KvPut (CstInt 0) (CstInt 1))
392                 (If (Eql (Var "i") (CstInt 2)) (Div (CstInt 0) (CstInt 0)) (CstInt (-1)))
393             )
394         ),
395       ValInt 1
396     )
397   ]
398
399 -- Failing concurrent expressions.
```

```
400 expectingError :: [(String, Exp)]
401 expectingError =
402   [ ( "Error",
403       Div (CstInt 42) (CstInt 0)
404     ),
405     ( "BothOf error",
406       BothOf (CstInt 0) (Div (CstInt 42) (CstInt 0))
407     ),
408     ( "OneOf <failure> <failure>",
409       OneOf
410         (CstInt 1 `Div` CstInt 0)
411         (CstInt 1 `Div` CstInt 0)
412     )
413   ]
414
415 taskD :: TestTree
416 taskD =
417   testGroup
418     "Task D: InterpSim"
419     $ map (\(desc, e, val) -> simEvalTest desc e val) expectingVal
420       ++ map (\(desc, e) -> simEvalTestFail desc e) expectingError
421
422 taskE :: TestTree
423 taskE =
424   testGroup
425     "Task E: KVDB"
426     [ testCase "put 1 2 -> get" $ do
427         kvdb <- startKVDB
428         kvPut kvdb (1 :: Int) (2 :: Int)
429         x <- kvGet kvdb 1
430         x @?= 2,
431       testCase "put False True -> get" $ do
432         kvdb <- startKVDB
433         kvPut kvdb True False
434         x <- kvGet kvdb True
435         x @?= False,
436       testCase "put True False -> put True True -> get" $ do
437         kvdb <- startKVDB
438         kvPut kvdb True False
439         kvPut kvdb True True
440         x <- kvGet kvdb True
441         x @?= True,
442       testCase "blocking" $ do
443         kvdb <- startKVDB
444         c <- newChan
445         _ <- forkIO $ writeChan c =<< kvGet kvdb True
446         threadDelay 1000
447         kvPut kvdb True 42
448         x <- readChan c
449         x @?= (42 :: Int),
450       testCase "blocking (two threads)" $ do
451         kvdb <- startKVDB
452         c1 <- newChan
453         c2 <- newChan
454         _ <- forkIO $ writeChan c1 =<< kvGet kvdb True
455         _ <- forkIO $ writeChan c2 =<< kvGet kvdb True
456         threadDelay 1000
457         kvPut kvdb True 42
458         x <- readChan c1
459         x @?= (42 :: Int)
460         y <- readChan c2
461         y @?= (42 :: Int)
462     ]
463
464 taskF :: TestTree
465 taskF =
466   testGroup
467     "Task F: InterpConcurrent"
```

```
    $ map (\(desc, e, val) -> concurrentEvalTest desc e val) expectingVal
      ++ map (\(desc, e) -> concurrentEvalTestFail desc e) expectingError

-- Parser tests related to the original language, just to check that
-- they haven't ruined the parser somehow.
oldParserTests :: TestTree
oldParserTests =
  testGroup
    "Old parser tests"
    [ testGroup
        "Constants"
        [ parserTest "123" $ CstInt 123,
          parserTest " 123" $ CstInt 123,
          parserTest "123 " $ CstInt 123,
          parserTestFail "123f",
          parserTest "true" $ CstBool True,
          parserTest "false" $ CstBool False
        ],
      testGroup
        "Basic operators"
        [ parserTest "x+y" $ Add (Var "x") (Var "y"),
          parserTest "x-y" $ Sub (Var "x") (Var "y"),
          parserTest "x*y" $ Mul (Var "x") (Var "y"),
          parserTest "x/y" $ Div (Var "x") (Var "y"),
          parserTest "x==y" $ Eql (Var "x") (Var "y")
        ],
      testGroup
        "Operator priority"
        [ parserTest "x+y+z" $ Add (Add (Var "x") (Var "y")) (Var "z"),
          parserTest "x+y-z" $ Sub (Add (Var "x") (Var "y")) (Var "z"),
          parserTest "x+y*z" $ Add (Var "x") (Mul (Var "y") (Var "z")),
          parserTest "x*y*z" $ Mul (Mul (Var "x") (Var "y")) (Var "z"),
          parserTest "x/y/z" $ Div (Div (Var "x") (Var "y")) (Var "z"),
          parserTest "x+y==y+x" $ Eql (Add (Var "x") (Var "y")) (Add (Var "y") (Var "x"))
        ],
      testGroup
        "Function application"
        [ parserTest "x y z" $
            Apply (Apply (Var "x") (Var "y")) (Var "z"),
          parserTest "x(y z)" $
            Apply (Var "x") (Apply (Var "y") (Var "z")),
          parserTestFail "x let v = 2 in v",
          parserTestFail "x \\x -> x",
          parserTestFail "x if x then y else z",
          parserTest "get x x" $
            Apply (KvGet (Var "x")) (Var "x"),
          parserTest "(get x) x" $
            Apply (KvGet (Var "x")) (Var "x")
        ],
      testGroup
        "Let-binding"
        [ parserTest "let x = y in z" $
            Let "x" (Var "y") (Var "z"),
          parserTest "let x = y+2 in z z" $
            Let "x" (Add (Var "y") (CstInt 2)) (Apply (Var "z") (Var "z")),
          parserTestFail "let true = y in z"
        ],
      testGroup
        "Conditional expressions"
        [ parserTest "if x then y else z" $ If (Var "x") (Var "y") (Var "z"),
          parserTest "if x then y else if x then y else z" $
            If (Var "x") (Var "y") $
              If (Var "x") (Var "y") (Var "z"),
          parserTest "if x then (if x then y else z) else z" $
            If (Var "x") (If (Var "x") (Var "y") (Var "z")) (Var "z"),
          parserTest "1 + if x then y else z" $
            Add (CstInt 1) (If (Var "x") (Var "y") (Var "z"))
        ],
```

```
536       testGroup
537         "Anonymous functions"
538         [ parserTest "\\x -> 2" $ Lambda "x" (CstInt 2),
539           parserTest "\\x -> \\y -> x+y" $
540             Lambda "x" $
541               Lambda "y" $
542                 Add (Var "x") (Var "y"),
543           parserTest "1 + \\x -> 2" $ Add (CstInt 1) (Lambda "x" (CstInt 2))
544         ],
545       testGroup
546         "Lexing edge cases"
547         [ parserTest "iff x thenn y elsee z" $
548             foldl Apply (Var "iff") [Var "x", Var "thenn", Var "y", Var "elsee", Var "z"],
549           parserTest "2 " $ CstInt 2,
550           parserTest " 2" $ CstInt 2,
551           parserTest "getx" $ Var "getx",
552           parserTest "putx" $ Var "putx",
553           parserTestFail "2x"
554         ],
555       testGroup
556         "put/get"
557         [ parserTest "put x y" $ KvPut (Var "x") (Var "y"),
558           parserTest "get x" $ KvGet (Var "x"),
559           parserTest "get x + y" $ Add (KvGet (Var "x")) (Var "y"),
560           parserTest "y + get x" $ Add (Var "y") (KvGet (Var "x"))
561         ]
562     ]
563
564 tests :: TestTree
565 tests =
566   testGroup
567     "Tests"
568     [ taskA,
569       taskB,
570       taskC,
571       taskD,
572       taskE,
573       taskF,
574       oldParserTests
575     ]
```